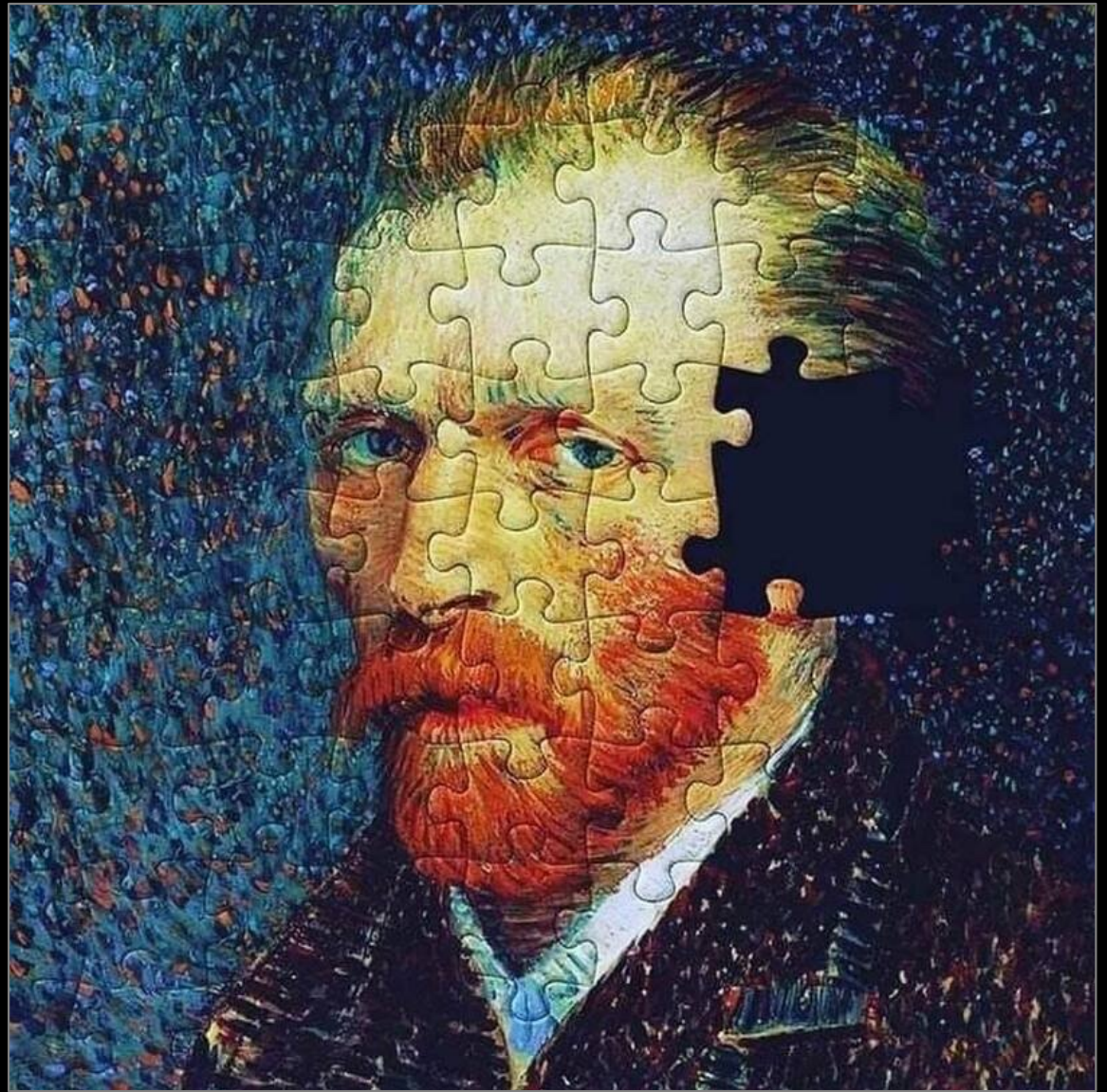


Dezvoltarea aplicațiilor Web

**aplicații Web:
aspecte
arhitecturale**



Prof. Sabin Corneliu Buraga
profs.info.uaic.ro/sabin.buraga/

„Fiecare vis începe cu un visător.”

Harriet Tubman



Care sunt arhitecturile software tipice
pe baza cărora sunt dezvoltate
aplicațiile Web de anvergură?

arhitecturi

Calitatea aplicațiilor Web este influențată
de arhitectura pe care se bazează

architecture

Web application architecture

body of code

that's seen by developers as a single unit

group of functionality

that business customers see as a single unit

initiative

that those with the money see as a single budget

arhitecturi: principii

start with needs

do less

design with data

do the hard work to make it simple

iterate. then iterate again

build for inclusion

understand context

build digital services, not Websites

be consistent, not uniform

make things open; it makes things better

exemplu pentru [gov.uk](https://www.gov.uk) – conform Paul Downey & David Heath

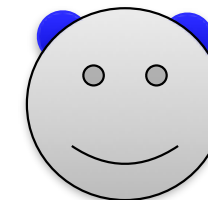
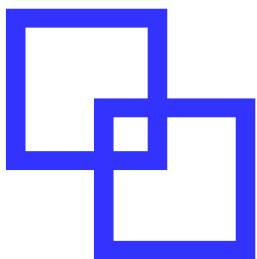
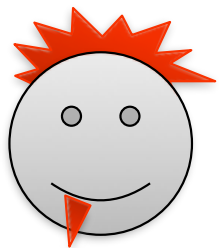
scopuri
psihologie
comportament

interacțiune
controale
limbi naturale

funcționalități
tehnologii
algoritmi

indexare
structurare
meta-date

instrumente
metodologii
stimuli



utilizatori

interfață

software

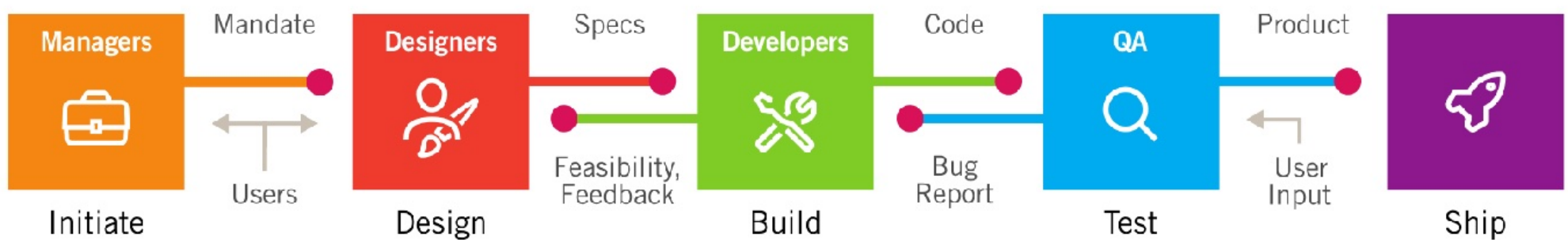
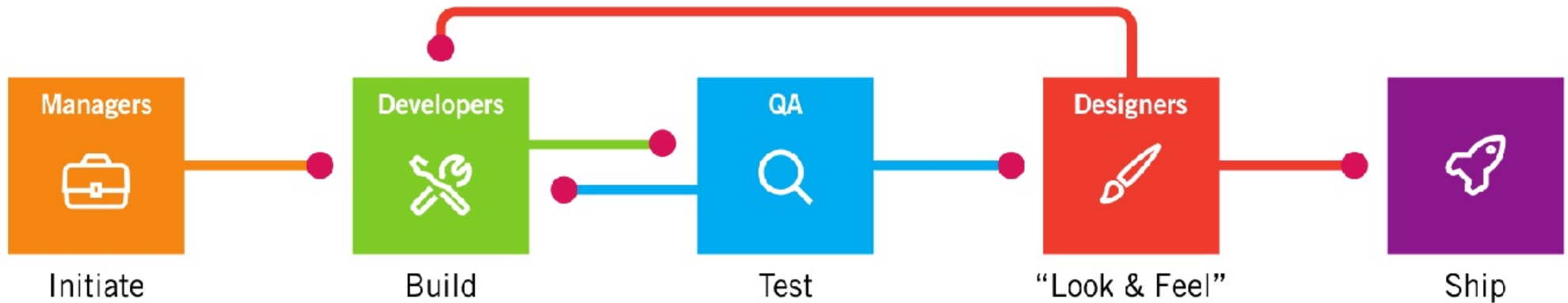
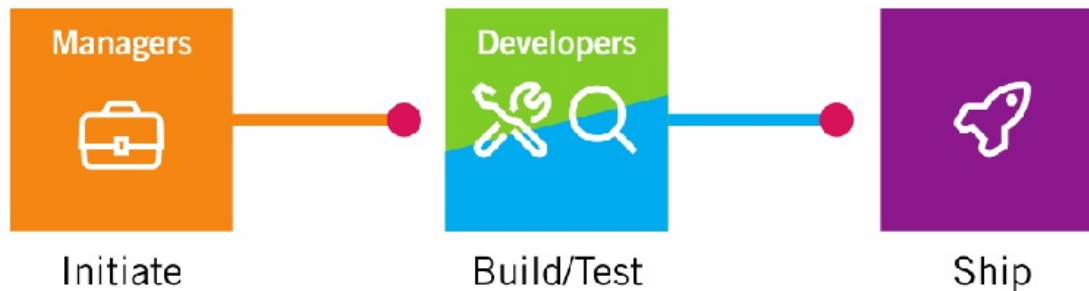
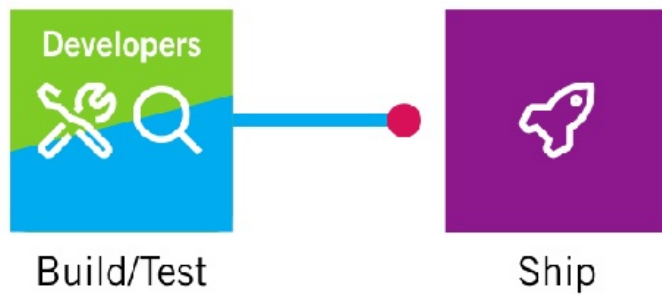
conținut

creatori

adaptare după Crumlish & Malone (2009)

evoluția manierei de dezvoltare a produselor digitale (software)

Alan Cooper *et al.* (2014)



client(i)

mandatar (*proxy*)

zid de protecție (*firewall*)

intermediar(i) (*middleware*)

server(e) Web

server(e) de aplicații Web

cadre de lucru, biblioteci, alte componente

server(e) de stocare persistentă – *e.g.*, baze de date

server(e) de conținut multimedia

server(e) de management al conținutului – *e.g.*, CMS, *wiki*

aplicații/sisteme tradiționale (*legacy*)

„ingrediente” tipice

client(i)

mandatar (*proxy*)

zid de protecție (*firewall*)

intermediar(i) (*middleware*)

server(e) Web

server(e) de aplicații Web

cadre de lucru, biblioteci, alte componente

server(e) de stocare persistentă – *e.g.*, baze de date

server(e) de conținut multimedia

server(e) de management al conținutului – *e.g.*, CMS, *wiki*

aplicații/sisteme tradiționale (*legacy*)

eventual, recurgând la servicii în „nori” – *cloud computing*

partajarea la cerere a resurselor de calcul și a datelor cu alte calculatoare/dispozitive pe baza tehnologiilor Internet (găzduire, infrastructură scalabilă, procesare paralelă, monitorizare,...)

Esențialmente, de considerat:

preluarea și dirijarea cererilor – *dispatch*

oferirea funcționalităților de bază – *core services*

asocierea dintre construcții/abstracțiuni software
(*e.g.*, obiecte) și modele de date – *mapping*

managementul datelor – *data*

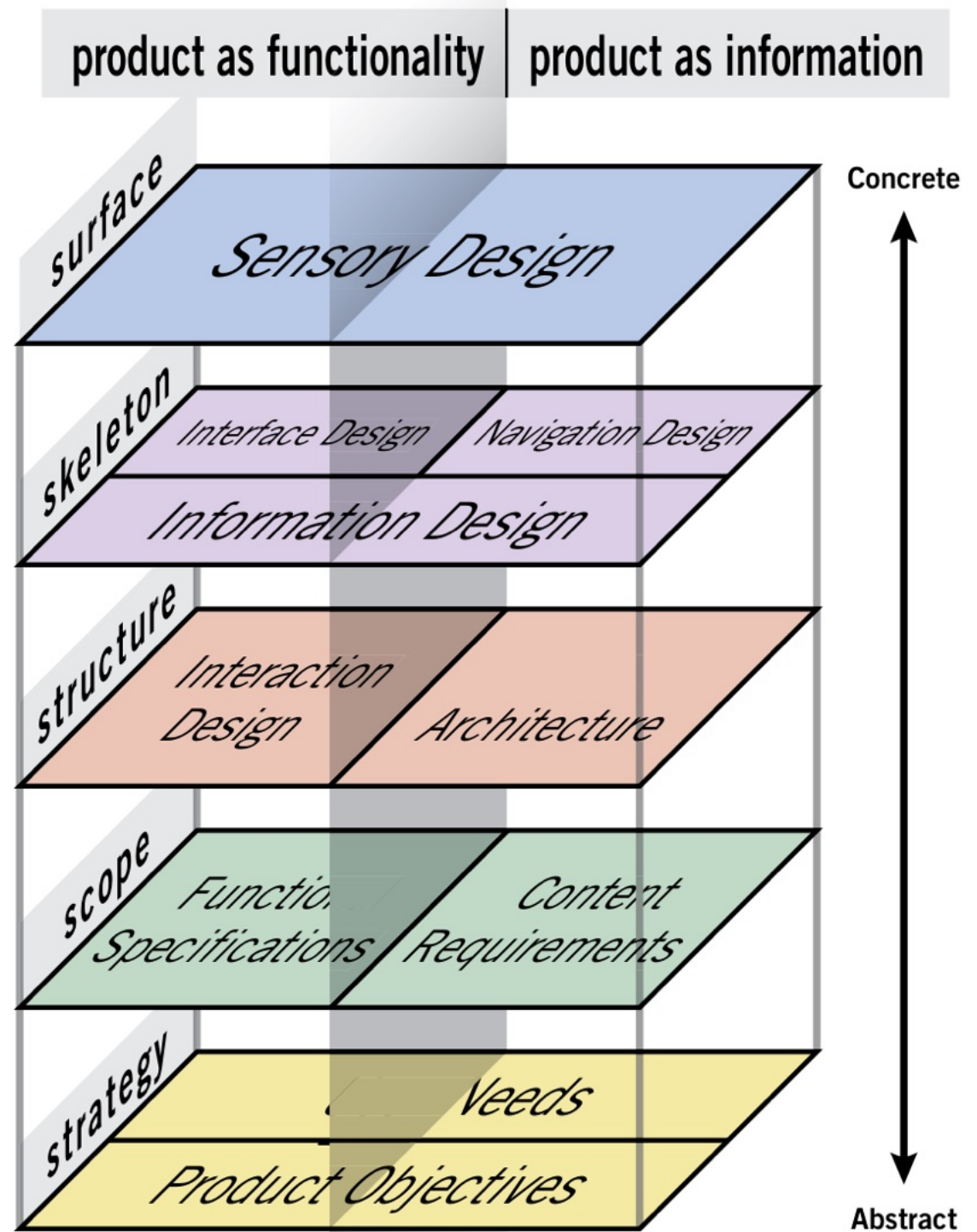
monitorizarea și evaluarea sistemului – *metrics*

adaptare după Matt Ranney, “*What I Wish I Had Known
Before Scaling Uber to 1000 Services*”, GOTO Chicago (2016)

highscalability.com/blog/2016/10/12/lessons-learned-from-scaling-uber-to-2000-engineers-1000-ser.html

aplicație Web
(produs software)

funcționalitate
+
informații oferite



arhitecturi web

Stratificate (*layered*)

Conduse de evenimente (*event-driven*)

Extensibile (*microkernel / plug-in*)

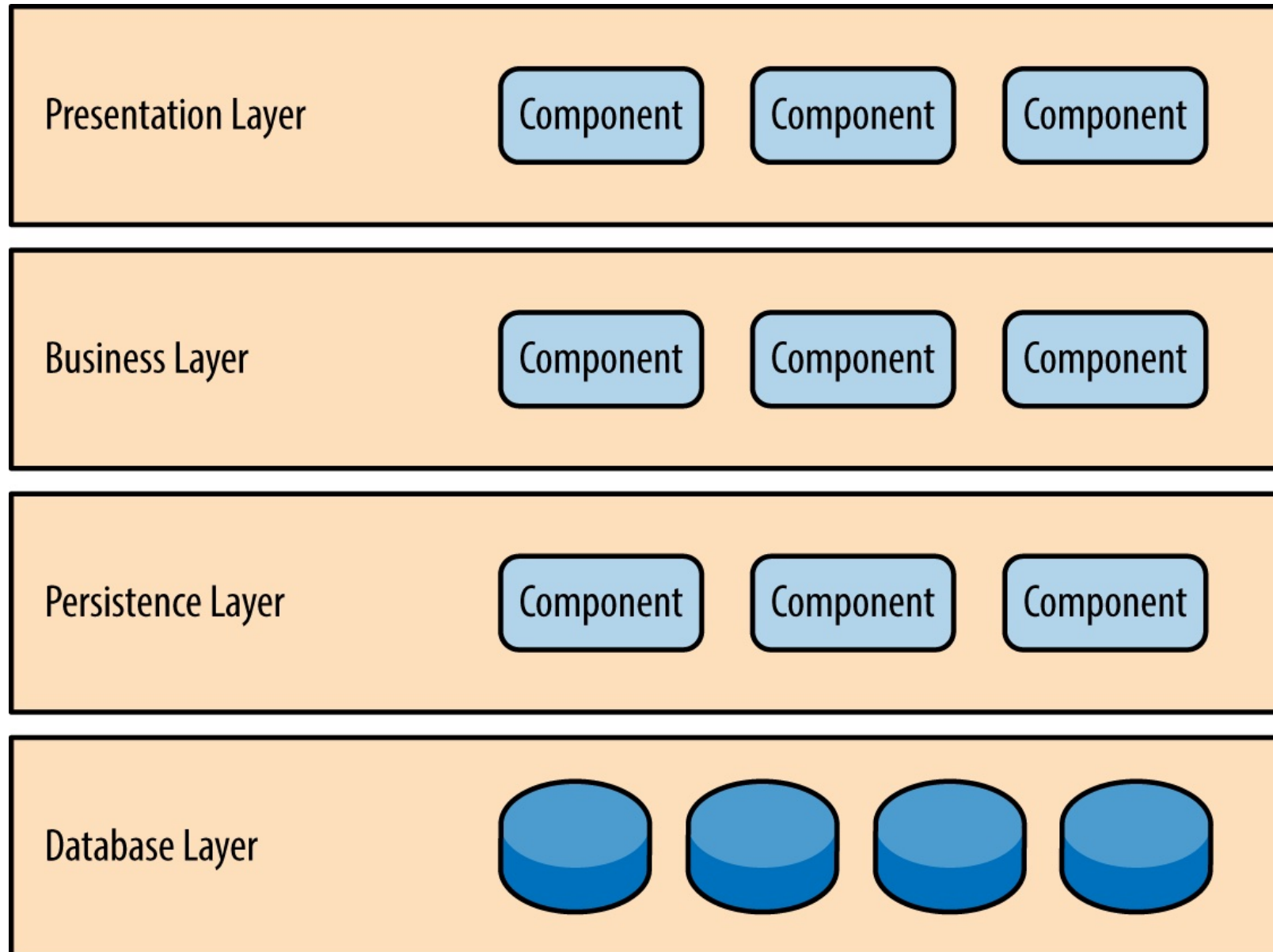
Folosind micro-servicii (*microservices*)

„În nori” (*space-based, cloud*)

conform M. Richards, *Software Architecture Patterns*, O'Reilly (2015)

Stratificate (*layered*)

N-tier architecture – abordare de facto



Stratificate (*layered*)

demarcarea responsabilităților (*separation of concerns*)

fiecare strat are un rol bine-stabilit, componentele
unui strat vizând funcționalitățile acestuia

Stratificate (*layered*)

izolare (*layers of isolation*)

modificările operate la un anumit strat nu au impact
sau nu afectează componentele din alt strat

Stratificate (*layered*)

architecture sinkhole anti-pattern

fluxul de cereri traversează fiecare strat, fără a se efectua procesări semnificative în cadrul acestuia

arhitecturi web

Conduse de evenimente (*event-driven*)
uzual în contextul aplicațiilor distribuite asincrone
▶ scalabilitate

topologii principale:

mediator

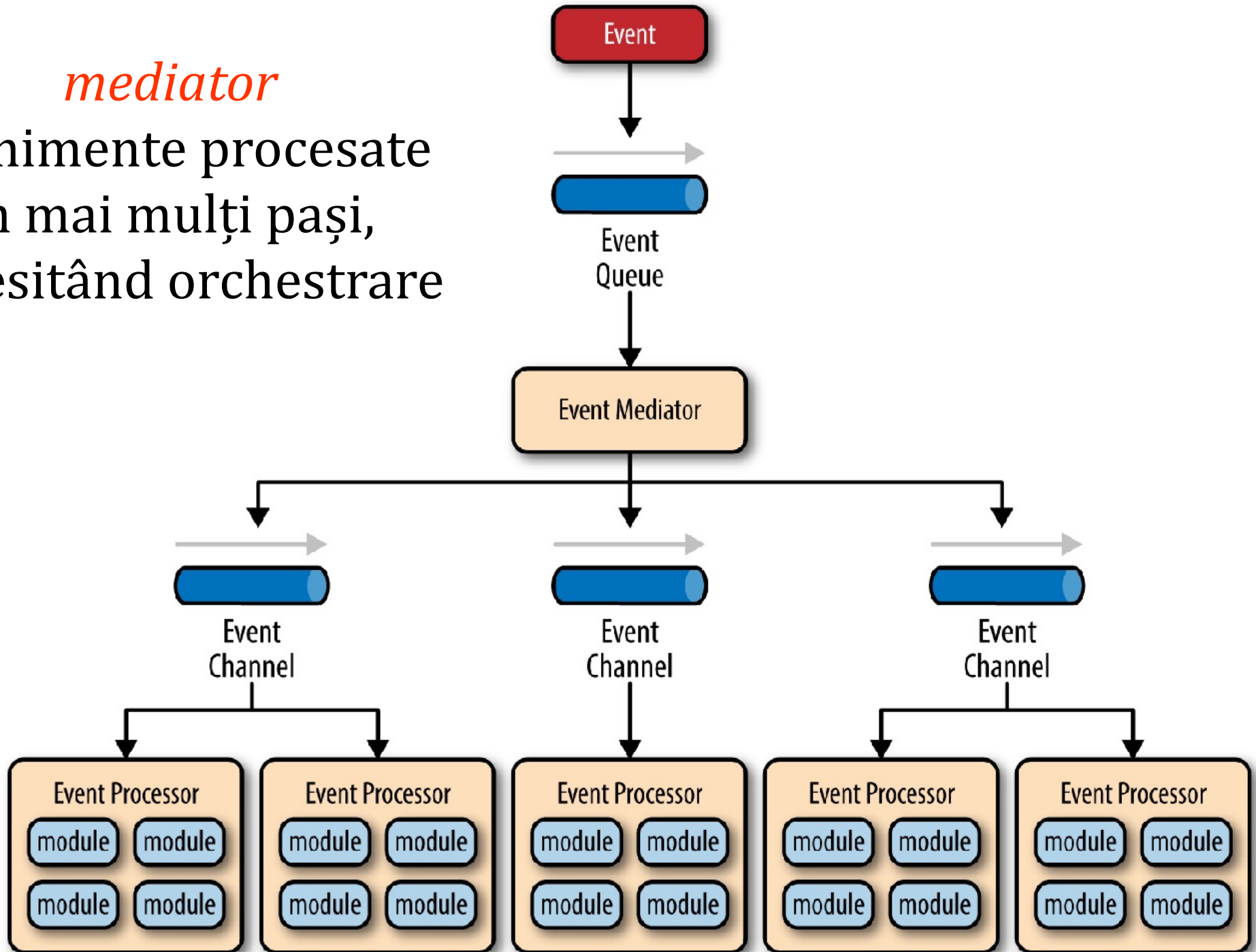
sau

broker

Conduse de evenimente (*event-driven*)

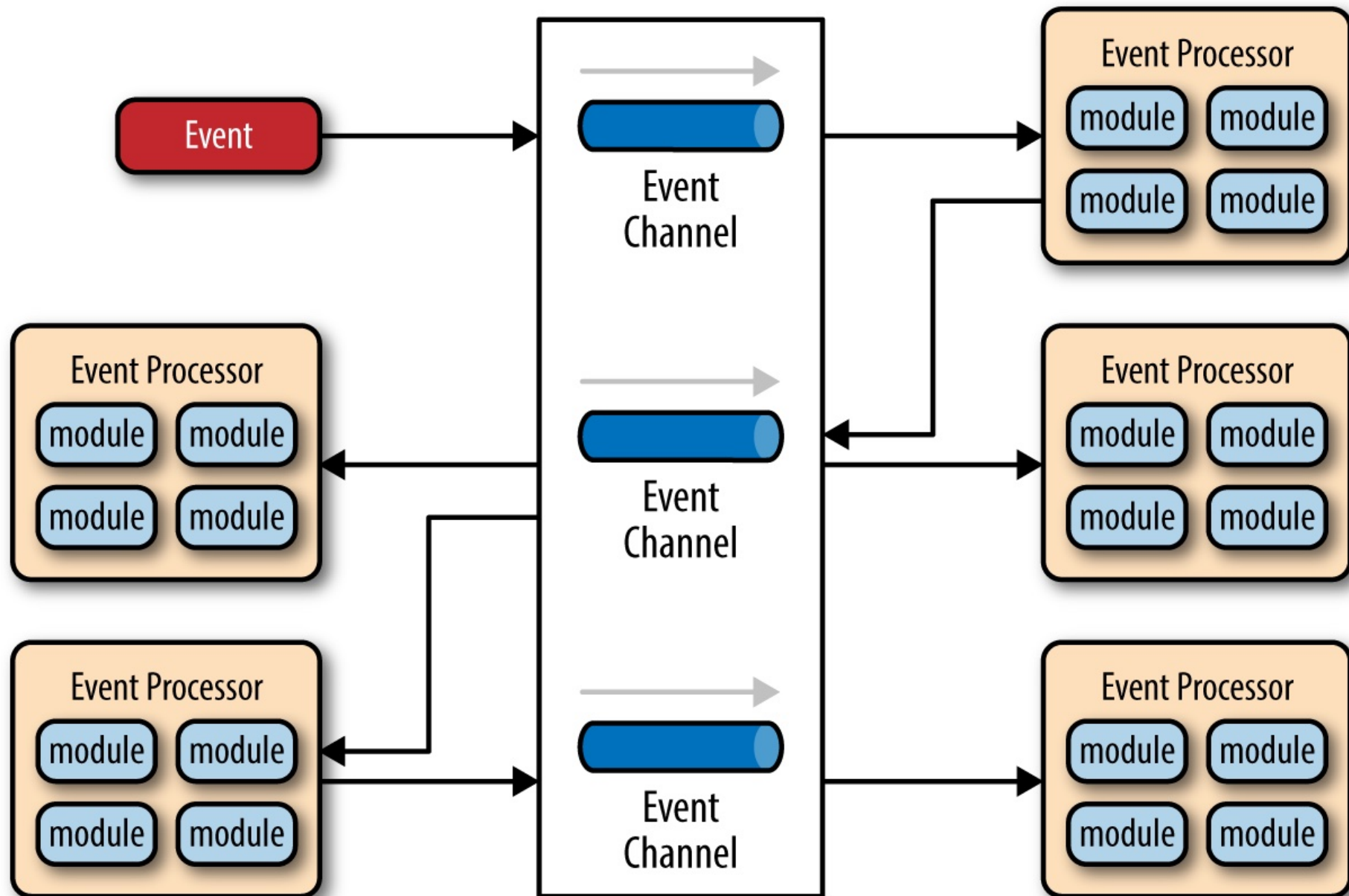
mediator

evenimente procesate
în mai mulți pași,
necesitând orchestrare



Conduire de evenimente (*event-driven*)

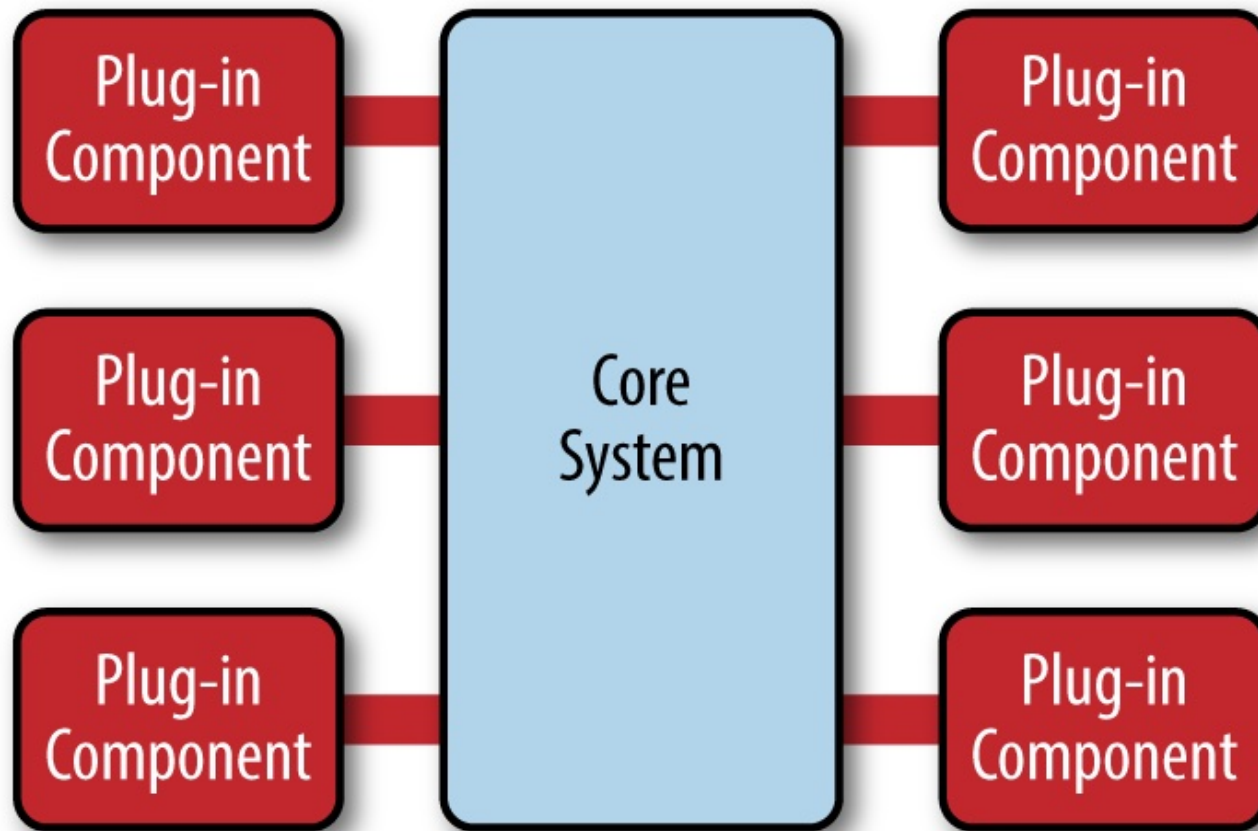
broker – fluxul de mesaj este distribuit componentelor de procesare a evenimentelor



Extensibile (*microkernel / plug-in*)
sistem principal (*core system*)

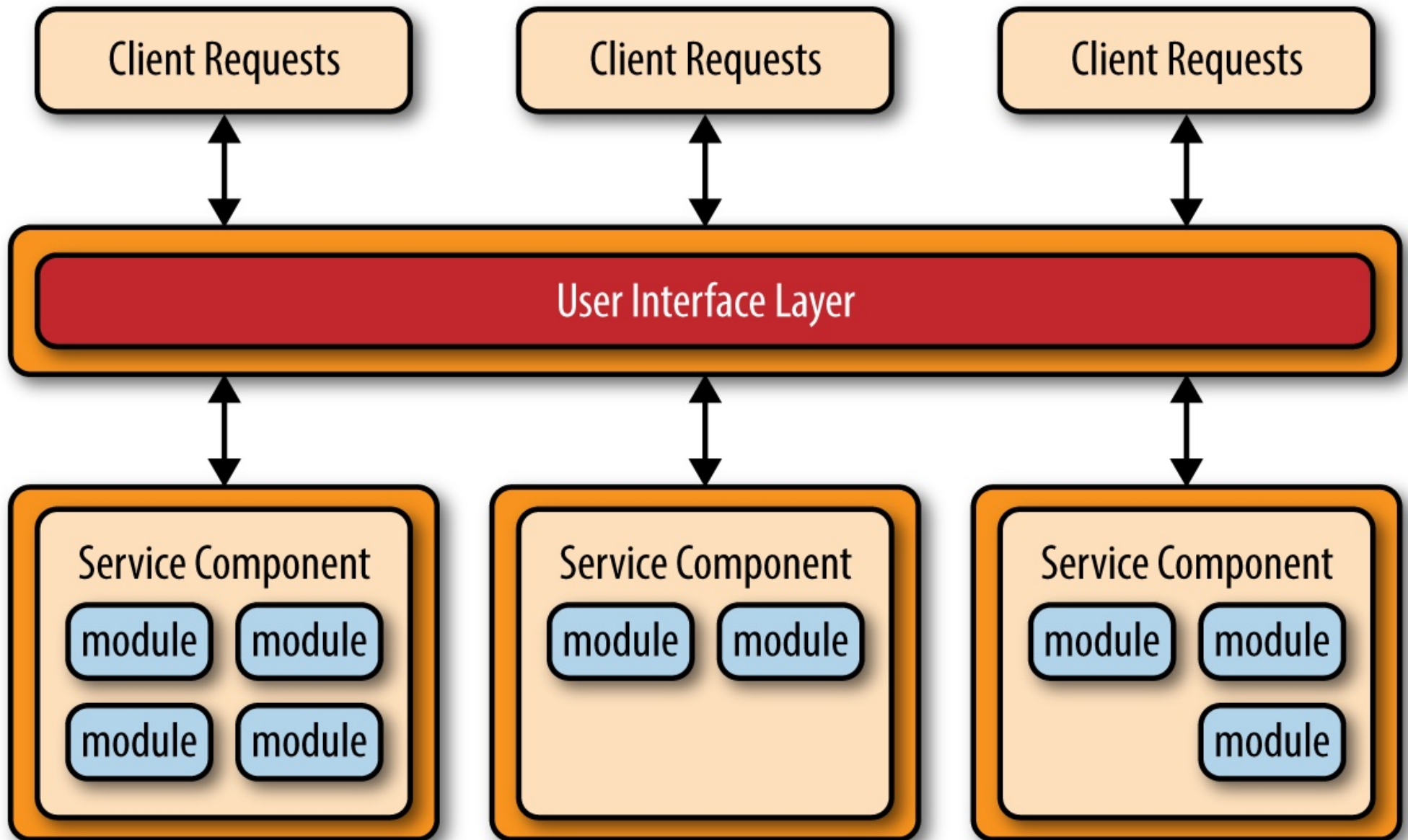
+

module independente de tip extensie (*plug-in*)



o astfel de arhitectură poate fi inclusă/utilizată
ca parte a altei abordări arhitecturale

Folosind micro-servicii
componente separate, distribuite
(*separately deployed units*) ▶ decuplare maximă



arhitecturi web

Folosind micro-servicii

abordări:

bazate pe API-uri proprii (*API-based*)

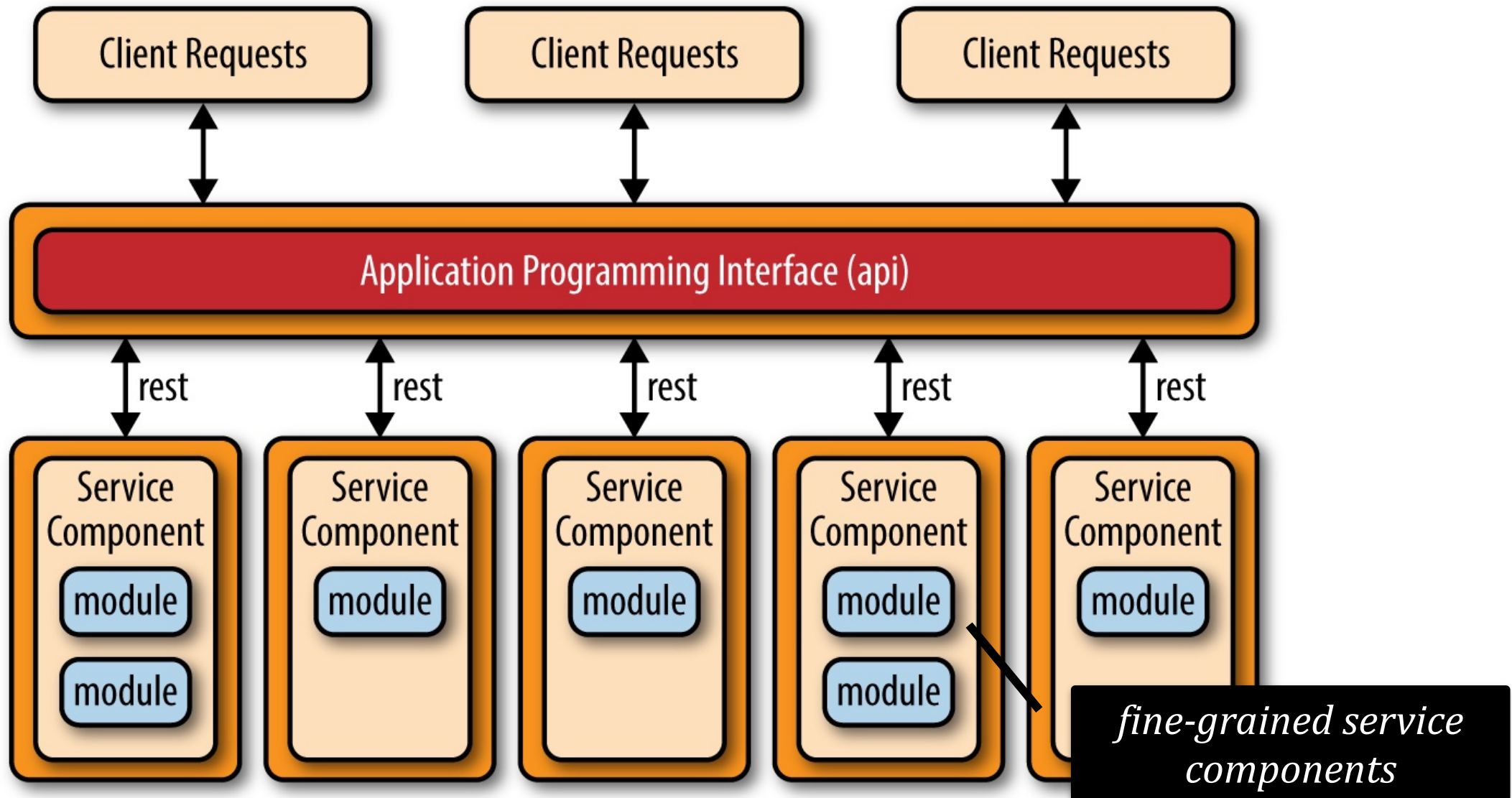
aplicație recurgând la REST (*application REST-based*)

mesagerie centralizată (*centralized messaging*)

Folosind micro-servicii

API-based

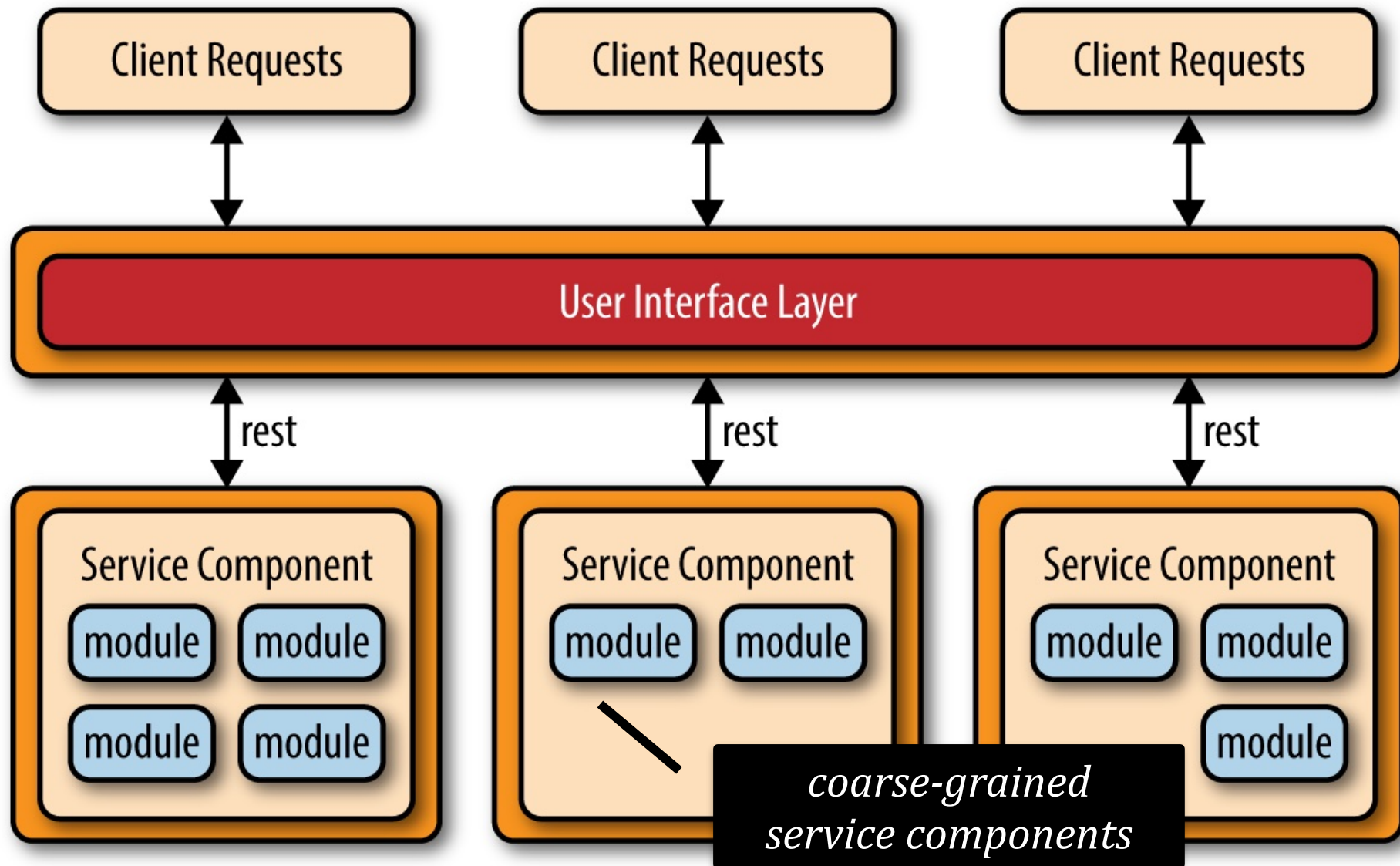
aplicația Web expune servicii individuale, punctuale, de sine-stătătoare (*self-contained*) via un API



Folosind micro-servicii

application REST-based

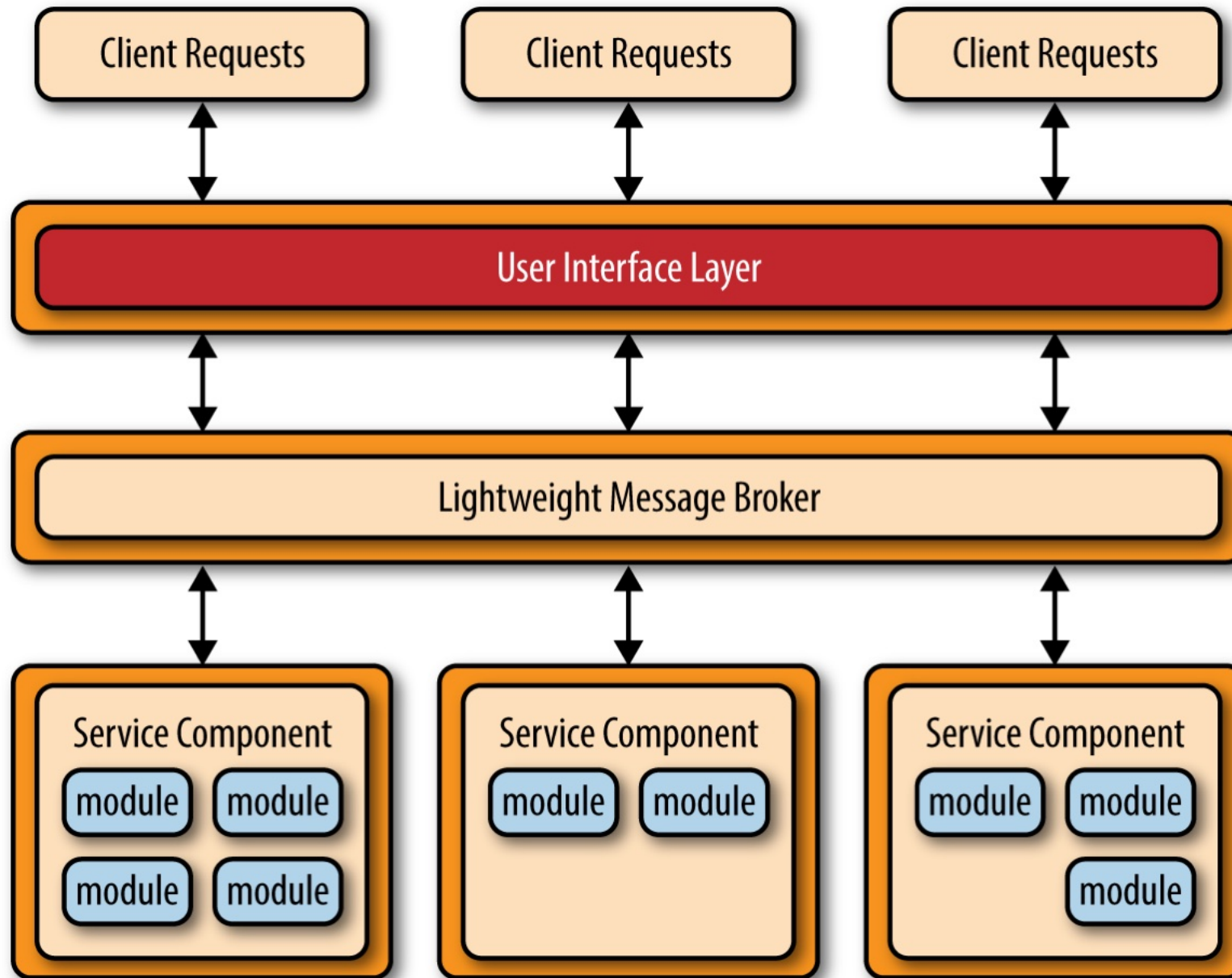
cererile sunt recepționate tradițional (nu prin API)
fiecare funcționalitate este accesată intern via REST



Folosind micro-servicii

centralized messaging

accesare a componentelor interne via un *broker* „ușor”



arhitecturi web

„În nori” (*space-based, cloud*)

consideră și rezolvă problemele vizând scalabilitatea și concurența unui volum impredictibil de mare de cereri

arhitecturi web

„În nori” (*space-based, cloud*)

consideră și rezolvă problemele vizând scalabilitatea și concurența unui volum impredictibil de mare de cereri

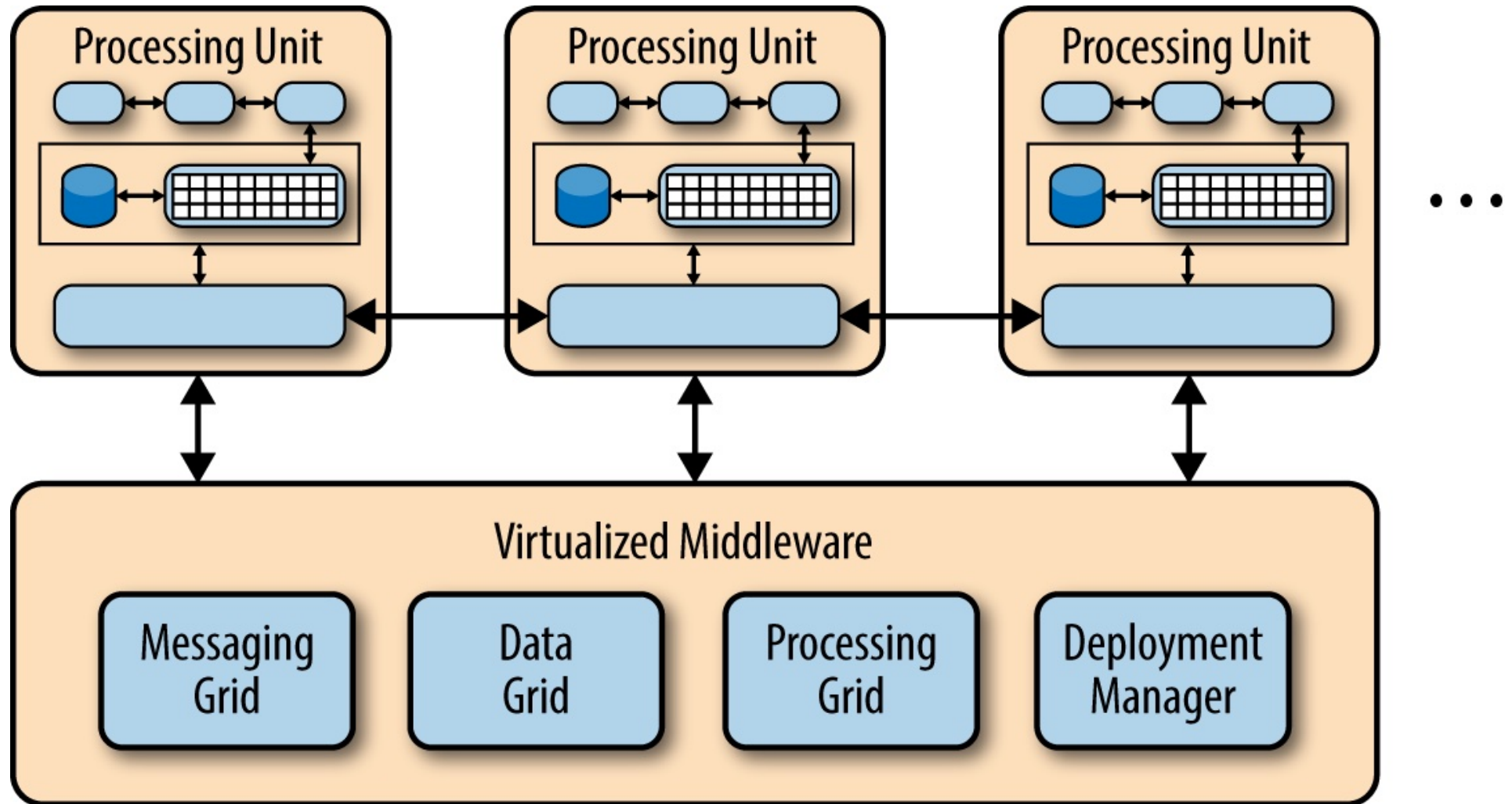
tuple space

datele aplicației sunt păstrate în memorie și replicate de toate unitățile de procesare active

fără stocare centralizată ► *distributed shared memory*

wiki.c2.com/?TupleSpace

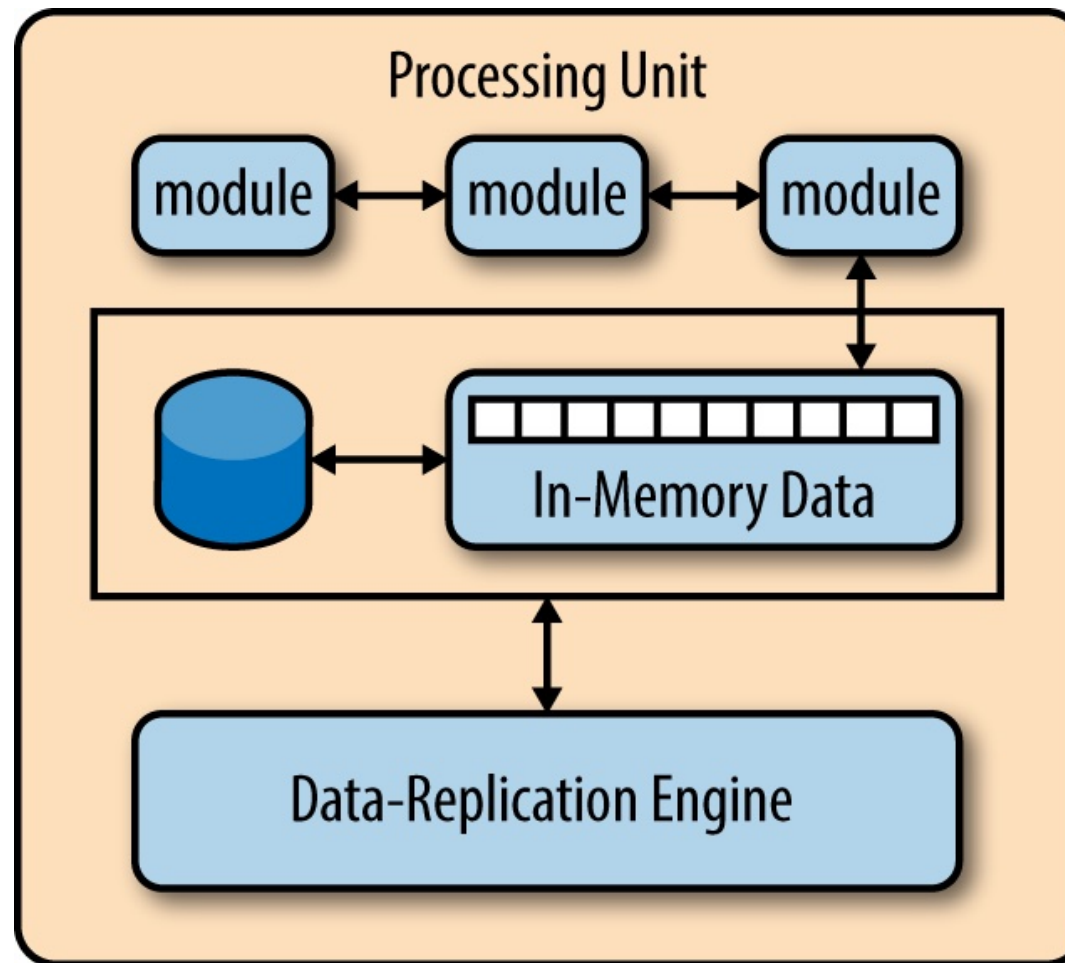
„În nori” (*space-based, cloud*)



middleware virtualizat

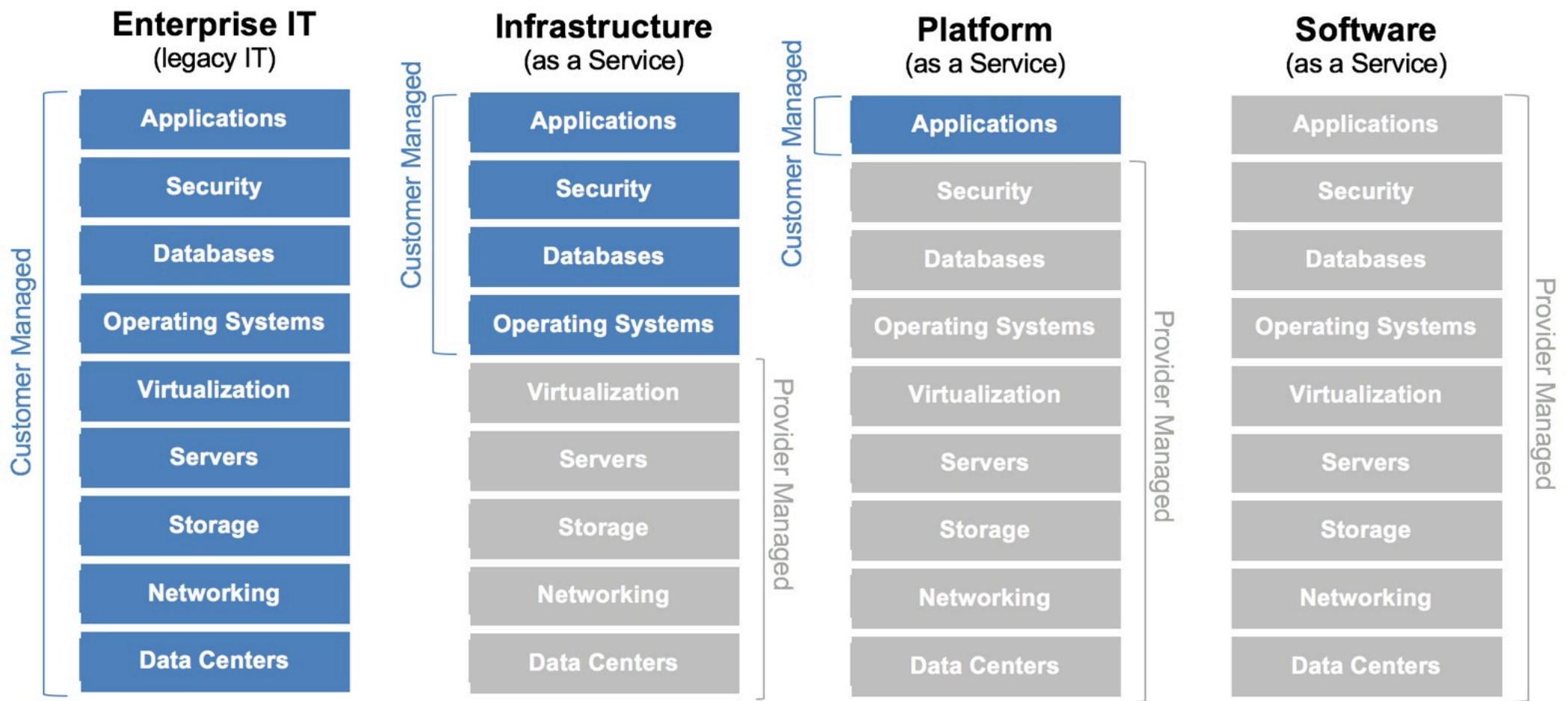
include componente controlând sincronizarea datelor,
procesarea cererilor,
accesul la platforma de execuție (*deployment*),...

„În nori” (*space-based, cloud*)



unitate de procesare

reprezentată de un (micro-)serviciu Web sau
o componentă software tradițională la nivel de *backend*



anumite funcționalități pot fi gestionate
„în propria ogradă” (*on-premises*)
sau de un furnizor de servicii disponibile „în nori”

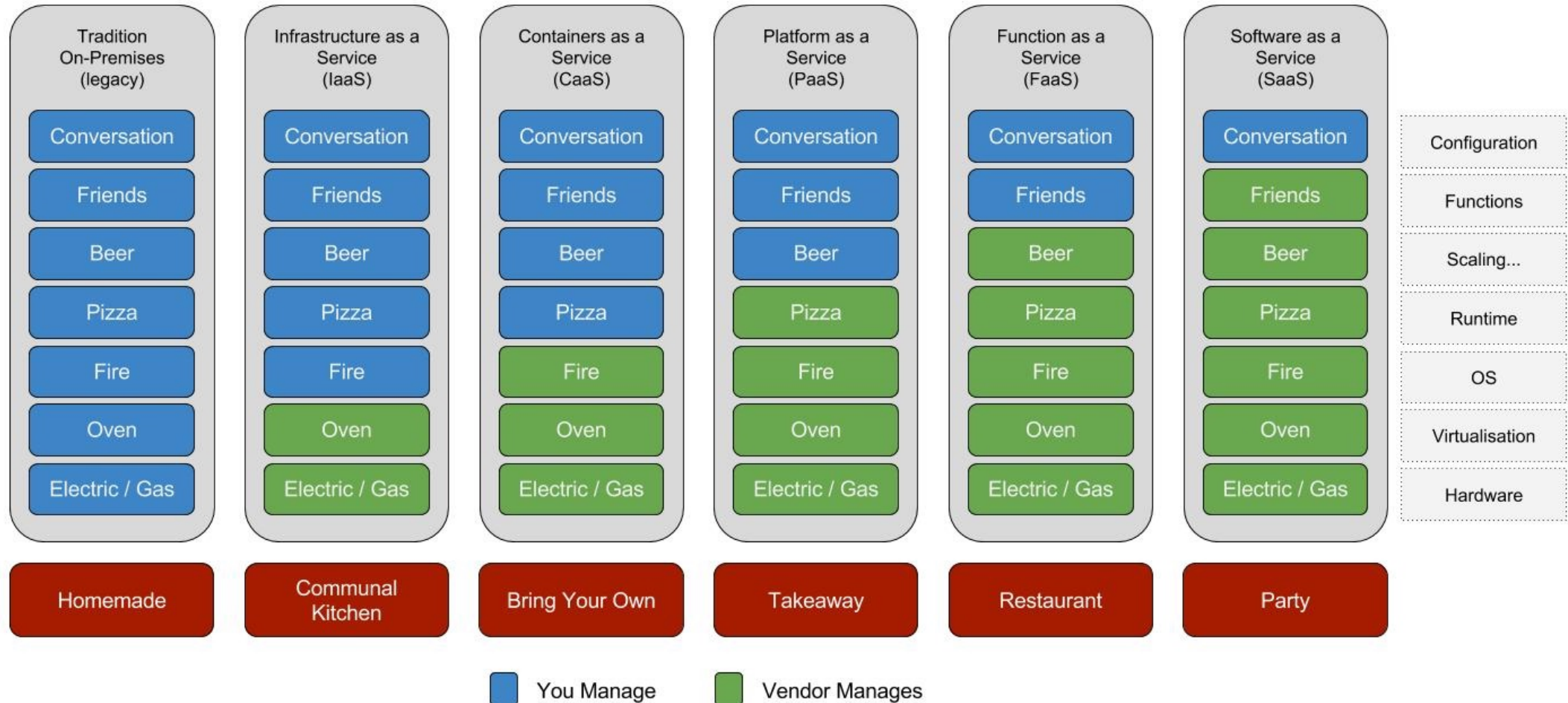
conform A. Eizadirad (2017)

www.linkedin.com/pulse/iaas-paas-saas-explained-compared-arsalan-eizadirad



Pizza as a Service 2.0

<http://www.paulkerrison.co.uk>



P. Kerrison, *Pizza As A Service 2.0* (2017)
www.paulkerrison.co.uk/random/pizza-as-a-service-2-0

Prin ce mijloace poate fi implementată o aplicație Web?



implementare

Server de aplicații Web

scop:

eficientizarea proceselor de dezvoltare
a aplicațiilor Web de anvergură

simplifică invocarea de programe (*script-uri*)
facilitând generarea de conținut (reprezentări de resurse)

implementare

Server de aplicații Web

poate fi integrat în unul/mai multe servere Web

de asemenea, poate oferi propriul server Web
sau mediu de execuție

de (re)parcurs prezentarea privind ingineria aplicațiilor Web:
edu.info.uaic.ro/web-technologies/web-film.html#week3

implementare

Server de aplicații Web

integrat într-o *stivă de tehnologii (software stack)*:
suită software (servere, instrumente, utilitare,...)
oferind suport pentru dezvoltarea de aplicații Web

disponibilă – uzual, în regim *open source* –
pentru o anumită platformă
(sistem de operare, server Web, server de baze de date,
server de aplicații, limbaj de programare)

implementare

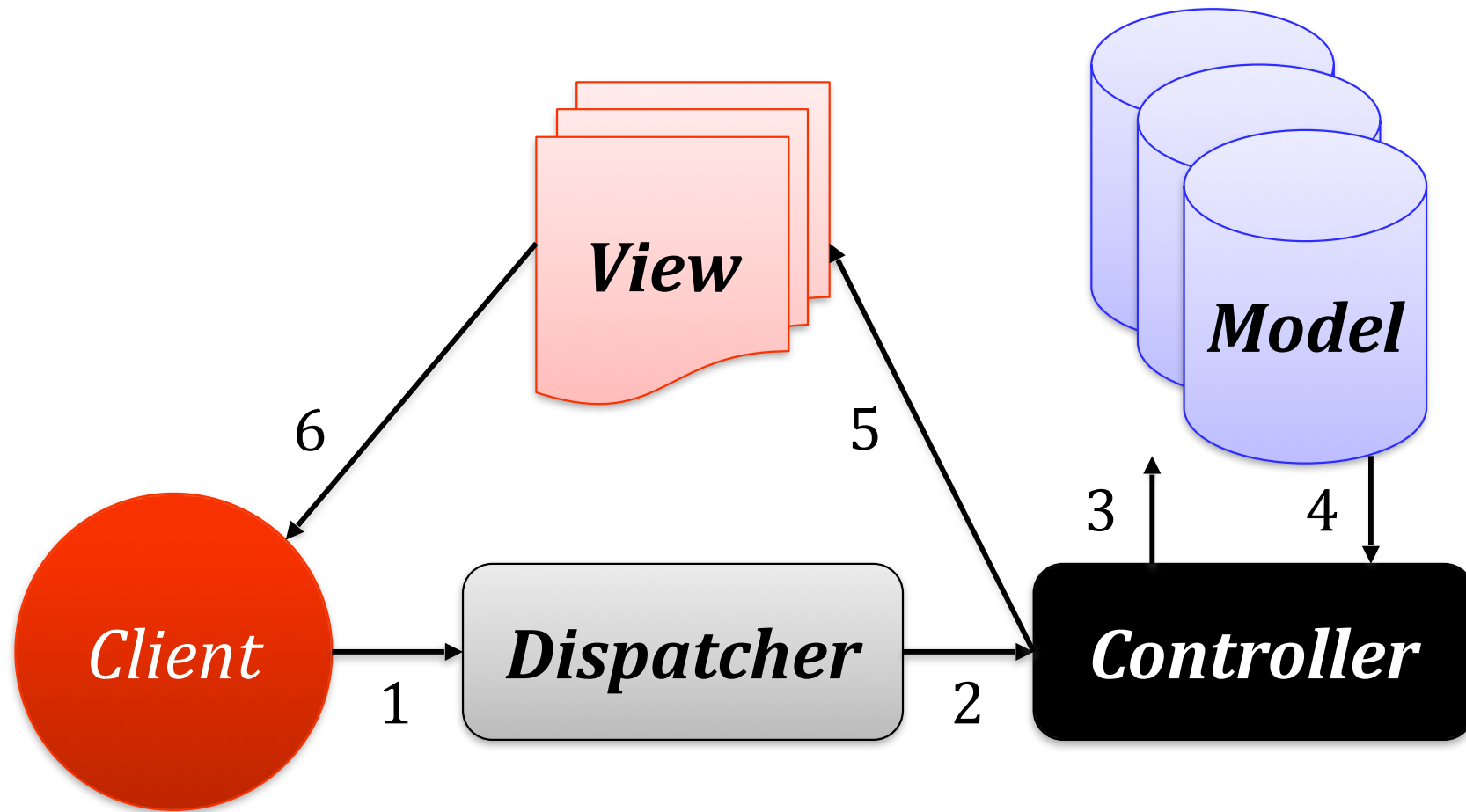
Server de aplicații Web

poate încuraja sau impune o viziune arhitecturală
privind dezvoltarea de aplicații Web

situație tipică:

MVC ori variații – Herberto Graca (2017)

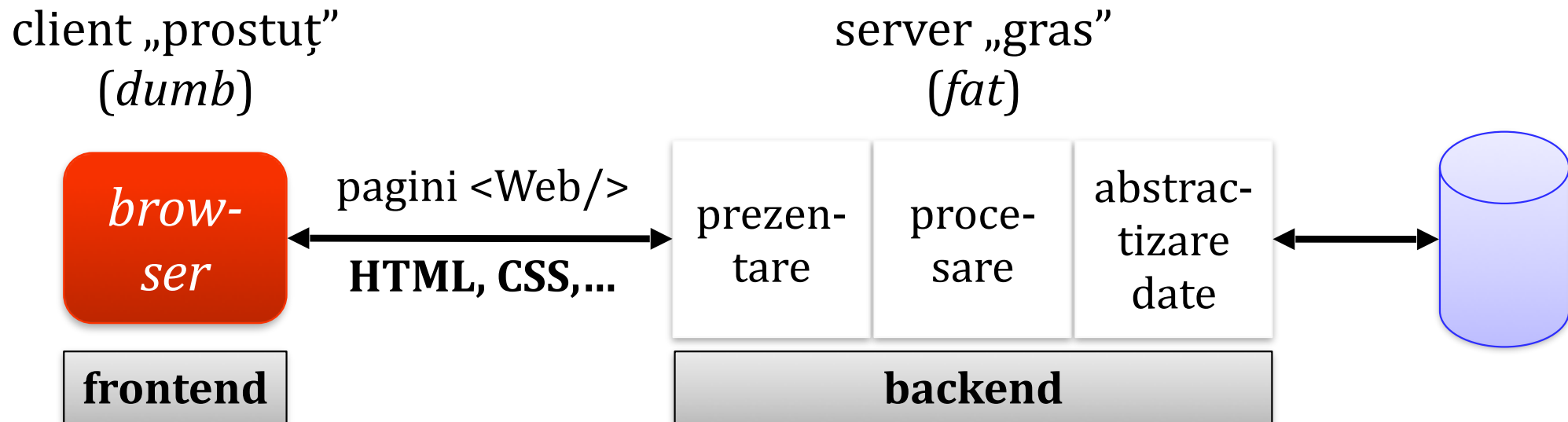
herbertograca.com/2017/08/17/mvc-and-its-variants/



etape tipice:

- (1) cerere trimisă de client – *e.g.*, navigator Web,
- (2) dirijare (*routing*) a cererii către *controller*,
- (3) recurgera la un *model*, (4) furnizare date dorite,
- (5) selectare a unui *view*, (6) transmite conținut la client

arhitectura aplicațiilor Web: abordarea MV* tradițională



frecvent, **aplicație monolitică**
(*e.g.*, un WAR: 2.2 M linii de cod, 418 .jar-uri,
startare în 12 min. – conform plainoldobjects.com)

implementare

Framework (cadru de lucru)

facilitează dezvoltarea de aplicații Web complexe,
simplificând unele operații uzuale
(*e.g.*, acces la baze de date, *caching*, generare de cod,
management de sesiuni, control al accesului)
și/sau încurajând reutilizarea codului-sursă

implementare

Diverse *framework*-uri care facilitează dezvoltarea de aplicații Web aliniate problematicilor Web-ului semantic:

Apache Jena (Java) – jena.apache.org

Comunica (TypeScript) – comunica.dev

Eclipse RDF4J (Java) – rdf4j.org

LODQA (JavaScript + Ruby) – github.com/lodqa/lodqa

RDFShape API (Scala) – www.weso.es/rdfshape-api/

Semantic MediaWiki (PHP) – www.semantic-mediawiki.org

SPARQL Anything (Java) – sparql-anything.cc

implementare

Biblioteca Web (*library*)

colecție de resurse computaționale reutilizabile
– *i.e.*, structuri de date + cod –
oferind funcționalități (comportamente) specifice
implementate într-un limbaj de programare

poate fi referită de alt cod-sursă (software):
server de aplicații, *framework*, bibliotecă,
serviciu, API ori componentă Web

Biblioteci cu acces liber la codul-sursă

exemple – aliniate problematicilor Web-ului semantic:

banana-rdf (Scala) – github.com/banana-rdf/banana-rdf

dotNetRDF (C#) – www.dotnetrdf.org

FaCT++ (C++) – bitbucket.org/dtsarkov/factplusplus

graphy (JavaScript) – graphy.link

OWL API (Java) – github.com/owlcs/owlapi

RDF2Go (Go) – github.com/deiu/rdf2go

RDFLib (Python) – rdflib.dev

Redland RDF Libraries (C) – librdf.org

Sgvizler (JavaScript) – mgskjaeveland.github.io/sgvizler/

sparql (Ruby) – rubygems.org/gems/sparql

implementare

Serviciu Web

software – utilizat la distanță de alte aplicații/servicii –
oferind o funcționalitate specifică

implementarea sa nu trebuie cunoscută
de programatorul ce invocă serviciul



a se revedea
cursul anterior

implementare

Serviciu Web

aspecte de interes – vizând, în special, SOA:

tipul serviciului

rolul serviciului în cadrul arhitecturii software

implementare

Serviciu Web

business service

“abstract, high-level, coarse-grained services that define the core business operations that are performed at the enterprise level”

Mark Richards (2016)

implementare

Serviciu Web

business service

independent de implementare

specifică numele operațiilor oferite,
tipul parametrilor de intrare, tipul rezultatului
(*e.g.*, via WSDL – *Web Service Description Language*
sau *OpenAPI Specification*)

implementare

Serviciu Web

business service

optional, pot fi definite reguli de orchestrare
(contextul execuției serviciului)

uzual via BPEL – *Business Process Execution Language*

implementare

Serviciu Web

enterprise service

reprezintă un serviciu concret
– *e.g.*, componentă *middleware* –
implementând funcționalitățile
definite de serviciul de tip *business*

abordare *coarse-grained*

implementare

Serviciu Web

enterprise service

în relație *one-to-one* sau *one-to-many*
cu serviciile abstracte de tip *business*

poate fi partajat la nivelul organizației

implementare

Serviciu Web

application service

*“fine-grained, application-specific service
that is bound to a specific application context”*

implementare

Serviciu Web

application service

oferă funcționalități specifice, punctuale

apelabil via o interfață cu utilizatorul ori
via un serviciu de tip *enterprise*

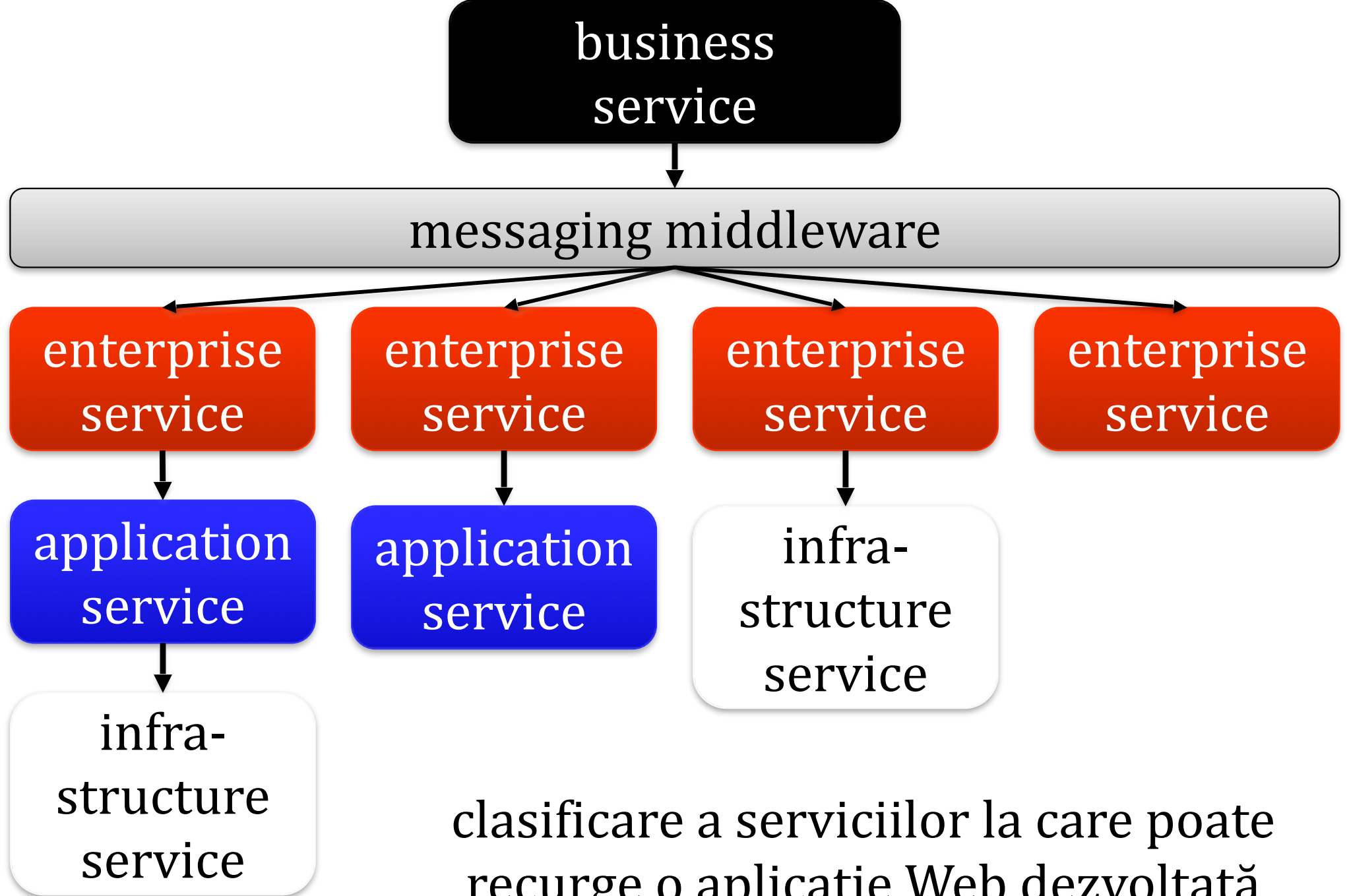
implementare

Serviciu Web

infrastructure service

pune la dispoziție operații interne (private)

invocat de servicii disponibile la nivel
de aplicație (*application services*) sau
organizațional (*enterprise services*)



clasificare a serviciilor la care poate
recurge o aplicație Web dezvoltată
conform principiilor SOA

M. Richards, *Microservices vs. SOA*, O'Reilly (2016)

implementare

Serviciu Web

messaging middleware

folosit la coordonarea apelurilor de servicii

denumit și *integration hub* ori ESB – *Enterprise Service Bus*
(magistrală de servicii la nivel organizațional)

implementare

Serviciu Web

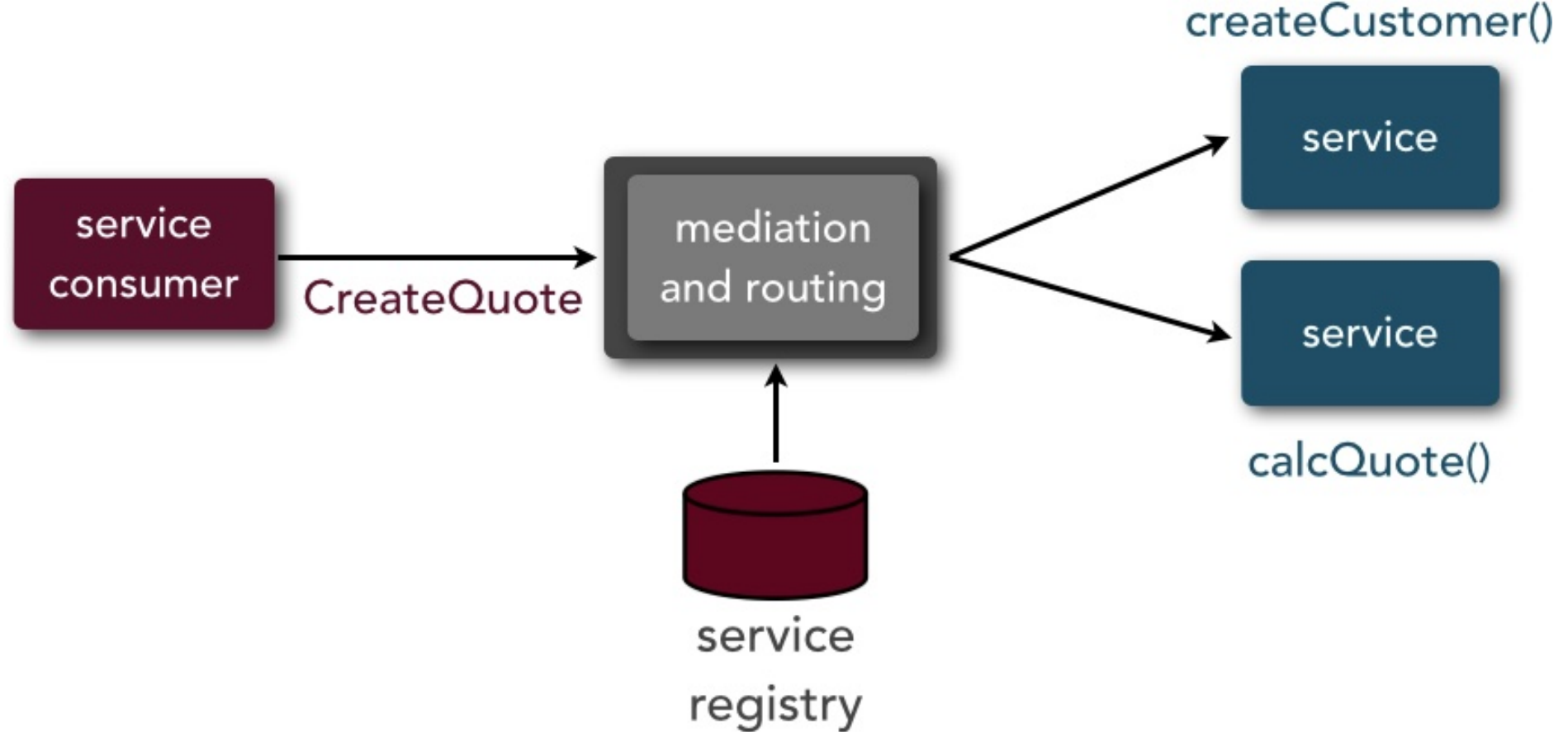
messaging middleware

roluri:

mediere (*mediation*)

și

dirijare (*routing*)



*“the capability of the architecture to **locate and invoke a service** (or services) based on a specific business or user request”* conform Richards (2016)

existența unei componente
de **descoperire a serviciilor** (*service registry*)

implementare

Serviciu Web

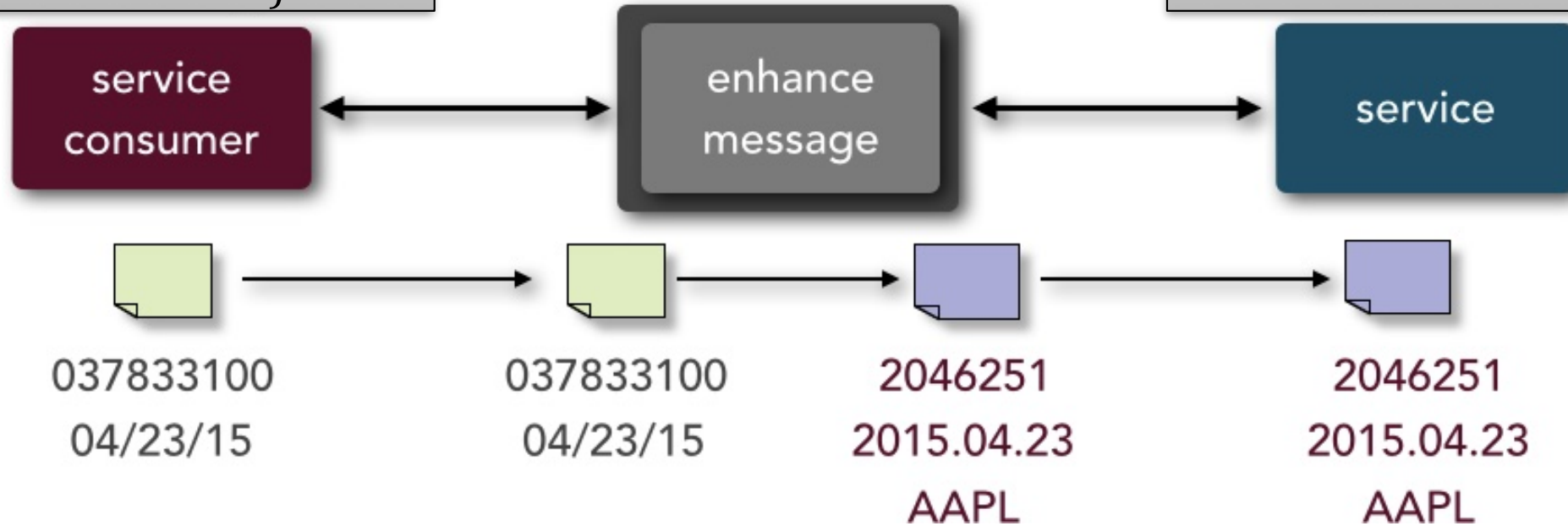
messaging middleware

rol:

message enhancement

USA: CUSIP (*Committee on Uniform Security Identification Procedures*)

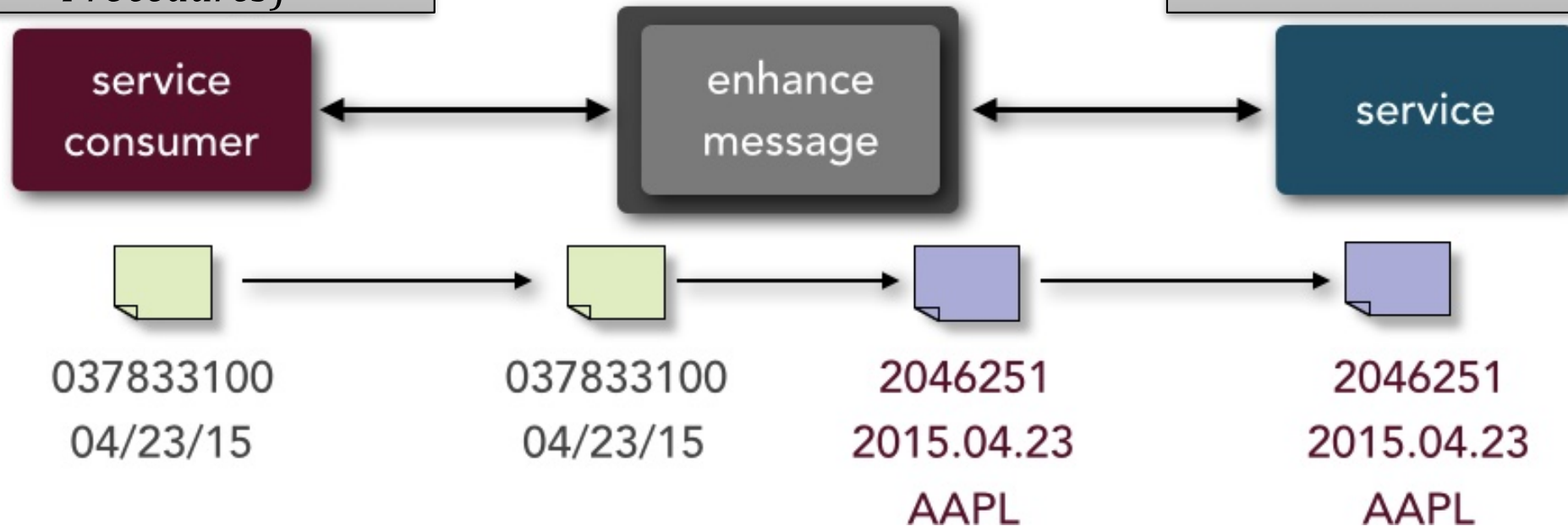
UK: SEDOL (*Stock Exchange Daily Official List*)



“the capability of the architecture to modify, remove, or augment the data portion of a request before it reaches the service” (Richards, 2016)

USA: CUSIP (*Committee on Uniform Security Identification Procedures*)

UK: SEDOL (*Stock Exchange Daily Official List*)



“the capability of the architecture to modify, remove, or augment the data portion of a request before it reaches the service” (Richards, 2016)

exemple: conversii privind formatul,
adăugarea unor date derivate/calculate *ad-hoc* etc.

implementare

Serviciu Web

messaging middleware

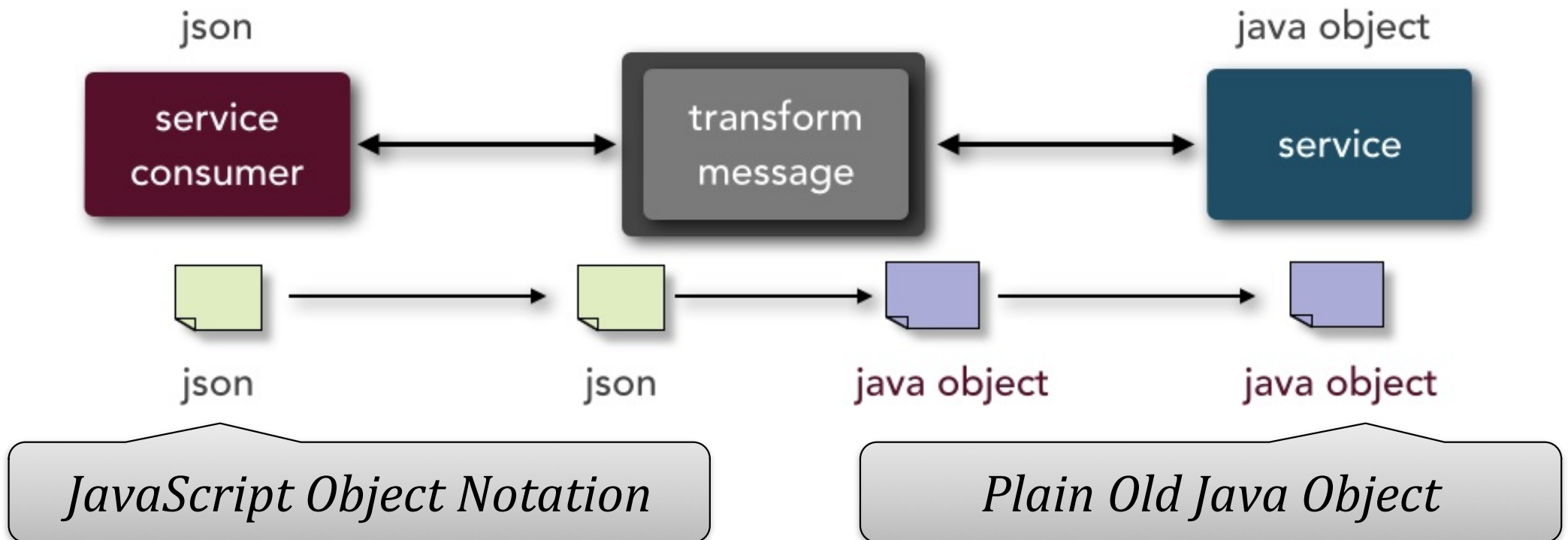
rol:

transformare a mesajelor/protocoalelor

implementare

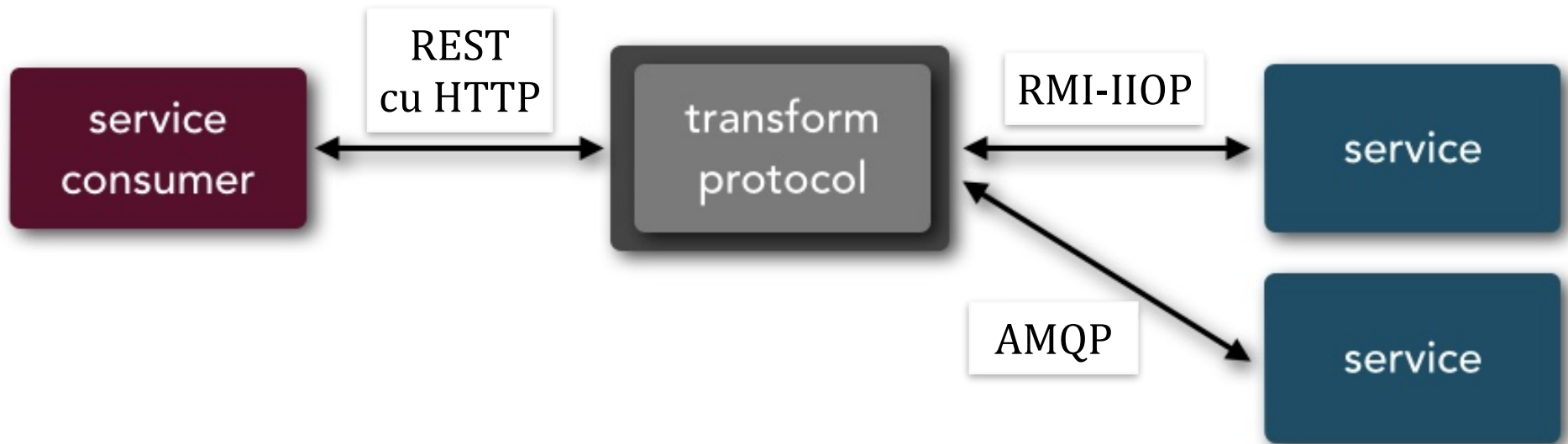
message transformation

e.g., JSON $\leftrightarrow$ Java, JSON $\leftrightarrow$ PDO (PHP Data Objects)



implementare

protocol transformation



RMI-IIOP

(Java Remote Method Invocation over Internet Inter-ORB Protocol)

www.oracle.com/technetwork/java/rmi-iiop-139743.html

AMQP (Advanced Message Queuing Protocol)

www.amqp.org

implementare

Micro-serviciu

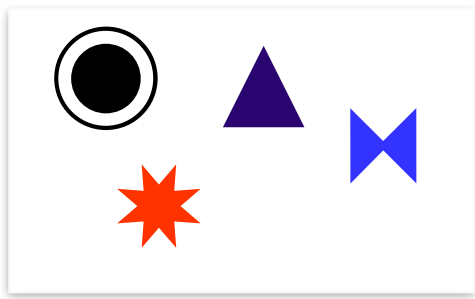
implementează o funcționalitate specifică,
oferită la nivel de unic proces

self-contained system

componentă la nivel de *backend* dezvoltată cu scopul
de a fi **înlocuită**, nu de a fi reutilizată

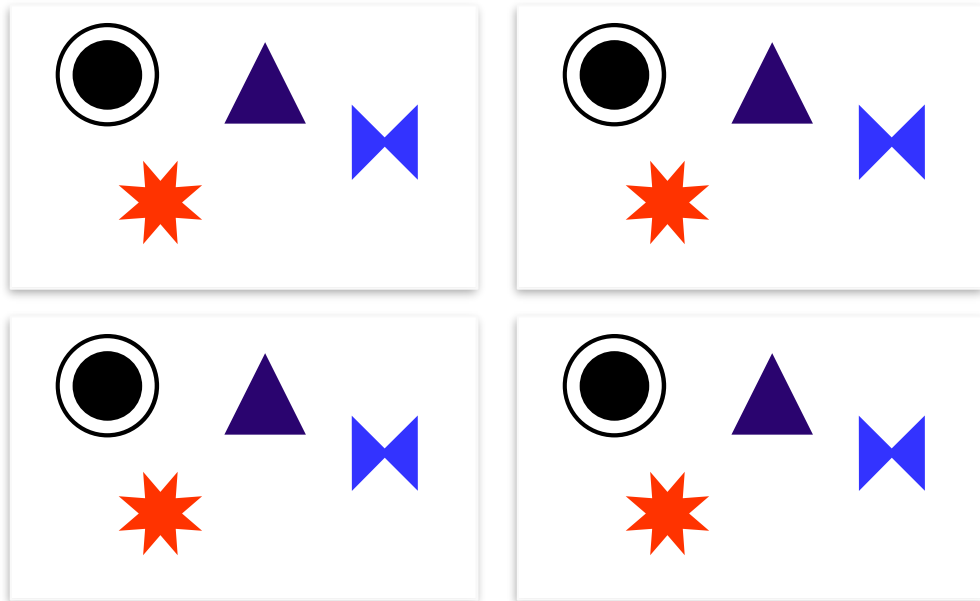
small
each running in its own process
lightweight communication mechanisms (usual, HTTP)
built around business capabilities
independently deployable
minimum of centralized management
may be written in different programming languages
may use different data storage mechanisms

caracteristici ale micro-serviciilor conform
James Lewis & Martin Fowler, *Microservices* (2014)
martinfowler.com/articles/microservices.html

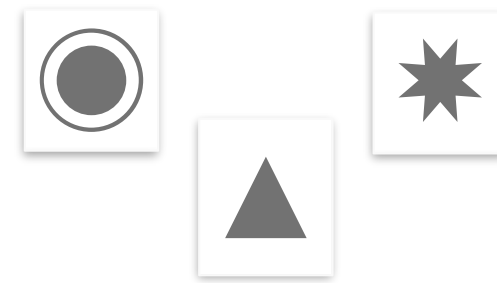


aplicație monolitică

(N funcționalități într-un unic proces)

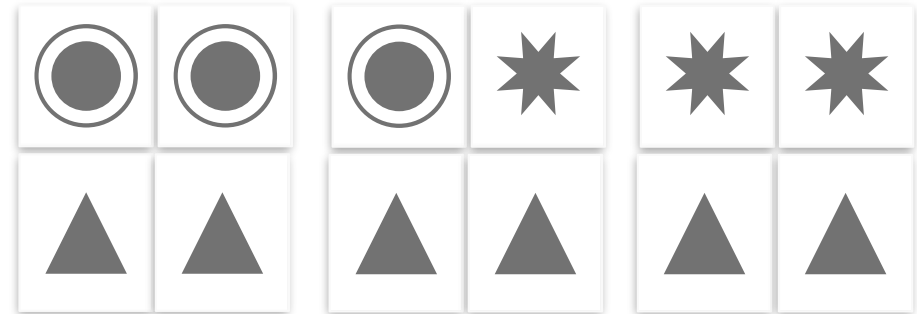


scalabilitate prin replicarea
monolitului pe S servere



micro-servicii

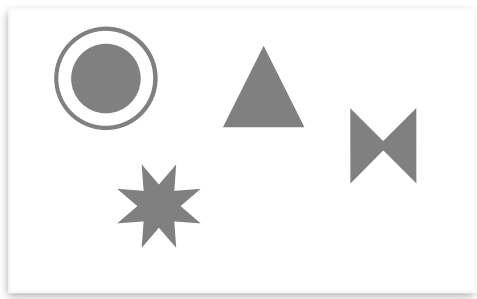
(fiecare funcționalitate $\equiv$ serviciu separat)



scalabilitate prin distribuirea
serviciilor între servere,
replicându-le conform
necesităților

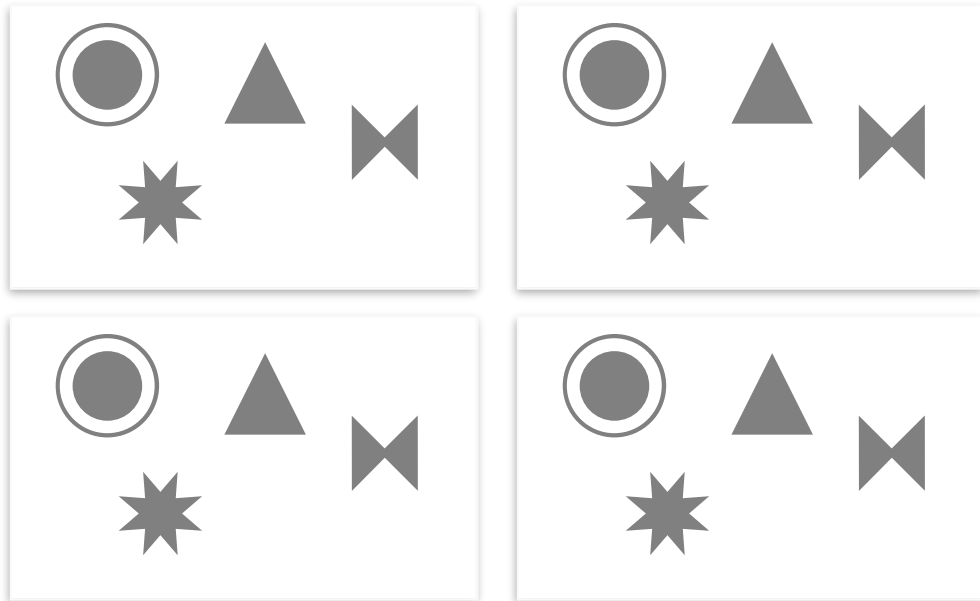
necesități:

modularitate, descentralizare și evoluție permanentă

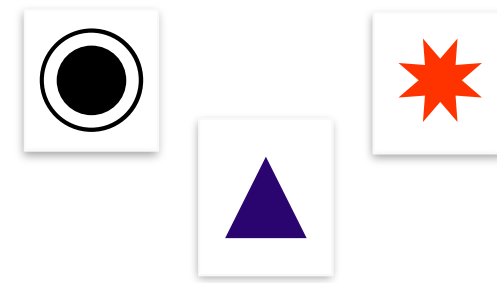


aplicație monolitică

(N funcționalități într-un unic proces)

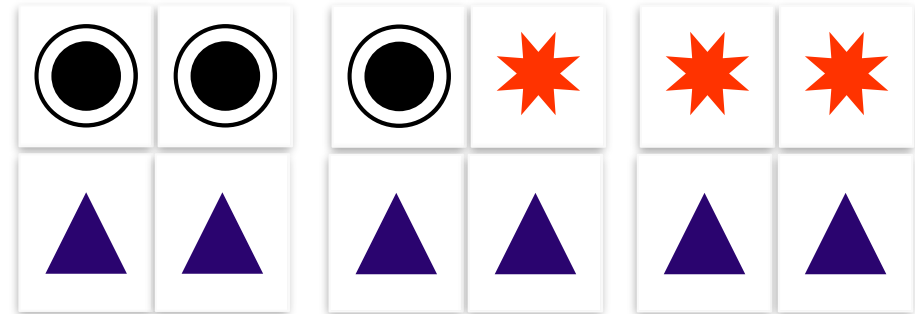


scalabilitate prin replicarea
monolitului pe S servere



micro-servicii

(fiecare funcționalitate $\equiv$ serviciu separat)



scalabilitate prin distribuirea
serviciilor între servere,
replicându-le conform
necesităților

exemple de bună practică + alte resurse: microservices.io

Microservices @ The New Stack (2025)

thenewstack.io/microservices/

Beneficii ale micro-serviciilor:

izolare

autonomie

scalabilitate individuală

reziliență

viteză

suport pentru experimentare

feedback rapid

flexibilitate

ușor de înlocuit

ecosistem

S. Tilkov, *A Question of Size – Modularization & Microservices*,
Java Forum Nord (2017):

speakerdeck.com/stilkov/a-question-of-size-modularization-and-microservices

micro-servicii

Funcționale (*functional services*)

implementează funcționalități specifice
(*business operations*)

micro-servicii

Funcționale (*functional services*)

implementează funcționalități specifice
(*business operations*)

expuse consumatorului de servicii

independente (fără efecte colaterale – *side effects*)

nu sunt partajabile uzual

micro-servicii

Control – infrastructură (*infrastructure services*)

implementează activități non-funcționale:
autentificare, autorizare, jurnalizare, monitorizare,...

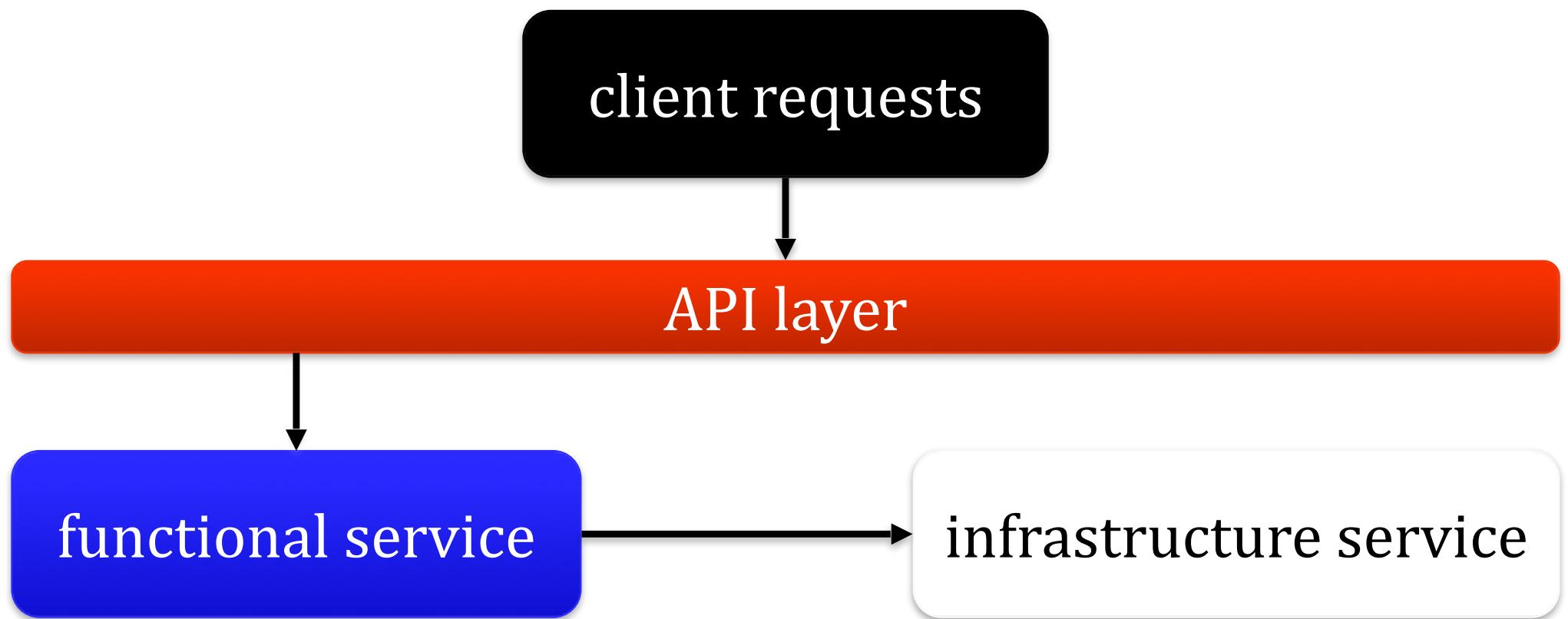
micro-servicii

Control – infrastructură (*infrastructure services*)

implementează activități non-funcționale:
autentificare, autorizare, jurnalizare, monitorizare,...

nu sunt expuse în exterior – private

pot fi partajate la nivel de aplicație ori servicii interne



de la cereri efectuate de client la obținerea răspunsului
oferit de API via micro-servicii funcționale
bazate pe cele vizând infrastructura

micro-servicii

Aspect de interes:

proprietar – *ownership*

+

coordonare – *coordination*

micro-servicii

Aspect de interes:

partajarea funcționalităților

share-as-much-as possible – SOA clasic

versus

share-as-little-as possible – micro-servicii

micro-servicii

Aspect de interes:

comunicarea – uzual, asincronă – între (micro-)servicii

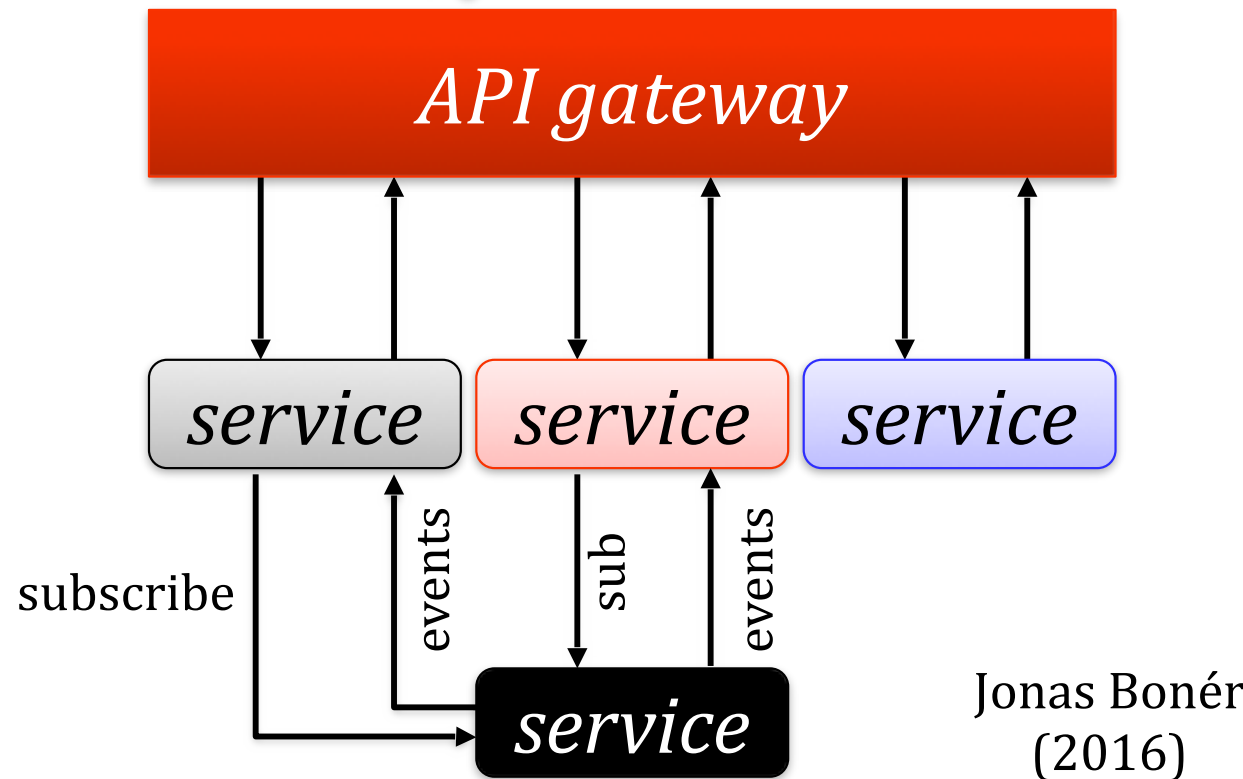
abordări:

point-to-point

sau

publish-subscribe

simplificarea
accesului clientului
via API



Jonas Bonér
(2016)

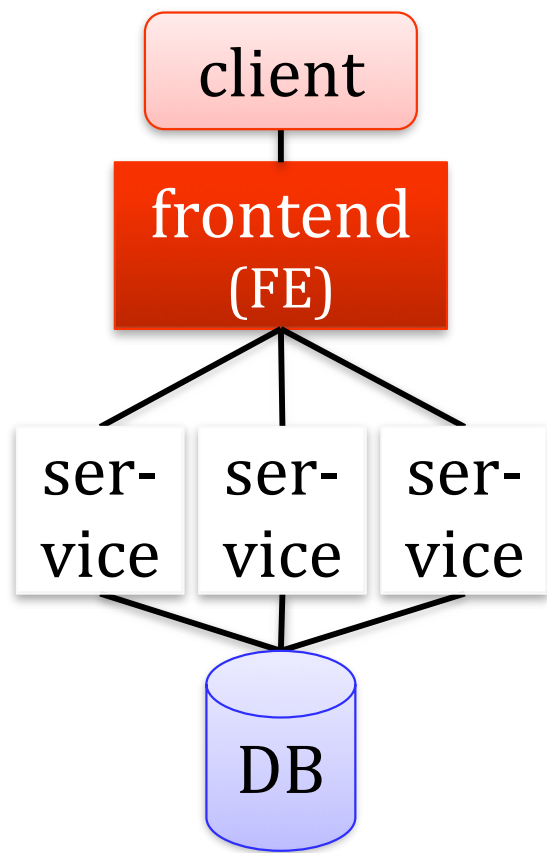
intern, (micro-)serviciile pot comunica recurgând la
publish-subscribe – vezi și specificațiile W3C
ActivityPub (2018), *WebSub* (2018)

micro-servicii

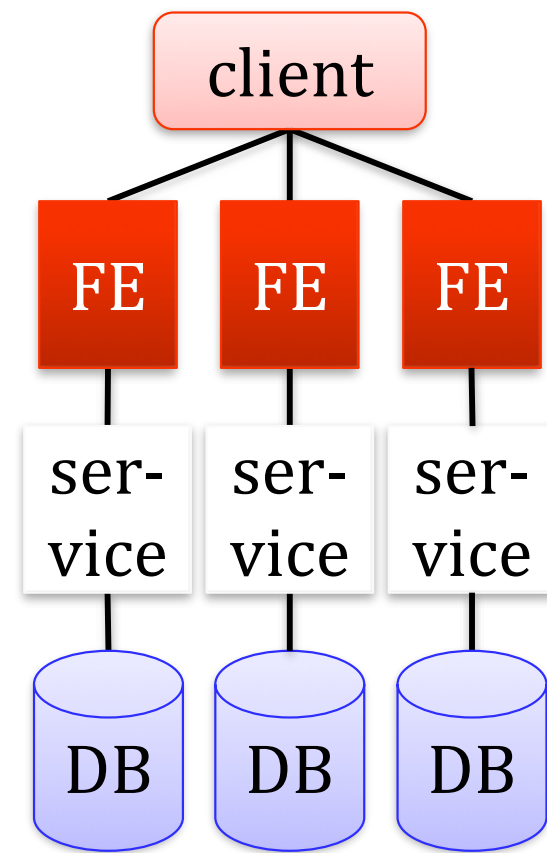
Uzual, arhitecturile ce recurg la micro-servicii
nu includ componente *middleware*
și nu oferă suport pentru abstractizarea interacțiunii dintre
producătorii și consumatorii de servicii
(*contract decoupling*)

► **μSOA** – *Micro-Service Oriented Architecture*

Sam Newman, *Building Microservices* (2nd Edition), O'Reilly (2021)



arhitectură bazată
pe servicii Web



arhitectură recurgând
la micro-servicii

Z. Dehghani, *How to break a Monolith into Microservices* (2018)
martinfowler.com/articles/break-monolith-into-microservices.html

pentru alte meta-date de interes folosite la catalogare,
a se consulta Z. Einy (2022): www.getport.io/blog/microservice-catalog

micro-servicii: dezvoltare

Platforme:

Dapr, Express Serverless, M3O, OpenWhisk, Triton,...

Framework-uri:

Akka, Finagle, Eclipse MicroProfile, Micronaut, Moleculer,
ScaleCube, Spin, Vert.X, Wangle etc.

SDK-uri multi-limbaj:

CoAP, gRPC, Hprose

limbaje de programare specializate:

Ballerina, Jolie, Wing

multe alte instrumente software enumerate la

github.com/mfornos/awesome-microservices

API

Accesul la un (micro-)serviciu are loc uzual pe baza unei interfețe de programare a aplicației
API (***Application Programming Interface***)

“any well-defined interface that defines the service that one component, module, or application provides to other software elements”
de Souza et al. (2004)

API

Componentă software concepută și invocată
via tehnologiile Web actuale
(URI, HTTP, formate de date: JSON, XML)

poate fi dezvoltată conform unui stil arhitectural
e.g., REST (*REpresentational State Transfer*)



vezi cursul trecut

API

API public

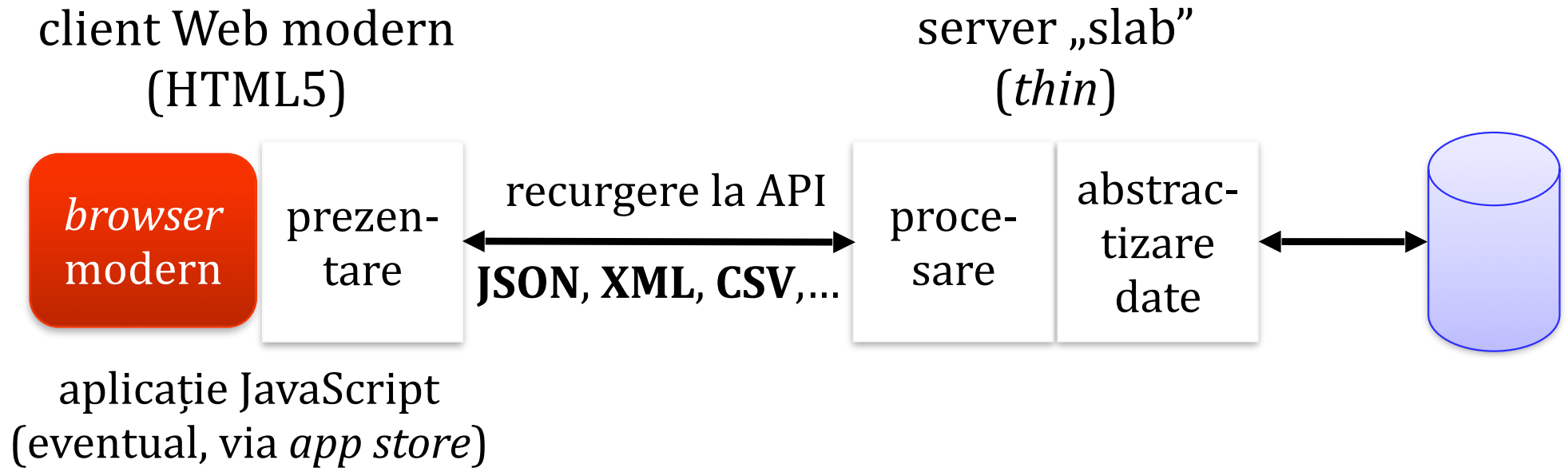
(disponibil pe baza unei licențe de utilizare)

versus

API privat

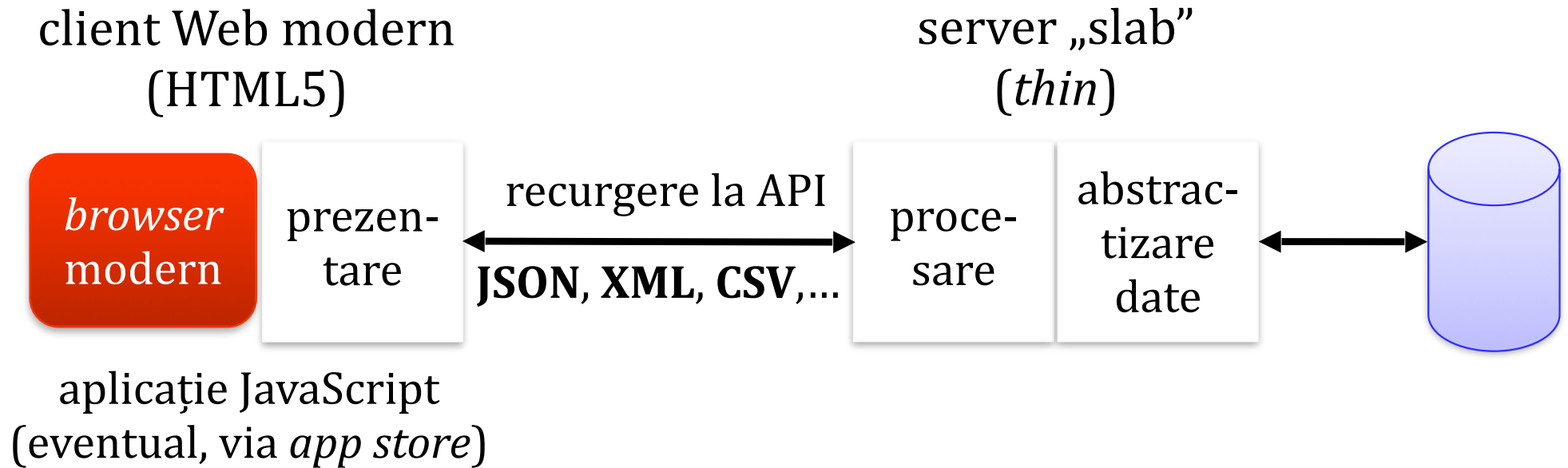
(pentru uz intern)

API: abordare client – JavaScript



browser Web pe calculatoare
convenționale, dispozitive
mobile și altele

API: abordare client – JavaScript



implementarea aplicației JavaScript poate recurge la biblioteci, *framework*-uri, componente specifice *e.g.*, Angular, htmx, Lit, React, Riot.js, Svelte, Vue

API: abordare client – JavaScript

La nivel de client (*frontend*),
crearea interfețelor cu utilizatorul poate fi facilitată de:

design systems

micro-frontends

design systems

*“A **design system** is a complete set of standards intended to manage design at scale using reusable components and patterns”*

Therese Fessenden (2021)

www.nngroup.com/articles/design-systems-101/

design systems

design repository



style guides

guidelines

design principles

visual references



UI component libraries (design libraries)



UI pattern libraries

people (team)



interaction designer(s)



visual designer(s)



UI developer(s)

design systems

Example:

designsystemsrepo.com/design-systems/

cazuri concrete:

Atlassian – atlassian.design

US Web Design System – designsystem.digital.gov

W3C – design-system.w3.org

micro-frontends

*“An **architectural style** where independently deliverable frontend applications are composed into a greater whole”*

Cam Jackson (2019)

martinfowler.com/articles/micro-frontends.html

micro-frontends

Aplicarea principiilor micro-serviciilor la proiectarea și implementarea componentelor de interfață cu utilizatorul

beneficii importante:

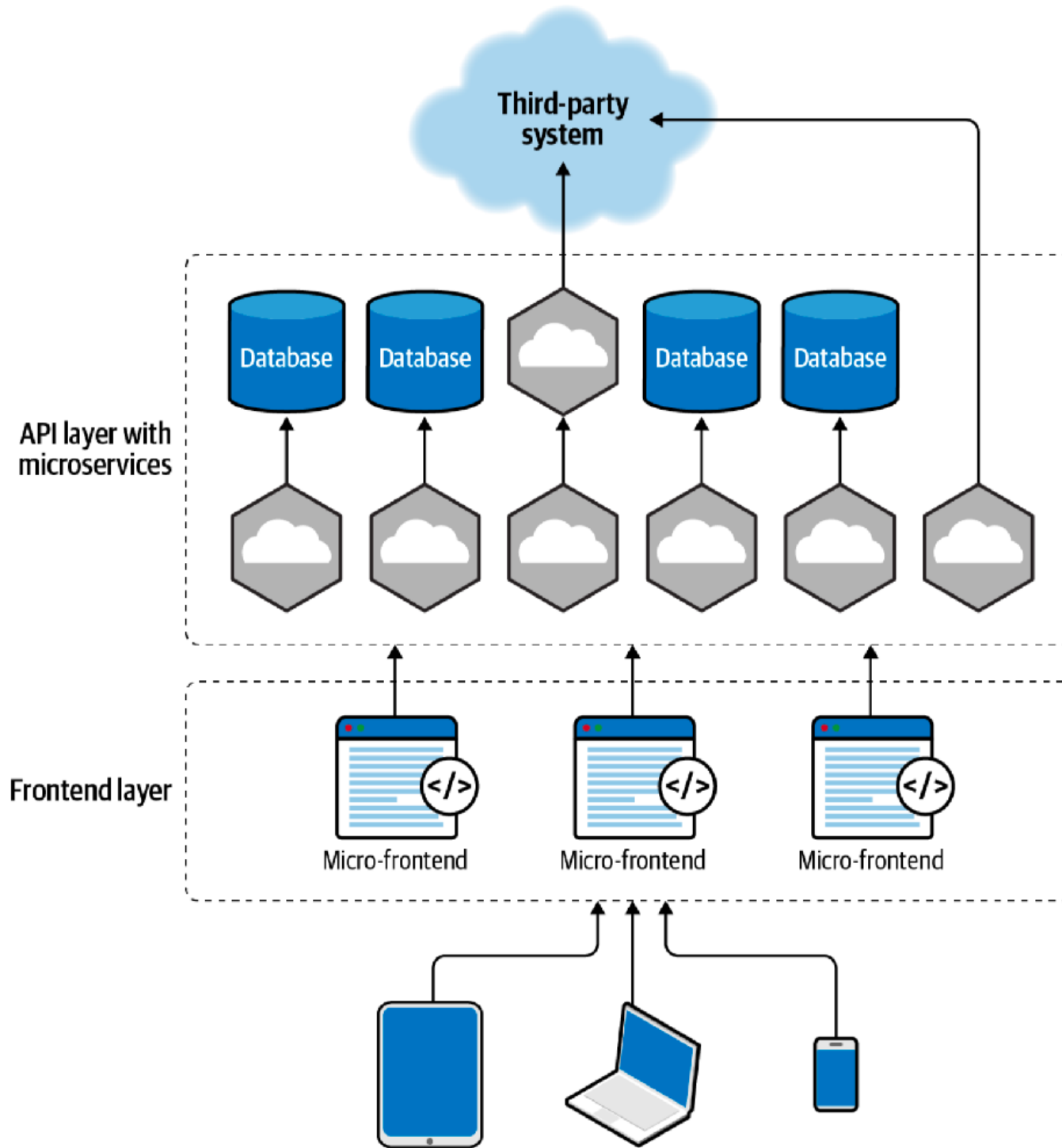
actualizări incrementale

simplificarea codului

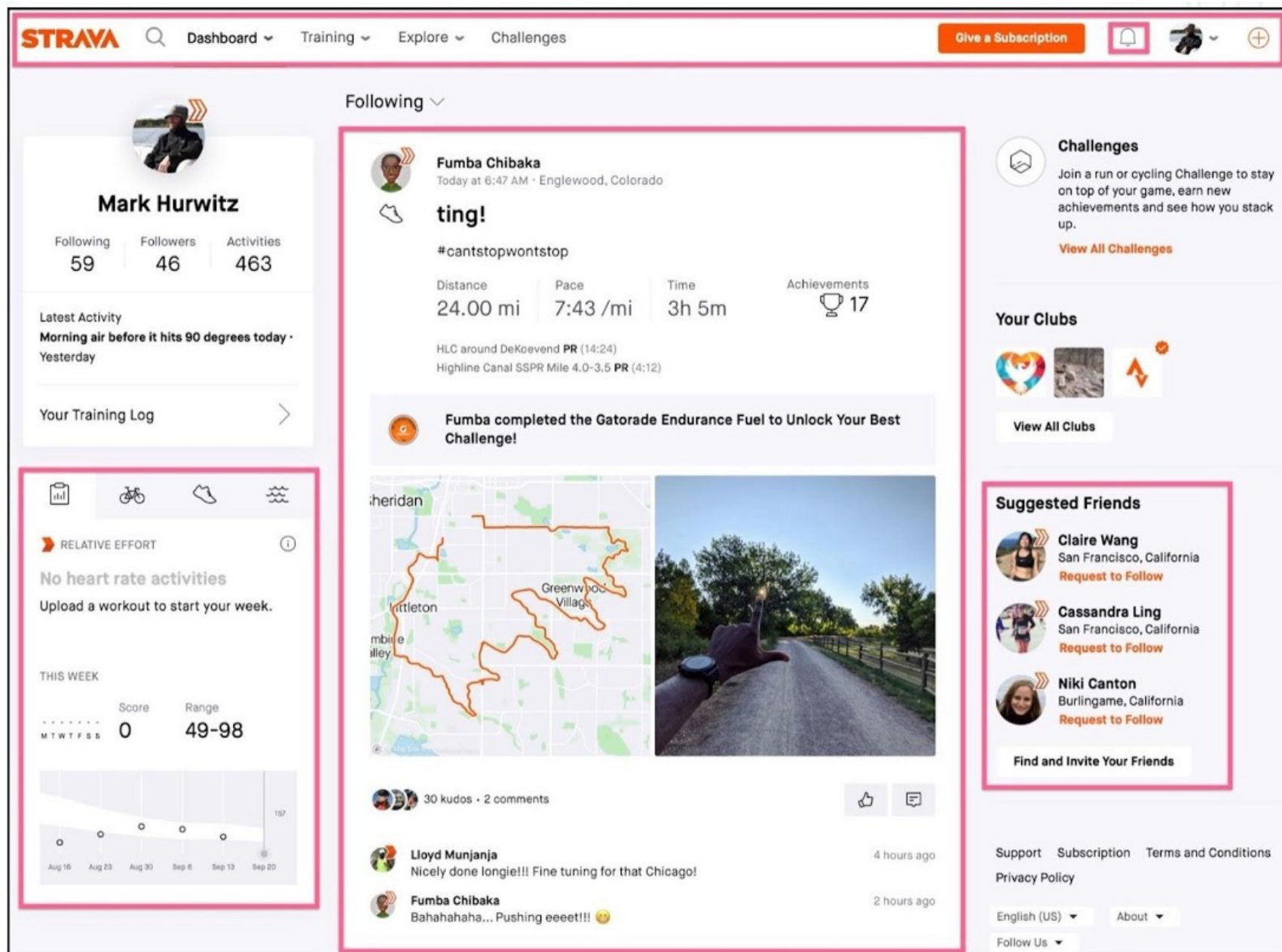
componente UI dezvoltate și exploatate independent

echipe de dezvoltare autonome

testare rapidă



Luca Mezzalana, *Building Microfrontends*, O'Reilly (2022)



caz concret: **Strava** (Mark Hurwitz, 2022)
main dashboard page decomposed into different micro-frontends
medium.com/strava-engineering/strava-web-microfrontends-79e9586c0222

Taste of Iberia

Menu:

\$25	Seafood paella	-	x1	+	\$25
\$27	Mixed tapas	-	x2	+	\$54
\$70	2012 Barbazul (Red)	-	x1	+	\$70

Total: \$149

Order now



Paella, tapas, and imported Spanish wines

Cod-sursă demonstrativ:
github.com/micro-frontends-demo

API: aplicații native

client nativ modern
(*smart device*)

sistem
de
operare

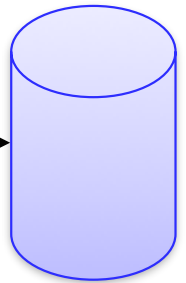
prezen-
tare

recurgere la API
JSON, XML, CSV,...

server „slab”
(*thin*)

proce-
sare

abstrac-
tizare
date



aplicație nativă
C#, Kotlin, Swift,...
(uzual, via *app store*)

*desktop și/sau mobile,
smart TV, home appliance,
dispozitiv ambiental*

API: aplicații native

client nativ modern
(*smart device*)

sistem
de
operare

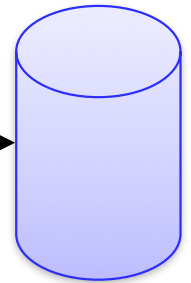
prezen-
tare

recurgere la API
JSON, XML, CSV,...

server „slab”
(*thin*)

proce-
sare

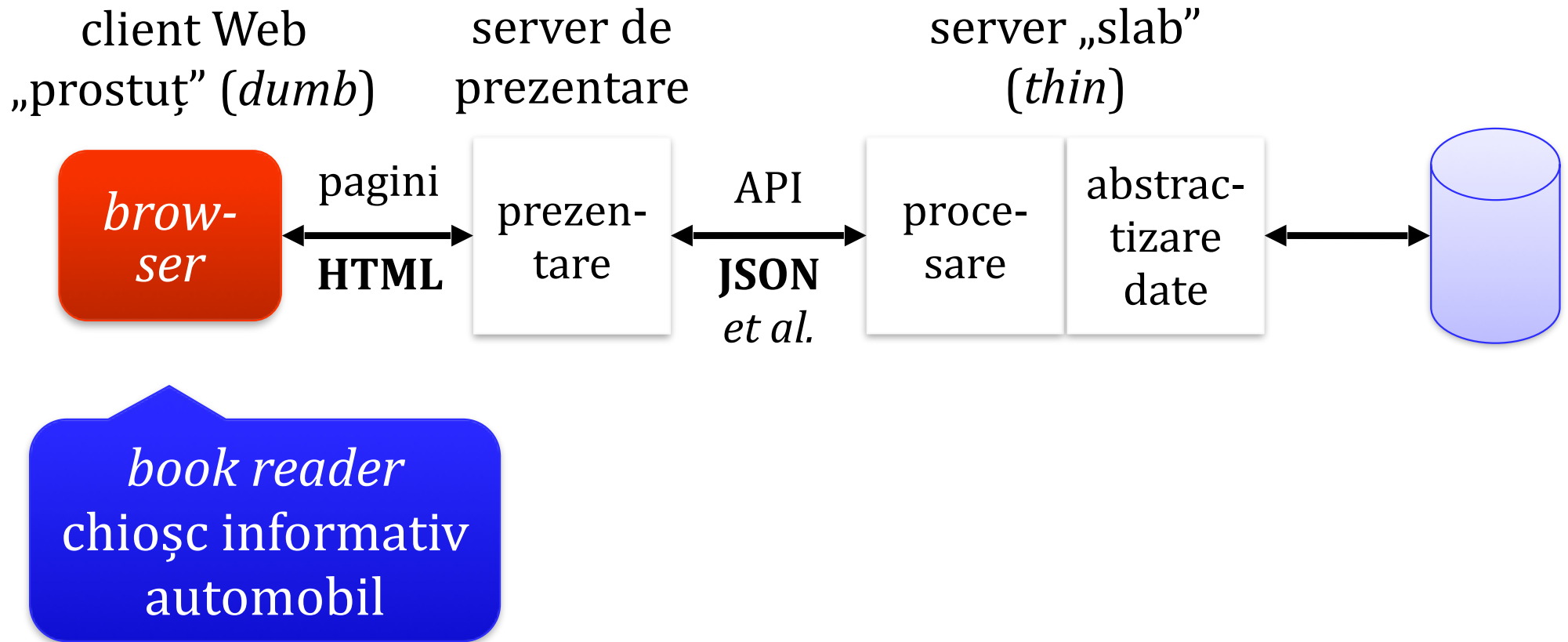
abstrac-
tizare
date



aplicație nativă
C#, Kotlin, Swift,...
(uzual, via *app store*)

implementarea aplicației native poate recurge la
biblioteci, *framework*-uri, componente specifice
e.g., Apache Cordova, Electron, Flutter, Ionic, React Native

API: abordare bazată pe intermediari



API: instrumente de testare

Utilitarul **curl** + biblioteca **LibcURL** – curl.haxx.se

HTTPIe (client în linia de comandă) – httpie.io

Hoppscotch – hoppscotch.io

Insomnia – insomnia.rest

Postman – www.postman.com

REST Test Test – resttesttest.com

API: în contextul *serverless*

Aplicația depinde semnificativ de
componente externe, disponibile în „nori”

(micro-)servicii expuse via API



abordarea *serverless*

API: în contextul *serverless*

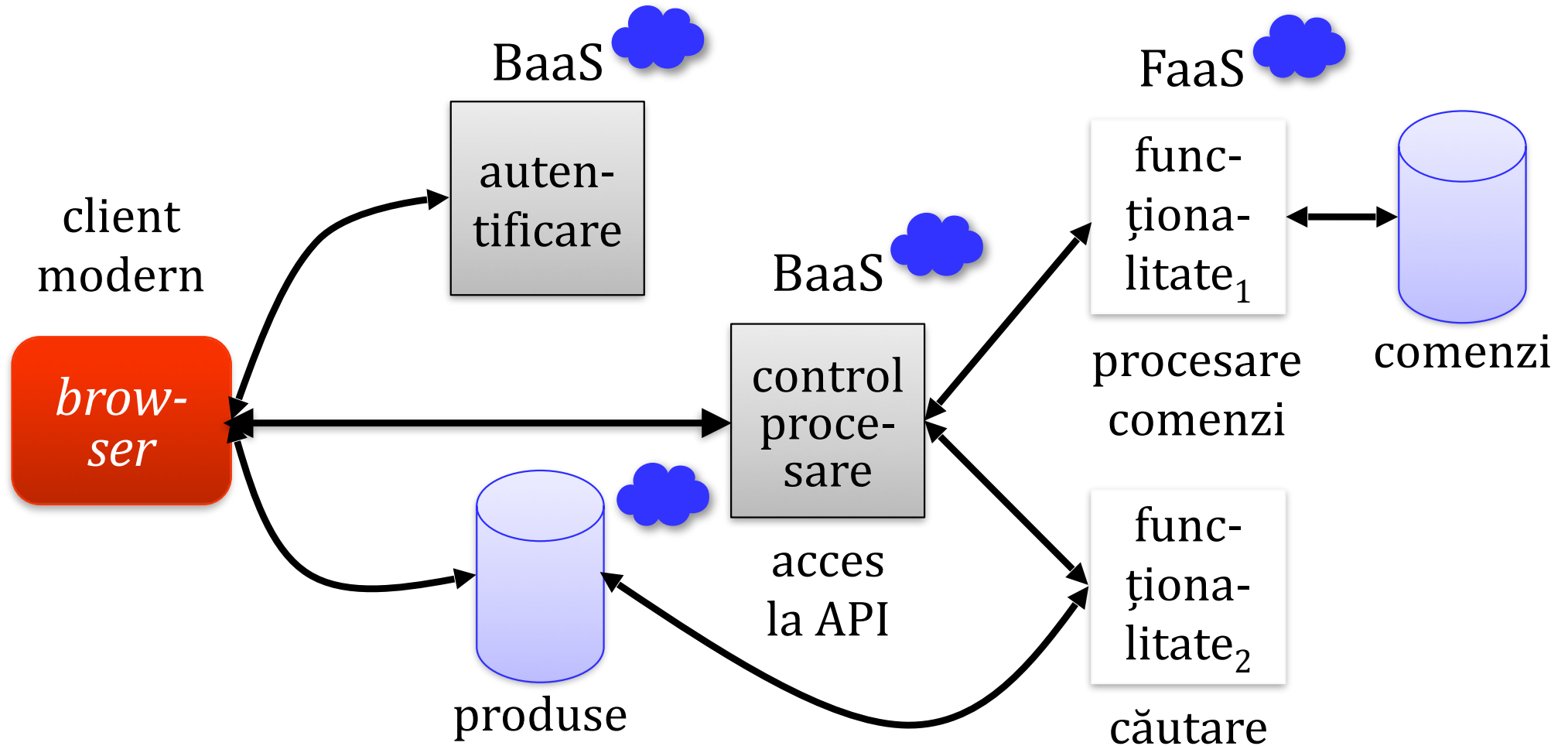
Serverless

strat de abstractizare a accesului la resursele
unei platforme de tip *cloud*

Mike Roberts (2018)

martinfowler.com/articles/serverless.html

API: în contextul *serverless*



BaaS = *(Mobile) Backend As A Service*

FaaS = *Functions As A Service*

API: în contextul *serverless*

FaaS – *Functions As A Service*

funcții (*cloud functions*) implementând funcționalități
expuse consumatorului de servicii

as small as possible

uzual, implementări sub 100 de linii de cod

API: în contextul *serverless*

FaaS – *Functions As A Service*

executate – la nivel de server – independent și asincron,
fără a cauza **efecte colaterale**

declanșate de **evenimente**

programatorul nu e preocupat de
managementul resurselor și alte sarcini

API: în contextul *serverless*

BaaS – *Backend As A Service*

încapsulează servicii de infrastructură ce implementează
activități non-funcționale
(autentificare, autorizare, jurnalizare, monitorizare etc.)

private – nu sunt expuse în exterior

pot fi partajate de serviciile interne

API: în contextul *serverless*

Serverless computing = FaaS + BaaS

documentații, resurse, soluții:

github.com/anaibol/awesome-serverless



Cum poate fi descrisă interfața unui API?

API: descriere abstractă

OpenAPI Specification (ex-Swagger) – openapis.org

RAML (*RESTful API Modeling Language*) – raml.org

API Blueprint – apiblueprint.org

alte resurse de interes:

github.com/Kikobeats/awesome-api

API: descriere abstractă

OpenAPI Specification

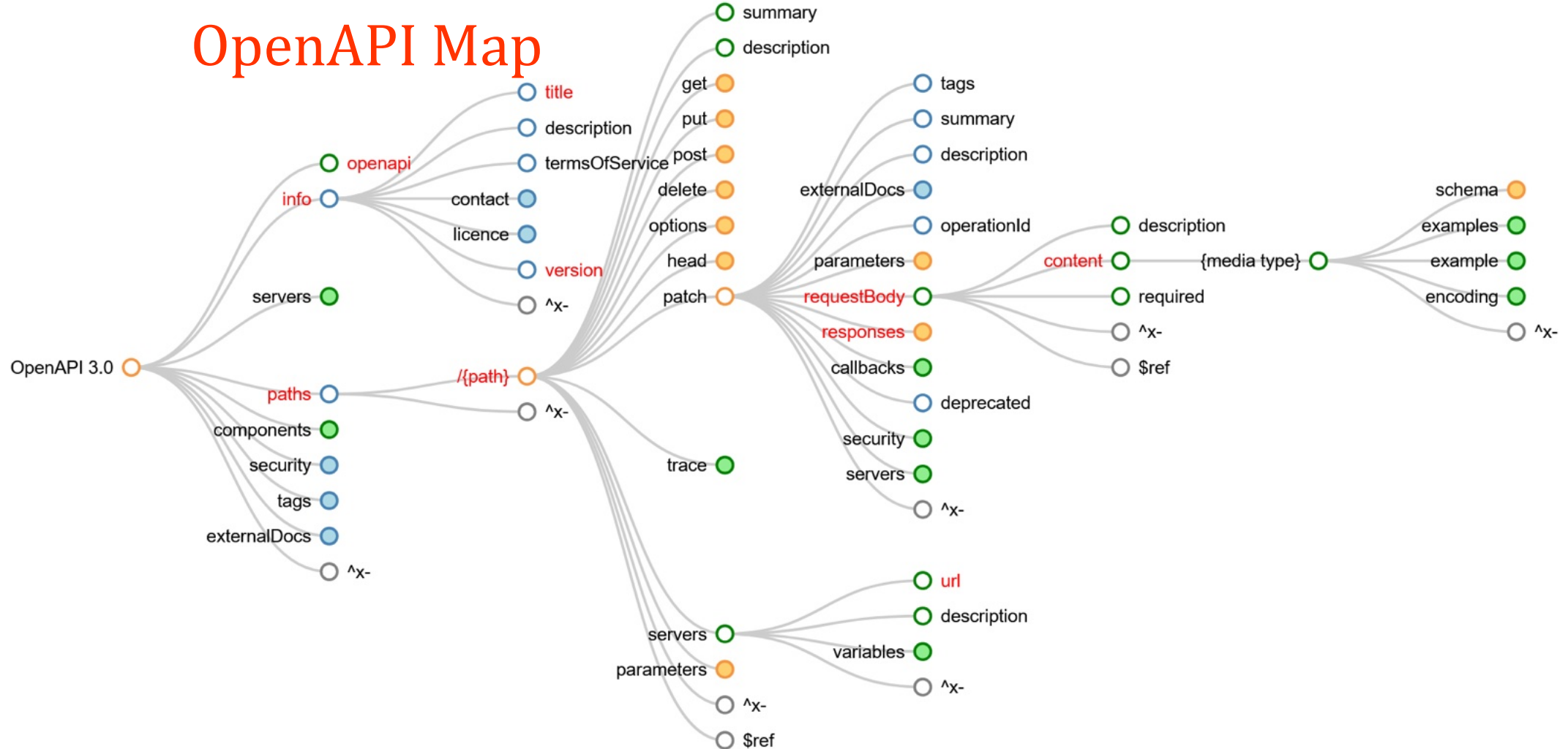
soluție modernă de a declara – independent de platformă –
interfața publică a unui API REST

versiune recentă: OpenAPI 3.2.0 (19 septembrie 2025)
suport și pentru OAuth, scalabilitate, JSON *streaming* etc.

formate folosite:

JSON și/sau YAML (*YAML Ain't Markup Language*)

OpenAPI Map



operationId property



string

Unique string used to identify the operation. The id MUST be unique among all operations described in the API. Tools and libraries MAY use the operationId to uniquely identify an operation, therefore, it is recommended to follow common programming naming conventions.

explorarea vizuală a construcțiilor OpenAPI Specification

openapi-map.apihandyman.io/?version=3.0

```

{ // specificăm accesul la o resursă Web: GET /resource?param
  "openapi": "3.0.0",
  "paths": {          // căi (URL-uri) spre resursele oferite de API-ul REST
    "/resource": { // acces la reprezentarea resursei desemnate de /resource
      "get": {      // metoda HTTP: GET, HEAD, POST, PUT, PATCH, DELETE
        "operationId": "service", // specifică numele unic al operației dorite
        "parameters": [ {        // lista parametrilor
          "name": "param",
          "in": "query",          // parametru adăugat URL-ului (în query string)
          "schema": { "type": "string" }
        } ],
        "responses": {           // răspunsuri oferite (succes, eroare,...)
          "200": {               // cod de stare HTTP: 200, 400, 403, 405, 500
            "description": "Success",
            "schema": {         // descrie (cu JSON Schema) răspunsul oferit
              "$ref": "#/definitions/Response"
            }
          }
        }
      }
    }
  }
}

```

„scheletul” unui document
OpenAPI specificând un API REST

Swagger Editor
Supported by SWARTBEAR

File

Edit

Generate Server

Generate Client

About

```
77 petstore_auth:
78   - write: $ref: string
79   - read: pet
80   post:
81     tags:
82       - pet
83     summary: Add a new pet to the store
84     description: Add a new pet to the store
85     operationId: addPet
86     requestBody:
87       description: Pet object that needs to be added to the store
88       content:
89         application/json:
90           schema:
91             $ref: '#/components/schemas/Pet'
92         application/xml:
93           schema:
94             $ref: '#/components/schemas/Pet'
95         application/x-www-form-urlencoded:
96           schema:
97             $ref: '#/components/schemas/Pet'
98       required: true
99     responses:
100       '200':
101         description: Successful operation
102         content:
103           application/json:
104             schema:
105               $ref: '#/components/schemas/Pet'
106           application/xml:
107             schema:
108               $ref: '#/components/schemas/Pet'
109       '405':
110         description: Invalid input
111     security:
112       - petstore_auth:
113         - write:pets
114         - read:pets
```

pet

Everything about your Pets

[Find out more](#)

PUT

/pet

Update an existing pet

⌵

🔒

POST

/pet

Add a new pet to the store

⌵

🔒

GET

/pet/findByStatus

Finds Pets by status

⌵

🔒

GET

/pet/findByTags

Finds Pets by tags

⌵

🔒

GET

/pet/{petId}

Find pet by ID

⌵

🔒

POST

/pet/{petId}

Updates a pet in the store with form data

⌵

🔒

DELETE

/pet/{petId}

Deletes a pet

⌵

🔒

POST

/pet/{petId}/uploadImage

uploads an image

⌵

🔒

store

Access to Petstore orders

[Find out more about our store](#)

GET

/store/inventory

Returns pet inventories by status

⌵

🔒

POST

/store/order

Place an order for a pet

⌵

GET

/store/order/{orderId}

Find purchase order by ID

⌵

SwaggerEditor – editarea specificației unui API REST

Bitbucket

Search everything...

```

/teams/{username}
/teams/{username}/followers
/teams/{username}/following
/teams/{username}/hooks
/teams/{username}/hooks/{uid}
/teams/{username}/members
/teams/{username}/pipelines_config/...
/teams/{username}/pipelines_config/...
/teams/{username}/repositories
/user
/user/emails
/user/emails/{email}
/users/{username}
/users/{username}/followers
/users/{username}/following
/users/{username}/hooks
/users/{username}/hooks/{uid}
/users/{username}/pipelines_config/v...
/users/{username}/pipelines_config/v...
/users/{username}/repositories
    
```

Data Types (88)

```

</> account
</> author
</> base_commit
</> branchrestriction
</> commit
</> commitstatus
    
```

/teams/{username}/hooks

Design

Source

PATH PARAMETERS (1)

```

username > No description. > string
    
```

QUERY PARAMETERS

HEADER PARAMETERS

OPERATIONS (2)

Get

Put

Post

Delete

Options

Head

Patch

INFO

PATH PARAMETERS (1)

QUERY PARAMETERS

HEADER PARAMETERS

REQUEST BODY

RESPONSES (3)

201 Created

403 Forbidden

404 Not Found

inspectarea specificației
unui API public
(aici, Bitbucket)

Name	Language	v3.2	v3.1	v3.0	GitHub
 Speakeasy OpenAPI - An OSS set of packages and tools used by Speakeasy for working with OpenAPI, Arazzo and Overlay Specification documents. Includes CLI tools for validation, bundling, inlining, upgrading, and more.	Go, CLI	✗	✓	✓	
libopenapi - Enterprise grade, fully featured OpenAPI 3.1, 3.0 and Swagger parser library for go. A complete toolset for reading and parsing OpenAPI and Swagger specifications. Comes complete with high and low-level APIs, diff engine, index and resolver.	go	✓	✓	✓	
openapi-format - A CLI to format an OpenAPI document by ordering fields in a hierarchical order, with the option to filter out flags, tags, methods, operationIDs; including the option to convert between OpenAPI 3.0, 3.1 and 3.2.	Node.js	✓	✓	✓	
api - Magical SDK generation from an OpenAPI definition	JavaScript, TypeScript	✗	✓	✓	
oas-normalize - Comprehensive tooling for working with OpenAPI definitions.	JavaScript, TypeScript	✗	✓	✓	
openapi-processor/openapi-parser - OpenAPI parser with (json schema) validation (currently 3.0 only). Separate apis for OpenAPI 3.0 & 3.1. Easily get resolved \$ref objects. Pluggable document reader & json/yaml converter. Minimal dependencies.	Java	✗	✓	✓	

instrumente OpenAPI pentru conversie, validare, documentare, generare de cod, testare etc.: openapi.tools



Search by algolia

61 APIs found

Image-Charts

v6.1.19



Charts, simple as a URL. A safe and fast replacement for Google Image Charts

EC2 Image Builder

v2019-12-02



EC2 Image Builder is a fully managed Amazon Web Services service that makes it

VirtualMachineImageT...

v2019-05-01-preview



Virtual Machine Image Template

Vehicle Image

v1.0



The vehicle images API offers access to original Mercedes-Benz vehicle images. It

Image Search Client

v1.0



The Image Search API lets you send a search query to Bing and get back a list of relevant

Image Moderation

v1.0.0

Our FREE API blocks images with nudity. Build from the ground up, accurate models,

Getty Images

v3



Custom Image Search ...

v1.0



The Bing Custom Image Search API lets you send an image search query to Bing and get

Facial Recognition Rev...

v1.02



Let your users search the Internet by face! Integrate FaceCheck facial search

AWS App Runner

v2020-05-15



App Runner App Runner is an application service that provides a fast, simple, and

Amazon EC2 Container...

v2015-09-21

Amazon Elastic Contai...

v2020-10-30

Amazon Rekognition

v2016-06-27

PowerTools Developer

v2021.1.01

Computer Vision

v1.0

o listă de API-uri: apis.guru



Aspecte privind securitatea API-urilor Web?

Securitatea serviciilor (API-urilor) Web

folosirea obligatorie a HTTPS
+
certIFICATE digitale actualizate

Securitatea serviciilor (API-urilor) Web

datele „sensibile” nu trebuie expuse în URL

corect:

[https://web.info/resourceCollection/\[ID\]/action](https://web.info/resourceCollection/[ID]/action)

incorect:

<http://web.info/controller/7/action?apiKey=s74b1901c07>

Securitatea serviciilor (API-urilor) Web

permiterea accesului la API doar prin cheie (*API key*)

+

verificarea acestei chei pentru fiecare cerere în parte

utilizatorii trebuie să recurgă
la un mecanism solid de autentificare

Securitatea serviciilor (API-urilor) Web

privilegii reduse: restricționarea metodelor HTTP
de pildă, doar GET

+

folosirea de liste de clienți/utilizatori agreeți (*whitelist*)

Securitatea serviciilor (API-urilor) Web

utilizarea OAuth pentru autorizare

a se considera și schimbul de date
via JWT (*JSON Web Tokens*) pentru controlul accesului

Am putea utiliza servicii Web (API-uri)
pentru autorizare și autentificare?



autorizare

Etape esențiale:

obținerea unei chei de acces 



autentificarea + autorizarea aplicației 



obținerea acordului utilizatorului   



apelarea funcționalităților serviciului (via API) 

autorizare

Pași uzuali de urmat pentru implementarea unei aplicații ce va invoca un serviciu Web pe baza unui API public:

(1) înregistrarea aplicației concepute
via situl entității furnizoare a serviciului

► cheie de acces 

API key, consumer key, developer key

autorizare

Pași uzuali de urmat pentru implementarea unei aplicații ce va invoca un serviciu Web pe baza unui API public:

(2) pe baza acestei chei, aplicația se va putea autentifica pentru a putea fi autorizată să acceseze serviciul dorit

pot fi impuse diverse politici de acces (*permissions*):



doar consultare (*read*), posibilitatea editării etc.

autorizare

Pași uzuali de urmat pentru implementarea unei aplicații ce va invoca un serviciu Web pe baza unui API public:

(3) autentificarea și autorizarea aplicației
au loc cu acordul utilizatorului

dacă utilizatorul nu este autentificat, i se vor solicita informațiile de autentificare (*e.g.*, nume + parola)   

– eventual, folosind **2FA** (*Two Factor Auth*) –,
apoi va putea autoriza aplicația să aibă acces la date
via serviciul Web furnizat

autorizare

Pași uzuali de urmat pentru implementarea unei aplicații ce va invoca un serviciu Web pe baza unui API public:

(4) aplicația apelează funcționalitățile oferite de serviciu pentru preluarea/modificarea datelor de interes, conform politicilor de acces

sesiunea curentă va fi stabilită și menținută pe baza unor **informații de autentificare**

– uzual, stocate și vehiculate via jetoane (*auth tokens*)

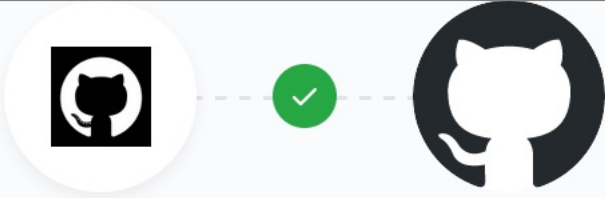
oauth

Autorizarea unei aplicații să acceseze date private
într-un mod standardizat – pe baza tehnologiilor
Web actuale – se poate realiza via **OAuth**

protocol deschis – RFC 6749

OAuth 1.0 (2010), **OAuth 2.0** (2012), OAuth 2.1 (în lucru)

oauth.net/2/



Authorize GraphQL API Explorer



GraphQL API Explorer by **github**
wants to access your **busaco** account



Discussions
Read access to Team discussions.



GPG Keys
Read access to your GPG keys.



Organizations and teams
Read-only access



Packages
Read packages from GitHub Packages. Delete private packages from GitHub Packages.



Public SSH keys
Read-only access



Repository webhooks and services
Read-only access



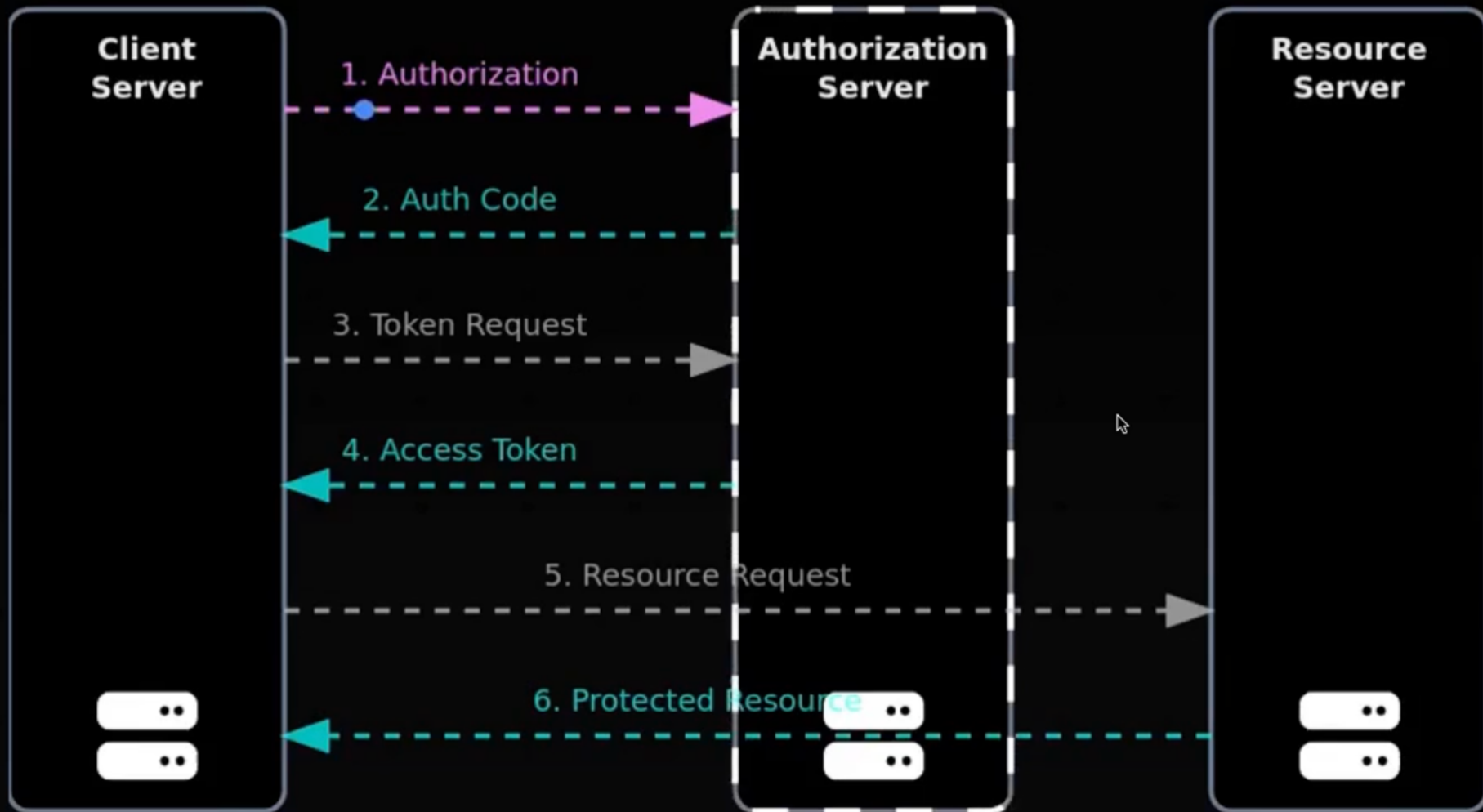
Repositories
Public and private



Personal user data
Full access

autorizarea prin **OAuth**
a aplicației Web
GraphQL API Explorer
pentru a obține accesul la datele
utilizatorului busaco
deținute de **GitHub**

■ User initiated action ■ Your/Developer's server initiated action
■ Authorization server initiated action



procesele de autorizare a accesului la o resursă Web

oauth

Biblioteci (server și/sau client) disponibile pentru
C, C++, Go, Erlang, Java, JavaScript, Objective-C, .NET,
Perl, PHP, Python, Ruby, Rust, Swift,...

oauth.net/code/

la nivel de *browser* Web:

helloJS, JSO, SalteAuth

oauth

Servicii *proxy* de autentificare/autorizare prin OAuth

Auth0 – auth0.com/docs/

Ory Hydra – www.ory.sh/hydra/

OAuth.io – github.com/oauth-io

Okta – developer.okta.com

SimpleLogin – simplelogin.io/developer/

SSQ singon – github.com/ssqsignon

oauth

Autorizzare via servizi Web specifiche – esempio:

GitHub

docs.github.com/en/developers/apps/building-oauth-apps

Google

developers.google.com/identity/protocols/oauth2

Stack Exchange

api.stackexchange.com/docs/authentication

WordPress

developer.wordpress.com/docs/oauth2/

Step 1 Select & authorize APIs

Select the **scope** for the APIs you would like to access or input your own OAuth scopes below. Then click the **"Authorize APIs"** button.

▶  Contact Center AI Platform API v1alpha1

▼  Contacts v3

<https://www.google.com/m8/feeds/>

▶  Container Analysis API v1

▶  Content API for Shopping v2.1

▶  Data Labeling API v1beta1

▶  Data Lineage API v1

▶  Data Portability API v1

▶  Data pipelines API v1

▶  Database Migration API v1

Input your own scopes

Authorize APIs

Step 2 Exchange authorization code for tokens

Step 3 Configure request to API

Request / Response

```
client_secret=...&grant_type=refresh_token&refresh_token=1%2F%2F04MYVWnB6dW0XCgYIARAAGAQSNwF-L9IrjmuJMZ8ppjjchrXUMGYnCXpZYeJ-xIfMrPNYJ7S0nUg-QjS9brBFj1mEQxXZcTCGkMo&client_id=407408718192.apps.googleusercontent.com
```

```
HTTP/1.1 200 OK
Content-length: 387
X-xss-protection: 0
X-content-type-options: nosniff
Transfer-encoding: chunked
Expires: Mon, 01 Jan 1990 00:00:00 GMT
Vary: Origin, X-Origin, Referer
Server: scaffolding on HTTPServer2
-content-encoding: gzip
Pragma: no-cache
Cache-control: no-cache, no-store, max-age=0, must-revalidate
Date: Fri, 16 May 2025 13:12:08 GMT
X-frame-options: SAMEORIGIN
Alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
Content-type: application/json; charset=utf-8
```

```
{
  "access_token": "ya29.a0AW4Xtxji9ru3u3CXro7swZm-MMehRoi7z9J8l_KE263ZFjAybpK-rx9ZjQdMELt4LMS9UrYy-60WDBS3BcWtYgAXCACRCbg8-NVLEDliqXeUQC_yYE4BMRfbbkECQaGNQhDyD3e43GKudlbGZ0vWeFPJvZQXu895wt4aCgYKAfASARASFQHGx2Mia3_SUGmGdY8LvvOiiQu2mA0175",
  "scope": "https://www.googleapis.com/auth/contacts",
  "expires_in": 3599,
  "refresh_token": "...",
  "refresh_token_expires_in": 604137
}
```

sesiunea curentă e păstrată
într-un jeton (*token*) JSON

de experimentat accesul la diverse date (e.g., persoane de contact
via Google Contacts API) pe baza **OAuth 2.0 Playground**
developers.google.com/oauthplayground/

autentificare

Metode de autentificare:

bazate pe sesiunea Web via SID (*Session Identifier*)

implicit, Web-ul este *stateless*

autentificare

Metode de autentificare:

folosind jetoane (*tokens*)

token based authentication

autentificare

Metode de autentificare:
fără parolă (*passwordless*)

one-time-use URL

exemplu: **Tumblr**

de studiat și TOTP (*Time-Based One-Time Password Algorithm*)
www.twilio.com/docs/glossary/totp • RFC 6238: tools.ietf.org/html/rfc6238

autentificare

Metode de autentificare:

SSO (Single Sign-On)

autentificarea utilizatorilor
în cadrul mai multor aplicații înrudite

autentificare

Metode de autentificare:

autentificare socială via alte conturi de utilizator

de exemplu, via rețele sociale
sau alte situri Web de încredere

autentificare

Metode de autentificare:

2FA (*Two-Factor Authentication*)

utilizatorul trebuie să ofere minim 2 probe (*factors*)

referitoare la identitatea sa:

cunoștințe ori obiecte deținute exclusiv de acea persoană

brainstation.io/cybersecurity/two-factor-auth

autenticare

Metode de autentificare:
cu cheie de acces (*passkey*)

mod de autentificare multi-factor:

*something you **know**: a PIN used to unlock the device*

*something you **have**: the authenticator*

(a security key or something embedded in a personal device)

*something you **are**: a fingerprint, a scan of user's face*

passkey.org

fidoalliance.org/passkeys/

autentificare

Metode de autentificare:

autentificare biometrică

amprentă (*fingerprint recognition*)

recunoașterea facială sau a unor organe

scanarea ochiului (iris, retină)

identificare vocală

analizarea codului genetic (*DNA matching*)

www.biometricsinstitute.org

autentificare

Metode de autentificare:

autentificare via dispozitiv hardware

exemple tipice:

smartcard

dispozitiv personal (*e.g.*, ceas) via interacțiuni fără fir

autentificare

Metode de autentificare – privire de ansamblu:

bazate pe sesiunea Web
folosind jetoane (*token based authentication*)
fără parolă (*passwordless*)
SSO (*Single Sign-On*)
autentificare socială
2FA (*Two-Factor Authentication*)
cu chei de acces (*passkey*)
autentificare biometrică
autentificare via dispozitiv hardware

detalii în *The Copenhagen Book* (2025): thecopenhagenbook.com

autentificare

În contextul aplicațiilor Web, a se considera
*Web Authentication: An API for accessing Public Key
Credentials – Level 2* (W3C Recommendation, apr. 2021)

www.w3.org/TR/webauthn-2/

ghid de utilizare:
webauthn.guide

cod demonstrativ:
webauthn.io

autentificare

Observație:

există cazuri în care o aplicație Web nu necesită managementul utilizatorilor și autentificarea acestora

exemplificări reale:

Awesome (free) Web apps that work without login

github.com/aviaryan/awesome-no-login-web-apps

autentificare

OpenID

manieră descentralizată de autentificare a utilizatorului
la nivel de Web pe baza paradigmei SSO

autentificare

OpenID

manieră descentralizată de autentificare a utilizatorului
la nivel de Web pe baza paradigmei SSO

utilizatorul poate demonstra că deține un URL specific
menit a-l identifica *on-line* via un ofertant (serviciu)
de identitate digitală (*identity provider*)
e.g., folosind o aplicație Web socială

openid

Fiecare identitate a unui utilizator e desemnată de un URL
(stabilit de către ofertantul de identitate digitală
– *identity provider*)

exemplu: steamcommunity.com/openid/id/steamid

pentru a-și confirma identitatea,
utilizatorul va trebui să se autentifice

openid

OpenID Connect

oferă un nivel vizând identitatea utilizatorului
(*identity layer*) pe baza protocolului OAuth 2

formatul de date folosit: *JWT – JSON Web Token*
standardizat de RFC 7519

openid.net/connect/

Un jeton (*token*) **JWT** include un obiect JSON folosit pentru interschimb „sigur” de date între două entități
– uzual, un client ce invocă un serviciu Web (API)

Un jeton (*token*) **JWT** include un obiect JSON folosit pentru interschimb „sigur” de date între două entități – uzual, un client ce invocă un serviciu Web (API)

cazuri concrete:

autentificare

identificarea utilizatorului după autentificarea cu succes se realizează via un jeton JWT (*ID token*) a cărui structură este precizată de specificația OpenID Connect

openid.net/specs/openid-connect-core-1_0.html#IDToken

Un jeton (*token*) **JWT** include un obiect JSON folosit pentru interschimb „sigur” de date între două entități – uzual, un client ce invocă un serviciu Web (API)

cazuri concrete:

autorizare

fiecare cerere a clientului autorizat pentru a invoca o funcționalitate a serviciului Web va include jetonul JWT corespunzător (în mod similar sesiunilor Web)

Un jeton (*token*) **JWT** include un obiect JSON folosit pentru interschimb „sigur” de date între două entități – uzual, un client ce invocă un serviciu Web (API)

cazuri concrete:

interschimb de informații

jetoanele JWT pot fi utilizate pentru asigurarea integrității schimbului de date (*e.g.*, detectarea tentativelor de modificare neautorizată a informațiilor de către terțe entități)

Un jeton (*token*) **JWT** include un obiect JSON folosit pentru interschimb „sigur” de date între două entități – uzual, un client ce invocă un serviciu Web (API)

datele vehiculate pot fi verificate și considerate de încredere via semnături digitale

Un jeton (*token*) **JWT** include un obiect JSON folosit pentru interschimb „sigur” de date între două entități – uzual, un client ce invocă un serviciu Web (API)

datele vehiculate pot fi verificate și considerate de încredere via semnături digitale

se folosesc diverși algoritmi de tip *digest* pentru a se asigura autenticitatea mesajelor (HMAC – *Hash-based Message Authentication Code*) în conjuncție cu perechi de chei criptografice publice/private

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJ0dXh5LnBpbmd1aW5uZXNzY29vbCI6ImV4cCI6MTcxNjIzOTAwMCwiaHR0cHM6Ly93ZWUiOiJhW5mby8jaXNfcm9vdCI6dHJ1ZX0.oKxec2WkzQG-z-rzEkyLXLMh-6N_XFWoVq69X_uSn8I

Issuer (who created and signed this token)

mesaje – de autentificare sau
interschimb de informații
de autorizare (*credentials*) –
vehiculate în format **JWT**
pe baza unui algoritm criptografic

resurse + instrumente: jwt.io

HEADER: ALGORITHM & TOKEN TYPE

alg. + tip de jeton

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

date propriu-zise

```
{
  "iss": "tuxy.penguinnesscool",
  "exp": 1716239000,
  "https://web.info/#is_root": true
}
```

VERIFY SIGNATURE

info de verif.
a semnăturii

```

    HMACSHA256(
        base64UrlEncode(header) + "." +
        base64UrlEncode(payload),
        your-256-bit-secret
    )

```

Antetul (*header*) descrie operațiunile criptografice aplicate
unui jeton JWT conform specificației JOSE
(*JSON Object Signing and Encryption*)

RFC 7520 – jose.readthedocs.io

Antetul (*header*) descrie operațiile criptografice aplicate unui jeton JWT conform specificației JOSE (*JSON Object Signing and Encryption*)

parametru de interes:

typ (*type*) – tipul de date al jetonului (**jwt**)

Antetul (*header*) descrie operațiile criptografice aplicate unui jeton JWT conform specificației JOSE (*JSON Object Signing and Encryption*)

parametru de interes:

alg (*algorithm*) – algoritmul folosit pentru securizarea jetonului conform RFC 7518 (*JSON Web Algorithms*)

exemple:

HS512 (*HMAC using Secure Hash Alg. SHA-512*)

ES256 (*Elliptic Curve Digital Signature Algorithm: ECDSA using P-256 and SHA-256*)

Datele propriu-zise (*payload*) ale jetonului JWT specifică
un set de declarații (asertiuni, *claims*)
ale căror valori depind de contextul utilizării aceluși jeton

predefinite (*registered*)
specifice (*custom*)

obligatorii sau opționale

publice ori private

Nume predefinite de proprietăți JWT (*registered claims*)

iss (*issuer*): entitatea care definește jetonul

sub (*subject*): subiectul declarațiilor JWT (*e.g.*, utilizatorul)

aud (*audience*): entitatea căreia i se adresează jetonul

exp (*expiration date*): data expirării jetonului

iat (*issued at time*): timpul de emitere a jetonului

jti (JWT ID): identificator unic
(asigură o unică folosire a jetonului)

Validarea jetonului JWT (este „de încredere”)

trebuie verificată validitatea înainte de a fi stocat și utilizat conform unui algoritm de semnare digitală

detalii în specificațiile

RFC 7515 (*JSON Web Signature*)

www.rfc-editor.org/info/rfc7515

RFC 7516 (*JSON Web Encryption*)

www.rfc-editor.org/info/rfc7516

Aspecte vizând securitatea:

JSON Web Token Vulnerabilities

0xn3va.gitbook.io/cheat-sheets/web-application/json-web-token-vulnerabilities

JWT attacks

portswigger.net/web-security/jwt



Alternative privitoare la REST?

model de acces la date: **graphql**

Graph Query Language

“a query language for APIs and a runtime for fulfilling those queries with your existing data”

graphql.org

sub jurisdicția *Linux Foundation*
gql.foundation

model de acces la date: graphql

Graph Query Language

declarativ

inspirat de JSON

strict (*strong-typed*)

model de acces la date: graphql

Graph Query Language

sunt permise interogări (*queries* – obiecte de tip **Query**),
desemnând operațiile de citire,
și actualizări (*mutations* – obiecte de tip **Mutation**),
adică operații de alterare a datelor

graphql.org/learn/queries/

model de acces la date: **graphql**

Graph Query Language

se oferă suport pentru a anticipa ce date
vor fi întoarse + structura acestora (*types + schemas*)

graphql.org/learn/schema/

model de acces la date: **graphql**

Graph Query Language

relațiile dintre diverse (categorii de) resurse
se pot specifica via legături – conexiuni (*connections*)

datele sunt modelate ca grafuri – resurse hipermedia

graphql.org/learn/thinking-in-graphs/

model de acces la date: graphql

Graph Query Language

răspunsul oferit include doar datele ce au fost solicitate

- ▶ îmbunătățirea performanței la nivel de client

model de acces la date: graphql

Graph Query Language

răspunsul oferit include doar datele ce au fost solicitate
► îmbunătățirea performanței la nivel de client

rezolvarea problemelor vizând *over/under fetching*
(preluare a mai multor sau prea puține date)

nordicapis.com/is-graphql-the-end-of-rest-style-apis/
hasura.io/blog/rest-view-of-graphql/
phil.tech/2017/graphql-vs-rest-overview/

	GraphQL	REST
entitate	resursă	resursă
format	JSON	orice <i>Media Type</i> (MIME) frecvent: JSON
protocol	HTTP – uzual, adoptă convenții proprii	independent de protocol (uzual, HTTP)
cine decide ce date vor fi întoarse	clientul	serverul
puncte terminale (<i>endpoints</i>)	un singur punct terminal pentru a oferi date conexe, dacă au fost specificate relații între ele	puncte terminale multiple (independente)
tipuri de date	<i>strong</i> (tipuri declarate explicit)	<i>weak</i> (verificarea tipurilor de date nu e obligatorie)
complexitate	<i>fat client—fat server</i>	<i>thin client—fat server</i>
documentare	autodescriptiv (<i>self-describing</i>)	necesită terțe soluții (<i>e.g.</i> , OpenAPI Specification)
viziune	limbaj de interogare, specificație, colecție de instrumente	stil arhitectural

GraphQL ca alternativă la dezvoltarea de servicii via paradigma REST

model de acces la date: graphql

Aspecte de considerat:

probleme de securitate

– *e.g.*, autentificare, autorizare, refuz al serviciilor

suportul pentru *caching* — uzual, oferit explicit

managementul versiunilor

blog.pusher.com/rest-versus-graphql/

aws.amazon.com/compare/the-difference-between-graphql-and-rest/

model de acces la date: **graphql**

Instrumente pentru dezvoltatori:

implementare de referință pentru server (Node.js)

GraphQL.js

graphql.org/graphql-js/

www.apollographql.com/blog/how-to-use-graphql-with-javascript-graphql-js-tutorial/

biblioteci disponibile pentru C, Go, Java, Kotlin,
.NET, PHP, Python, Swift etc. – graphql.org/code/

model de acces la date: graphql

Instrumente pentru dezvoltatori:

editoare Web interactive

Altair GraphQL Client – altairgraphql.dev

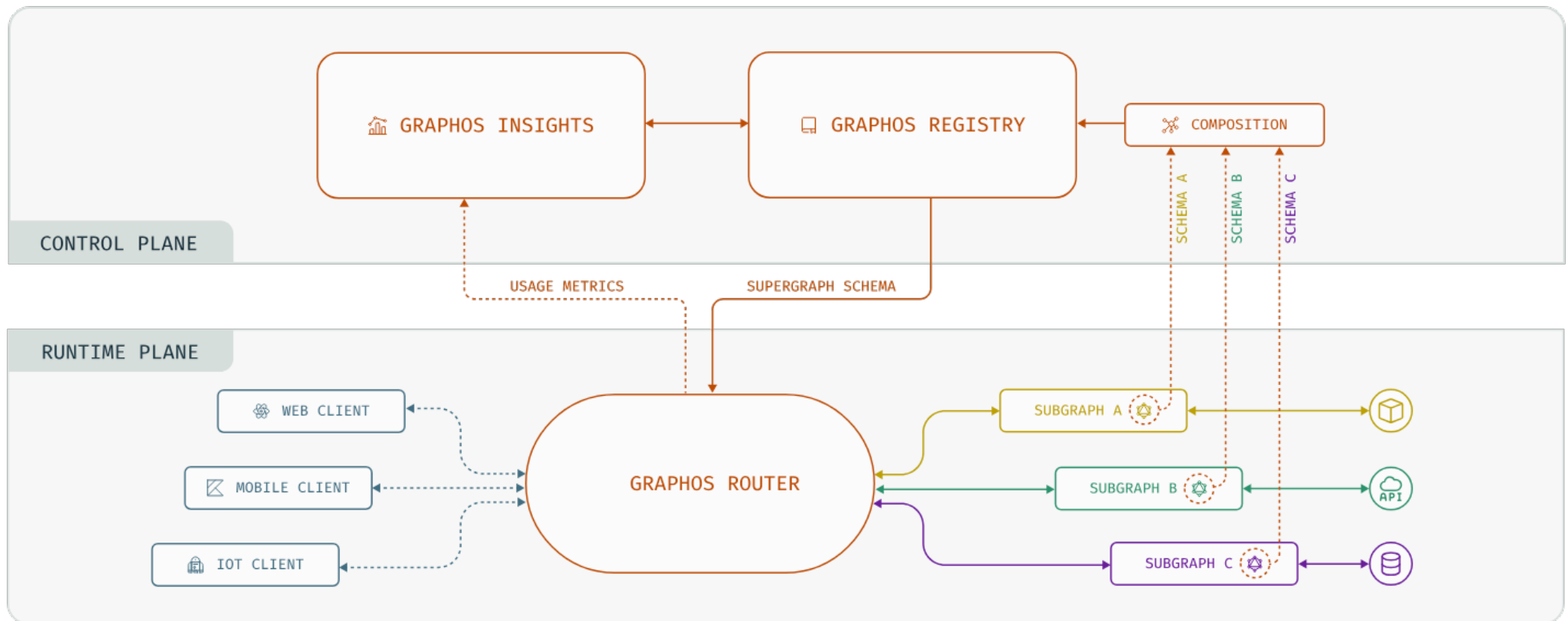
GraphiQL – github.com/graphql/graphiql

GraphQL Editor – graphqleditor.com

model de acces la date: **graphql**

Instrumente pentru dezvoltatori:

Apollo – www.apollographql.com/docs/



model de acces la date: graphql

Instrumente pentru dezvoltatori:

Apollo – www.apollographql.com/docs/

oferă suport la nivel de client (*front-end*) via JS/TypeScript:
biblioteca Apollo Client + Angular, Relay, Vue, Svelte etc.

componente Web specifice: apollo-elements

soluții native pentru Android (Kotlin) și iOS (Swift)

model de acces la date: graphql

Instrumente pentru dezvoltatori:

GraphQL Yoga 2 – the-guild.dev/graphql/yoga-server/docs

server GraphQL bazat pe specificația *GraphQL over HTTP*
(*e.g.*, suport pentru Fetch API, SSE – *Server-Sent Events*)

model de acces la date: **graphql**

Alte servere + *framework*-uri:

API Platform Framework (PHP) – api-platform.com

graphql-php (PHP) – webonyx.github.io/graphql-php/

Graphene (Python) – graphene-python.org

GraphQL Java – www.graphql-java.com

GraphQL Kotlin – github.com/ExpediaGroup/graphql-kotlin

GraphQL for .NET – github.com/graphql-dotnet/

gqlgen (Go) – gqlgen.com

Hot Chocolate (.NET) – chillicream.com/docs/hotchocolate

GraphiQL

Prettify History Explorer

interogare

```
1 query {
2   viewer {
3     name
4     createdAt
5     repositories (first: 1) {
6       nodes {
7         description
8         primaryLanguage {
9           name
10        }
11      }
12    }
13  }
14 }
```

Language.name: String!
The name of the current language.

rezultate

```
{
  "data": {
    "viewer": {
      "name": "Sabin Buraga",
      "createdAt": "2015-04-03T09:57:23Z",
      "repositories": {
        "nodes": [
          {
            "description": "Demo App for the DBMSs Ontology
            databases-ontology)",
            "primaryLanguage": {
              "name": "JavaScript"
            }
          }
        ]
      }
    }
  }
}
```

testarea interactivă cu instrumentul Web **GraphiQL**
a API-ului GitHub via GraphQL: docs.github.com/en/graphql

< User

RepositoryOwner

Represents an owner of a Repository.

schema

FIELDS

avatarUrl(size: Int): URI!

A URL pointing to the owner's public avatar.

id: ID!

login: String!

The username used to login.

repositories(

privacy: RepositoryPrivacy

orderBy: RepositoryOrder

affiliations: [RepositoryAffiliation]

ownerAffiliations: [RepositoryAffiliation] = [OWNER, COLLABORATOR]

isLocked: Boolean

after: String

before: String

first: Int

last: Int

isFork: Boolean

): RepositoryConnection!

A list of repositories that the user owns.

concepte definite prin
scheme de date ce pot
recurge la tipuri scalare
(Boolean, Int, String,...) sau
specificate de programator
– în acest caz, de pildă
URI, User, RepositoryOwner

[] specifică tablouri
(*arrays*) de obiecte

! indică date *non-nullable*
(vor fi oferite întotdeauna
valori nenule)

```

query {
  viewer {
    name
  }
}

```

1: cerere GraphQL formulată de client

name
__typename
twitterUsername
sponsorshipsAsMaintainer
organizationVerifiedDomainEmails
String The user's public profile name.

▼ POST

Scheme: https

Host: graphql.github.com

Filename: /graphql/proxy

Address: 34.203.109.182:443

2: trimiterea cererii prin POST

Status	200 OK ?
Version	HTTP/1.1
Transferred	1.53 kB (78 B size)

Status	Method	Domain	File	Initiator	Type	Transferred	Size
200	POST	graphql.github.com	proxy	graphql-explorer-f2ef65f56c75783cdd28.js:2 (fetch)	json	1.53 kB	78 B

```

{ "query": "query { \n  viewer { \n    name\n    createdAt\n  }\n}",
  "variables": {}
}

```

3: date JSON expediate serverului

```

{ "data": {
  "viewer": {
    "name": "Sabin Buraga",
    "createdAt": "2015-04-03T09:57:23Z"  }}
}

```

4: răspunsul oferit în format JSON

```
{
  "errors": [
    {
      "path": [ ... ],
      "extensions": {
        "code": "variableNotDefined", "variableName": "num"
      },
      "locations": [ {
        "line": 5,
        "column": 26
      } ],
      "message":
        "Variable $num is used by anonymous query but not declared"
    }
  ]
}
```

posibilă manieră de semnalare a erorilor/exceptiilor

```
1 {
2   country(code: "RO") {
3     capital
4     currency
5     emoji
6     languages {
7       name
8       native
9     }
10  continent {
11    name
12  }
13 }
14 }
```

name String!

interogare

Variables Headers

+ countries.trevorblades.com

```
{
  "data": {
    "country": {
      "capital": "Bucharest",
      "currency": "RON",
      "emoji": "🇷🇴",
      "languages": [
        {
          "name": "Romanian",
          "native": "Română"
        }
      ],
      "continent": {
        "name": "Europe"
      }
    }
  }
}
```

rezultate

MISS 267ms

country(
 code: ID!
): Country

schema

TYPE DETAILS

type Country {

 code: ID!

 name: String!

 native: String!

 phone: String!

 continent: Continent!

 capital: String

 currency: String

 languages: [Language!]!

 emoji: String!

 emojiU: String!

 states: [State!]!

}

informații despre țări
oferite de un API GraphQL:
countries.trevorblades.com

[CHANGE SCHEMA](#)
[< Type List](#)
[Business](#)

No Description

FIELDS

name: String
 id: String
 is_claimed: Boolean
 is_closed: Boolean
 url: String
 phone: String
 display_phone: String
 review_count: Int
 categories: [Category]
 rating: Float
 location: Location
 coordinates: Coordinates
 photos: [String]
 hours: [Hours]
 reviews: [Review]

Query:business

Businesses:business

Business:categories

Business:location

Business:coordinates

Business:hours

Business:reviews

Businesses
business [Business]
total Int

Reviews
review [Review]
total Int

Business
name String
id String
is_claimed Boolean
is_closed Boolean
url String
phone String
display_phone String
review_count Int
categories [Category]
rating Float
location Location
coordinates Coordinates
photos [String]
hours [Hours]
reviews [Review]

Category
title String
alias String

Location
address1 String
address2 String
address3 String
city String
state String
zip_code String
country String
formatted_address String

Coordinates
latitude Float
longitude Float

Hours
hours_type String
open [OpenHours]
is_open_now Boolean

Review
rating Int
user User
text String
time_created String
url String

Query
☐ Sort by Alphabet ☒ Skip Relay ☒ Skip deprecated ☒ Show leaf fields

 Powered by GraphQL Voyager

explorarea unui API GraphQL

– aici, Yelp API – via instrumentul Web GraphQL Voyager

apis.guru/graphql-voyager/

Status	Method	File	Initiator	Type	Transferred	Size
304	GET	tags.json	main-8a26b4a660344706.js:1 (fetch)	json	cached	24.22 kB
200	GET	i?stm=1741672572539&e=pp&url=	_app-ffe7ac3dcba3a84d.js:1 (fetch)	gif	705 B	43 B
200	GET	i?stm=1741672582544&e=pp&url=	_app-ffe7ac3dcba3a84d.js:1 (fetch)	gif	705 B	43 B
200	POST	graphql?consumer_key=94110	_app-ffe7ac3dcba3a84d.js:1 (fetch)	json	2.21 kB	41 B
200	GET	i?stm=1741672583644&e=ue&eid=	_app-ffe7ac3dcba3a84d.js:1 (fetch)	gif	705 B	43 B

Headers

Cookies

Request

Response

Filter Request Parameters

JSON

Raw ☐

```

operationName: "ItemDelete"
query: "\n mutation ItemDelete($itemId: ID!) {\n
      deleteSavedItem(id: $itemId)\n
    }\n"
variables: {...}
      itemId: "3148910115"

```

caz concret:

Pocket – o operație de eliminare (ștergere)
a unei adrese Web favorite (*bookmark*) via GraphQL

Alte exemplificări concrete:

Apache APISIX – *a dynamic, real-time, high-performance API gateway*

apisix.apache.org/blog/2022/03/02/apisix-integration-graphql/

AWS AppSync – sincronizare în timp-real a datelor
via servicii Amazon disponibile „în nori”

docs.aws.amazon.com/appsync/latest/devguide/

GitLab – soluție de gestiune a codului-sursă

docs.gitlab.com/ee/api/graphql/

Instagram – vezi prezentarea lui H. Nasser

How Instagram content is served through GraphQL (2022)

www.youtube.com/watch?v=rjZpGc9xWIM

Alte exemplificări concrete:

GraphQL is the definitive API for Shopify (2024)

www.shopify.com/partners/blog/all-in-on-graphql

Khan Academy – de la aplicația monolitică Python 2 la Goliath – un sistem modern implementat în limbajul Go și bazat pe servicii Web și GraphQL (via Apollo)

blog.quastor.org/p/khan-academy-rewrote-backend

Netflix – dezvoltare cu micro-servicii adoptând GraphQL

netflixtechblog.com/beyond-rest-1b76f7c20ef6

Product Hunt – recomandare de produse via GraphQL

api.producthunt.com/v2/docs

Yelp – acces la recenzii de produse/servicii

docs.developer.yelp.com/docs/graphql-intro

graphql: resurse + studii de caz

The Fullstack Tutorial for GraphQL

www.howtographql.com

GraphQL news

graphqleditor.com/blog/

Awesome GraphQL

github.com/chentsulin/awesome-graphql

GraphQL APIs

github.com/APIs-guru/graphql-apis

Alte abordări arhitecturale?



Existența (micro-)serviciilor și a API-urilor REST/GraphQL
– eventual, invocate asincron via Ajax –
oferă premisele dezvoltării de aplicații Web de tip:

SPA (*Single-Page Application*)

PWA (*Progressive Web Application*)

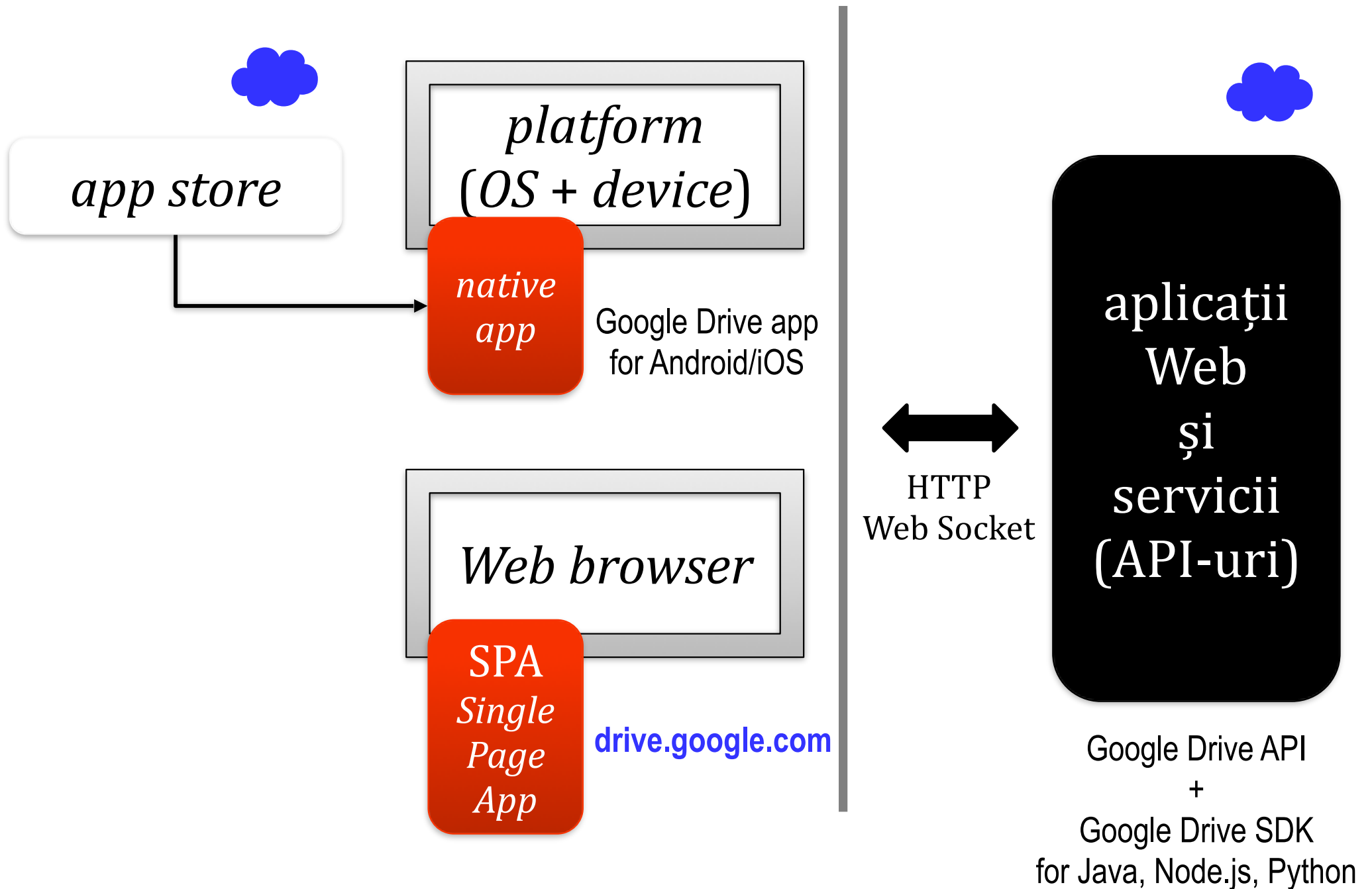
hibrid (*Web mash-up*)

joc Web – *e.g.*, MMORPG
(*Massively Multiplayer Online Role-Playing Game*)

detalii în prelegerea materiei „Tehnologii Web” (2025)
edu.info.uaic.ro/web-technologies/web-film.html#week14

spa

Rescrierea dinamică a conținutului paginii Web
– pe baza transferului asincron al datelor –
în urma interacțiunii cu utilizatorul



adaptare după Adrian Colyer

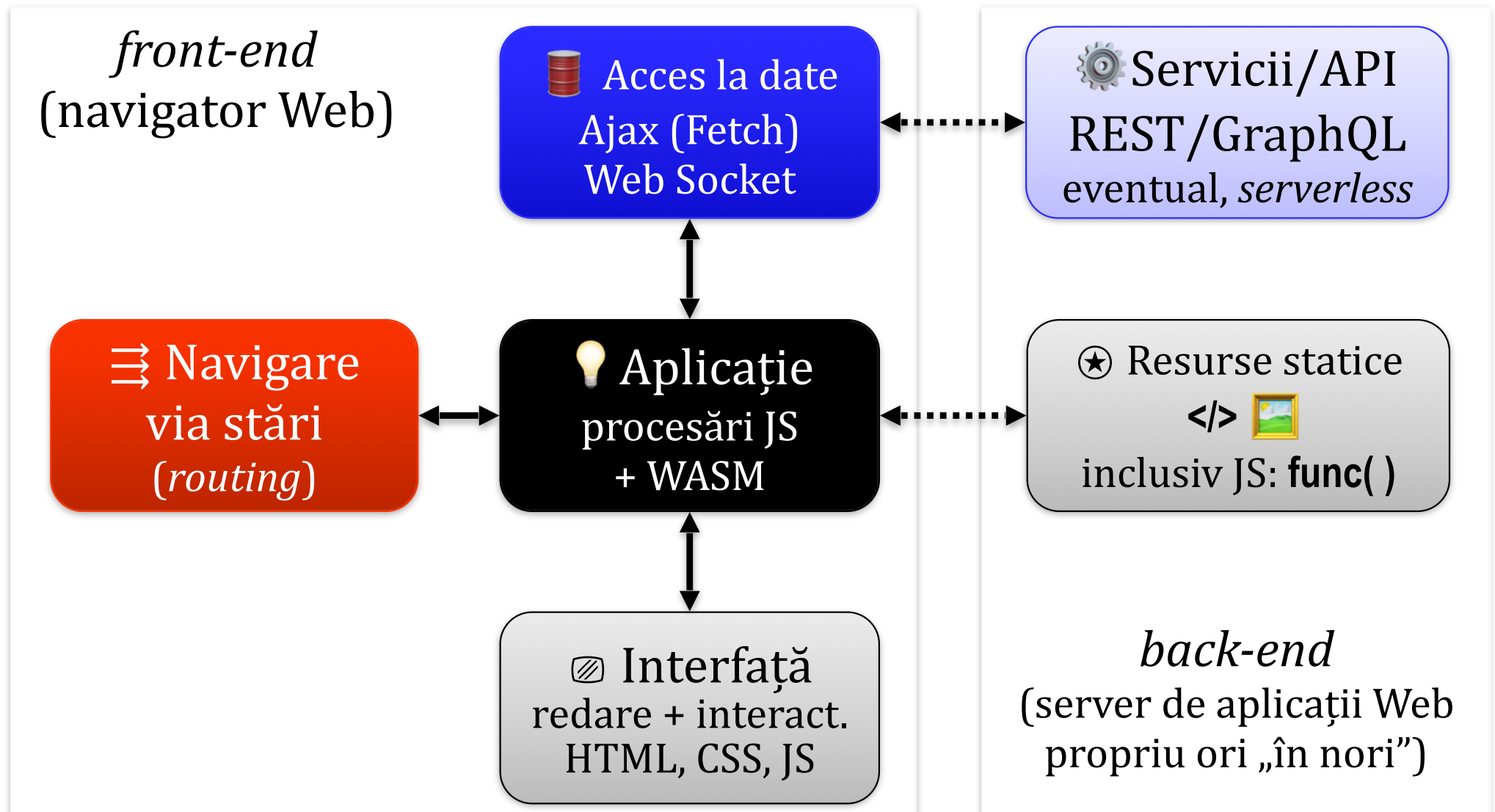
spa

Aplicația va fi disponibilă la același URL de bază, recurgând la un unic document HTML, conținutul fiind redat în aceeași fereastră a navigatorului Web

contrast cu abordările tradiționale
MPA (*Multi-Page Application*)

componente majore ale unei SPA

(adaptare după Patrick Ackerman)



pwa

Progressive Web Application

Aplicații Web capabile să ofere funcționalități adaptate la contextul de execuție

pwa

Disponibile pe diverse platforme la nivel de navigator Web

recurg la diverse API-uri oferite de *browser*
(ultima actualizare: 16 octombrie 2025)

html-now.github.io

pwa

Resurse de interes:

developer.mozilla.org/Web/Progressive_web_apps

web.dev/progressive-web-apps/

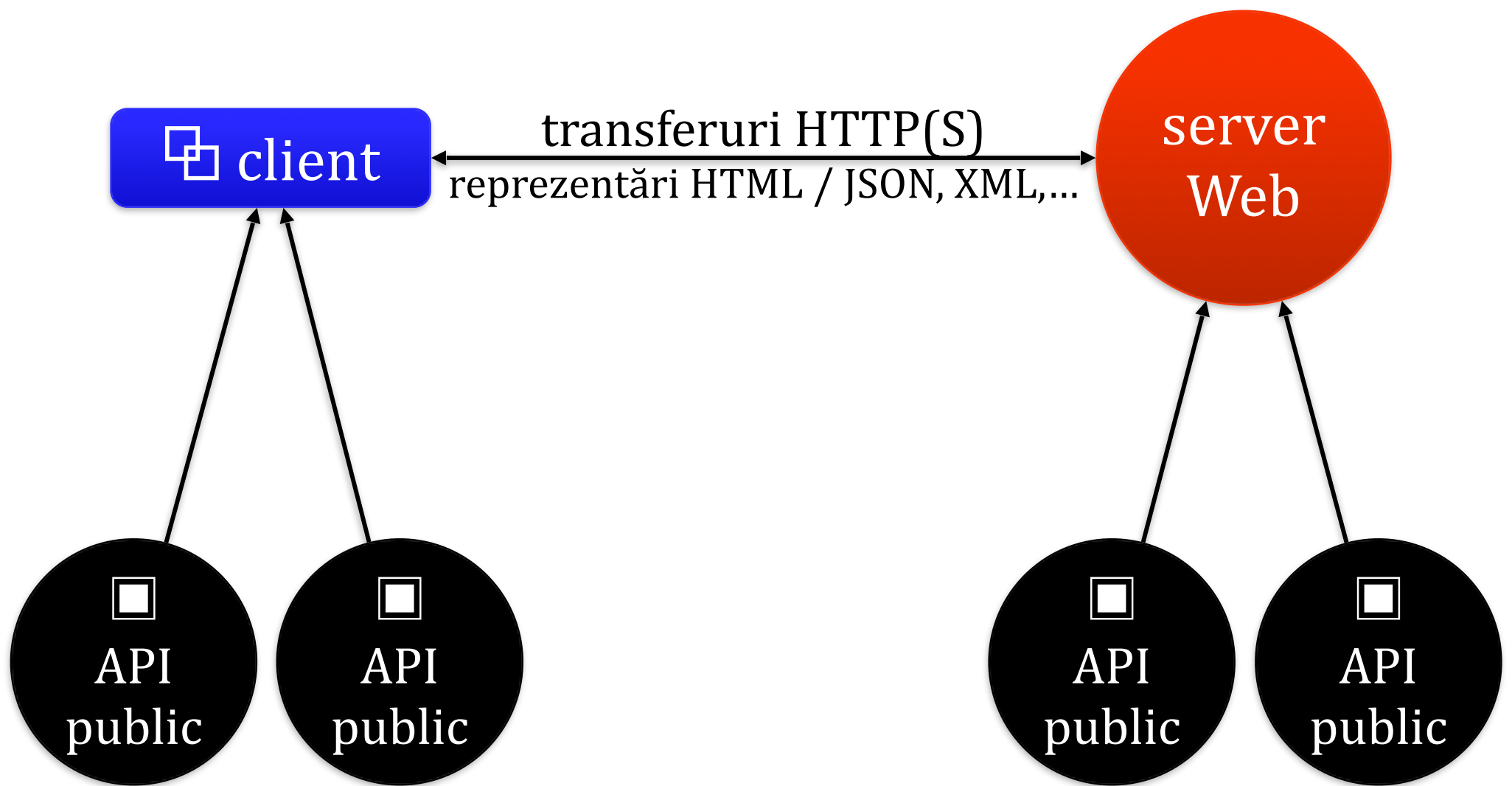
www.pwastats.com

mash-ups

Aplicații Web hibride

combinarea – la nivel de client și/sau server – a datelor ce provin din surse (situri) multiple, oferindu-se o funcționalitate/experiență nouă

conform „curentului” *SaaS* (*Software As A Service*)



un *mash-up* Web

procesează și redă datele la nivel de client și/sau server



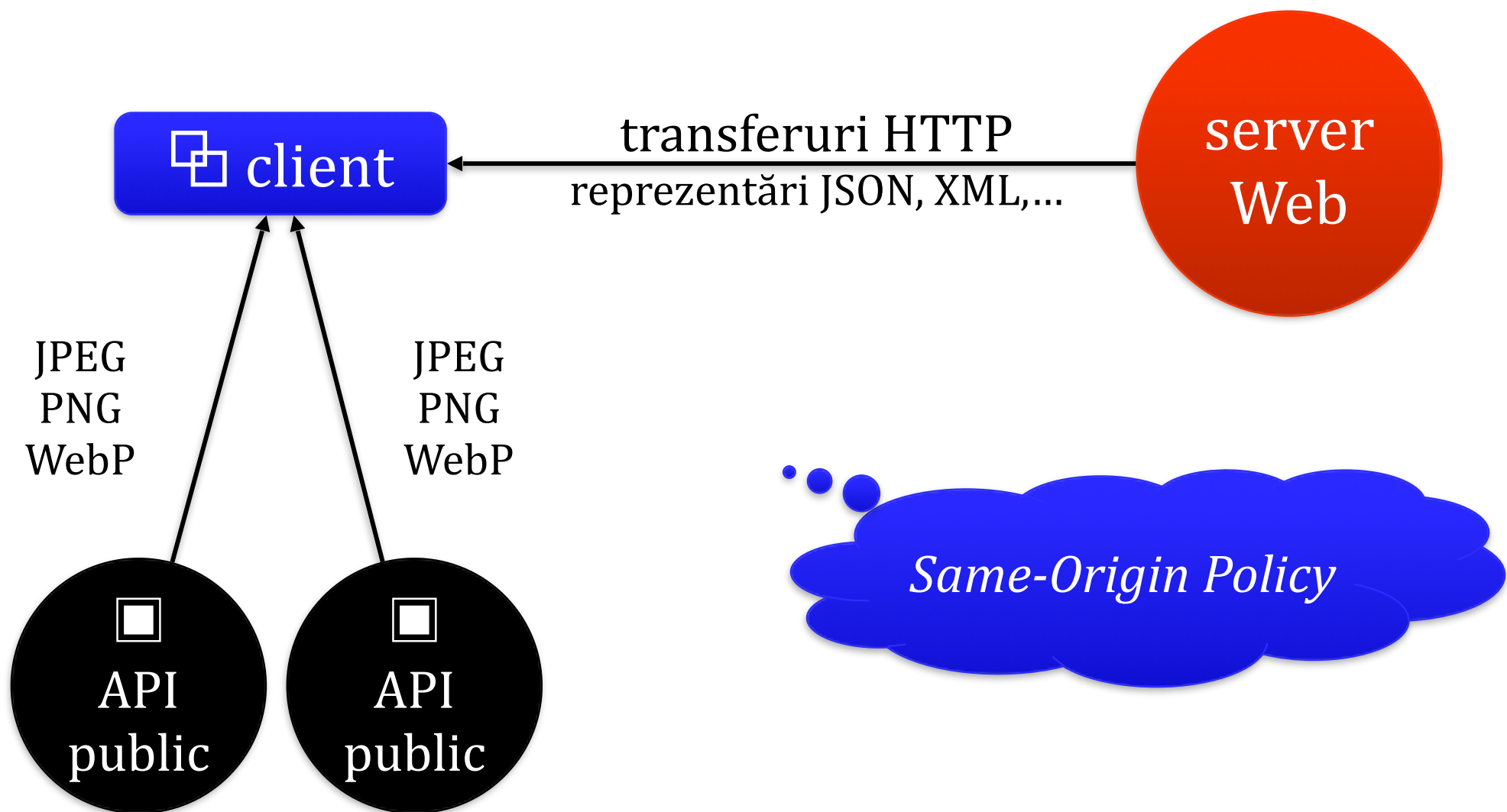
Probleme de securitate privind accesul
din *browser* la resurse via JavaScript?

cors

Same-Origin Security Policy

“restricts how a document or script loaded from one origin can interact with a resource from another origin”

astfel, un program JavaScript trebuie să acceseze
doar datele aparținând aceleiași origini
– *i.e.*, provenite din același domeniu Internet



se permit doar transferuri vizând reprezentări de resurse referitoare la imagini (image/*), fișiere CSS și alte programe JavaScript aparținând aceleași origini

cors

Same-Origin Security Policy

previne cazurile în care un document/program încărcat dintr-o origine să poată accesa/modifica proprietăți ale unui document aparținând altei origini

developer.mozilla.org/Web/Security/Same-origin_policy

cors

CORS (*Cross-Origin Resource Sharing*)

permite partajarea la nivel de client a resurselor provenind din **origini** (*e.g.*, domenii Internet) diferite

astfel, se pot emite cereri între origini (*cross-origin*)

Fetch Living Standard – CORS protocol (10 oct. 2025)

fetch.spec.whatwg.org/#cors-protocol

cors

Politicile de partajare a resurselor se bazează pe anumite câmpuri din antetul mesajelor HTTP

exemplu (se permit doar cereri GET având în antetul mesajului câmpuri specifice din partea unui client aparținând unei origini date):

Access-Control-Request-Method: GET

Access-Control-Request-Headers: Content-Type, Accept

Origin: <http://localhost:8000>

cors

Politicile de partajare a resurselor se bazează pe anumite câmpuri din antetul mesajelor HTTP

resurse de interes:

developer.mozilla.org/docs/Web/HTTP/CORS

de studiat și tutorialul lui Steve Hobbs (2019):

auth0.com/blog/cors-tutorial-a-guide-to-cross-origin-resource-sharing/

cors: caz real

Google Play acceptă cereri POST emise numai de serviciul Google Drive (originea este <https://drive.google.com>) ce includ câmpuri-antet vizând autorizarea

Access-Control-Request-Method: POST

Origin: <https://drive.google.com>

Access-Control-Request-Headers: authorization,x-goog-authuser



câmp nestandardizat

cors: caz real

Reddit.com permite cereri POST și OPTIONS formulate de orice origine, fiind acceptate anumite câmpuri-antet (controlul accesului depinde de timp)

Access-Control-Allow-Headers: Content-Type, Origin, Accept, X-Signature

Access-Control-Allow-Methods: POST, OPTIONS

Access-Control-Allow-Origin: *

Access-Control-Max-age: 1728000

JAMstack

Manieră modernă privind dezvoltarea de
aplicații Web la nivel de client

*“A modern Web development architecture based on
client-side JavaScript, reusable APIs, and prebuilt Markup.”*

Mathias Biilmann

viziune:

jamstack.org

JAMstack

Manieră modernă privind dezvoltarea de aplicații Web la nivel de client

JavaScript

procesările efective sunt realizate de client
(uzual, un *browser* Web) pe baza facilităților oferite de limbajul de programare JS și ecosistemului aferent

JAMstack

Manieră modernă privind dezvoltarea de aplicații Web la nivel de client

APIs

funcționalitățile și datele sunt încapsulate de **API-uri reutilizabile** – proprii sau oferite de terțe entități – accesate
(a)sincron prin HTTPS via programe JS

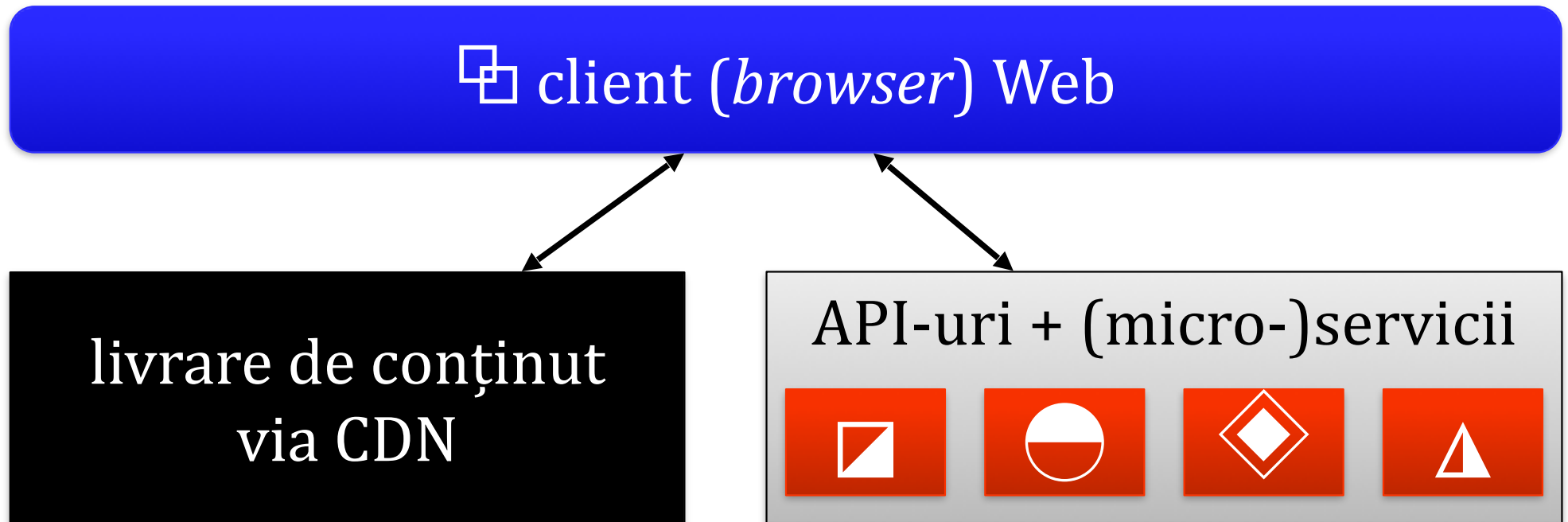
JAMstack

Manieră modernă privind dezvoltarea de aplicații Web la nivel de client

Mark-up

livrarea conținutului via machete de vizualizare (*templates*) definite de limbaje de marcare, pre-procesate în vederea prezentării la client (*pre-rendering*) și stocate redundant de CDN (*Content Distribution Network*), uzual pe baza unor instrumente specifice de tip SSG (*Static Site Generator*) ori *headless* CMS

JAMstack



resurse + instrumente:

github.com/automata/awesome-jamstack

de studiat și Mathias Biilmann & Phil Hawksworth,
Modern Web Development on the Jamstack, O'Reilly, 2019

www.netlify.com/oreilly-jamstack/



Câteva exemplificări privind arhitectura unor aplicații Web?

a se studia și resursele Web disponibile la
edu.info.uaic.ro/web-technologies/web-film.html#week4

studiu de caz: instacart

Scop: aplicație Web multi-platformă oferind servicii de livrare rapidă la domiciliu a produselor de băcănie 🥕🥦🍖🍞

produse: 500 de milioane
magazine locale (ofertanți): 40 de mii
localități: 5500 (US + Canada)
comenzi: milioane/an

stackshare.io/instacart/instacart/

studiu de caz: instacart



Python



React



NGINX



PostgreSQL



Ubuntu



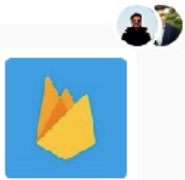
Redis



Amazon S3



Amazon EC2



Firebase



Ruby



Android SDK



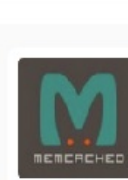
Amazon
CloudFront



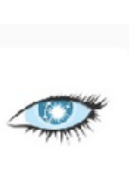
Rails



Objective-C



Memcached



Cassandra



R Language



Amazon
ElastiCache



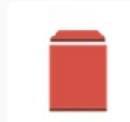
Pandas



NumPy



Amazon RDS for
PostgreSQL



Amazon EBS



Unicorn



Snowflake

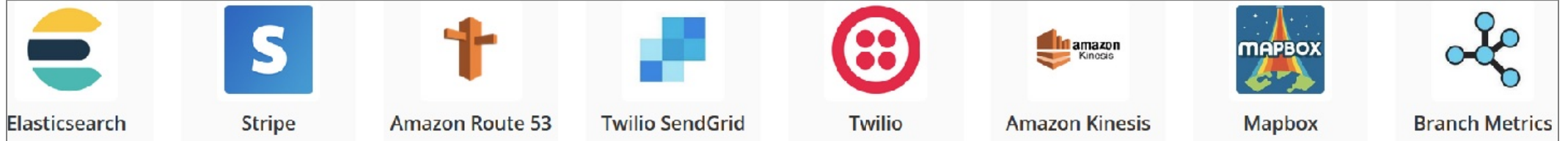


Druid

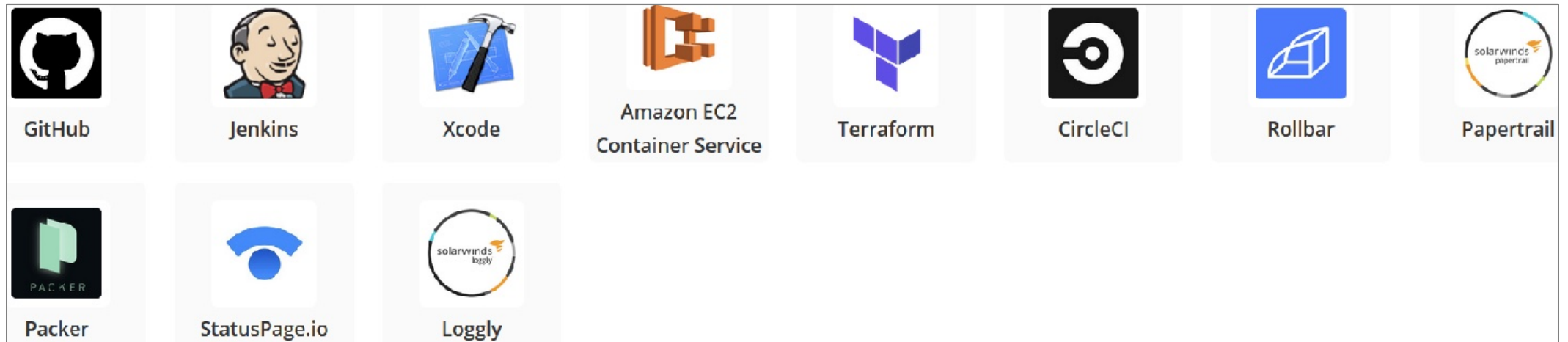


Domino

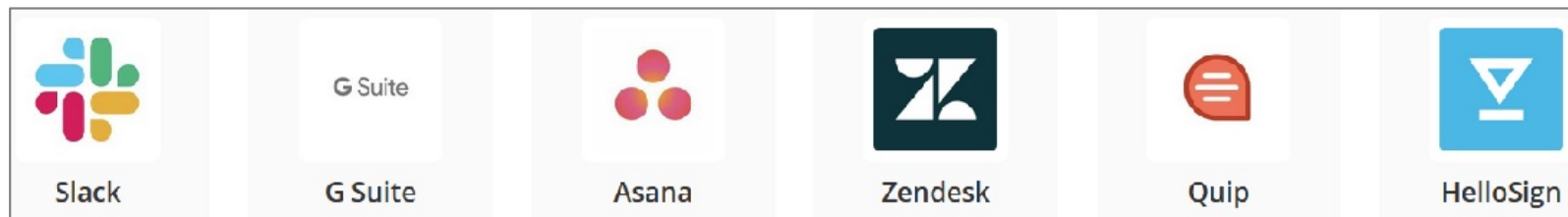
tehnologii + instrumente principale – *application & data*



instrumente adiționale, utilitare – *utilities*



instrumente pentru operațiuni de dezvoltare – *DevOps*



processe interne vizând afacerile – *business tools*

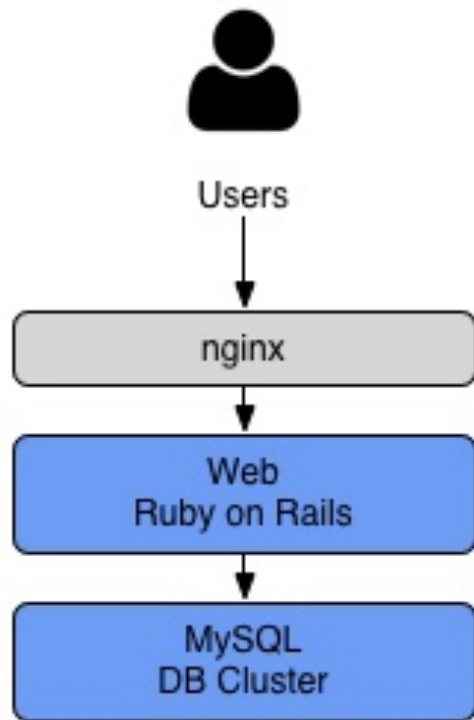
studiu de caz: **groupon**

Scop: prezentarea și comercializarea de oferte
de bunuri de consum + servicii diverse

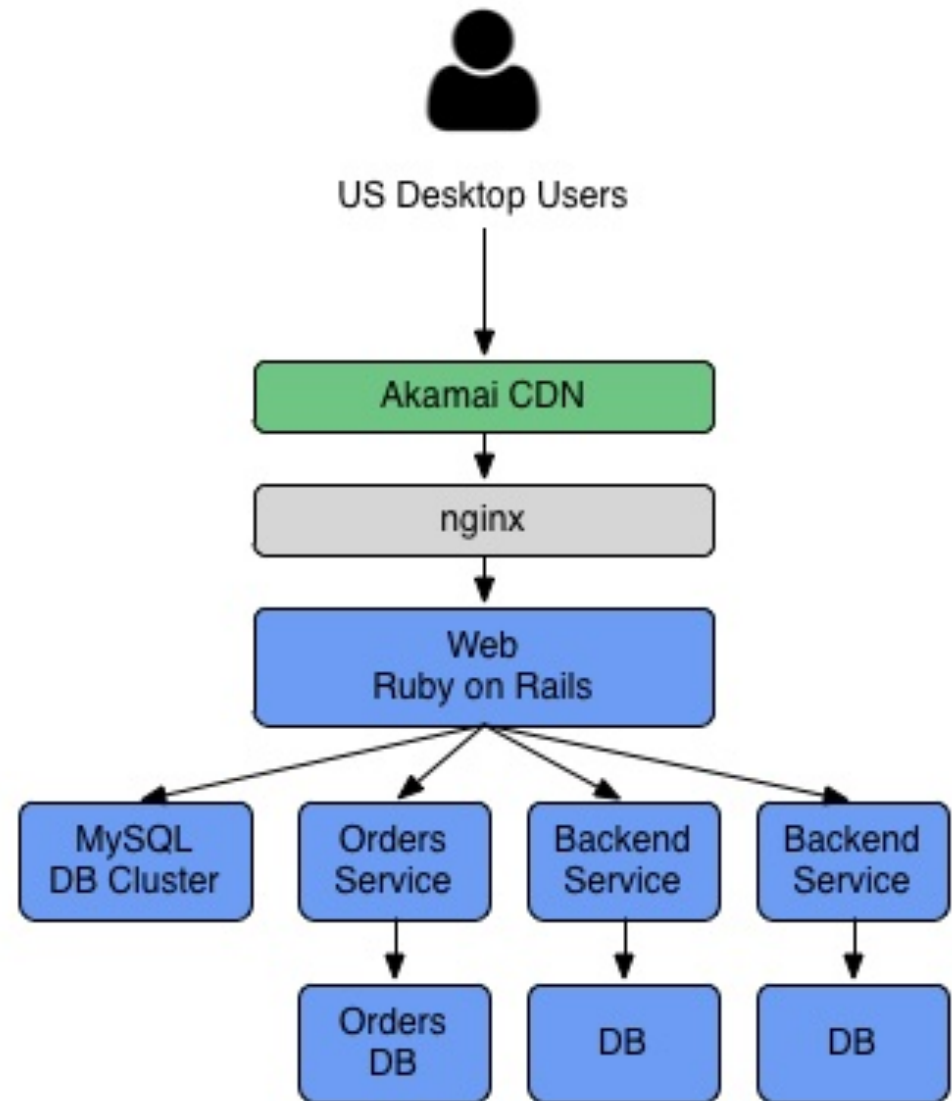


de la o arhitectură monolitică
la una adoptând (micro-)servicii/API-uri REST

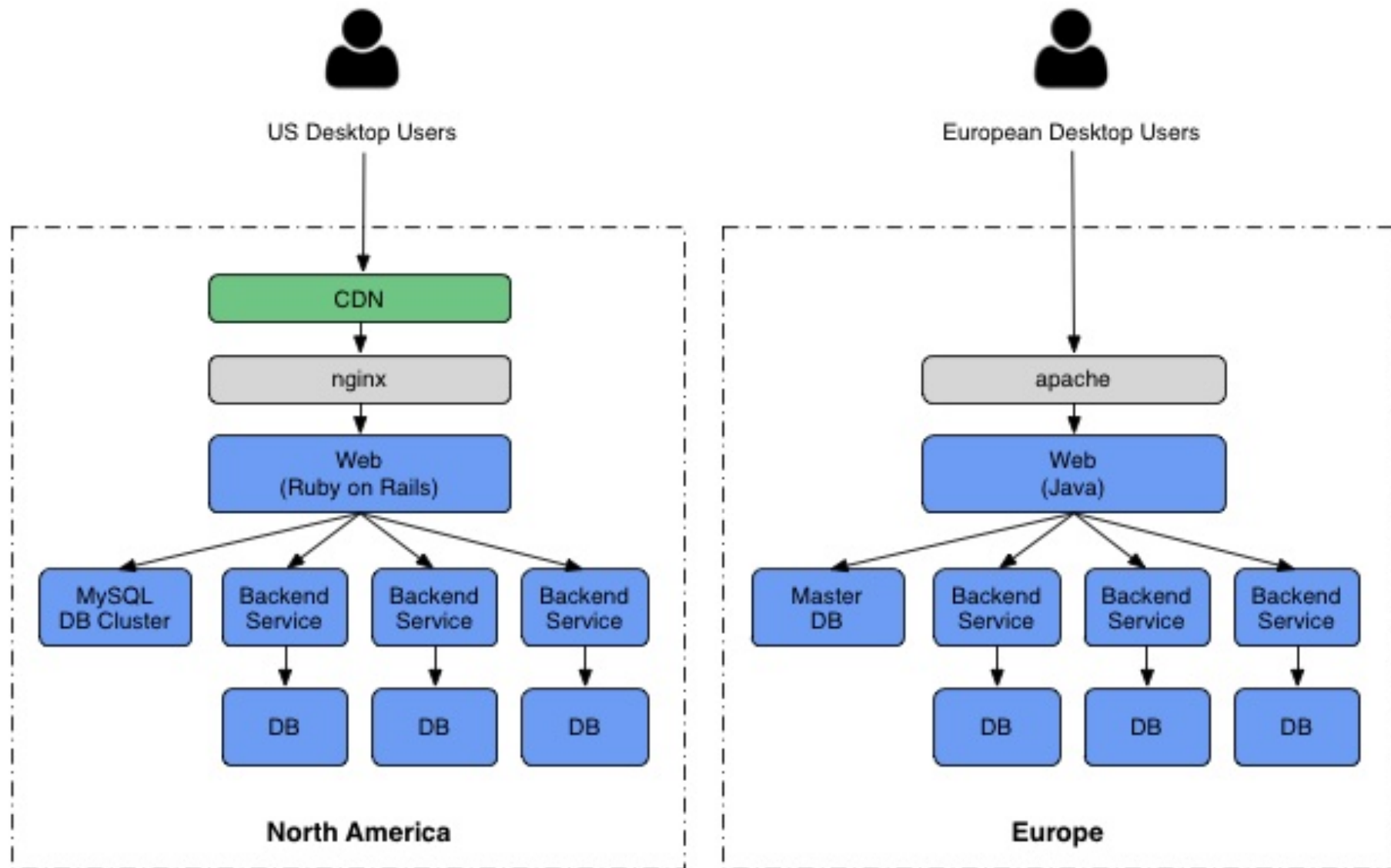
www.groupon.com/developers



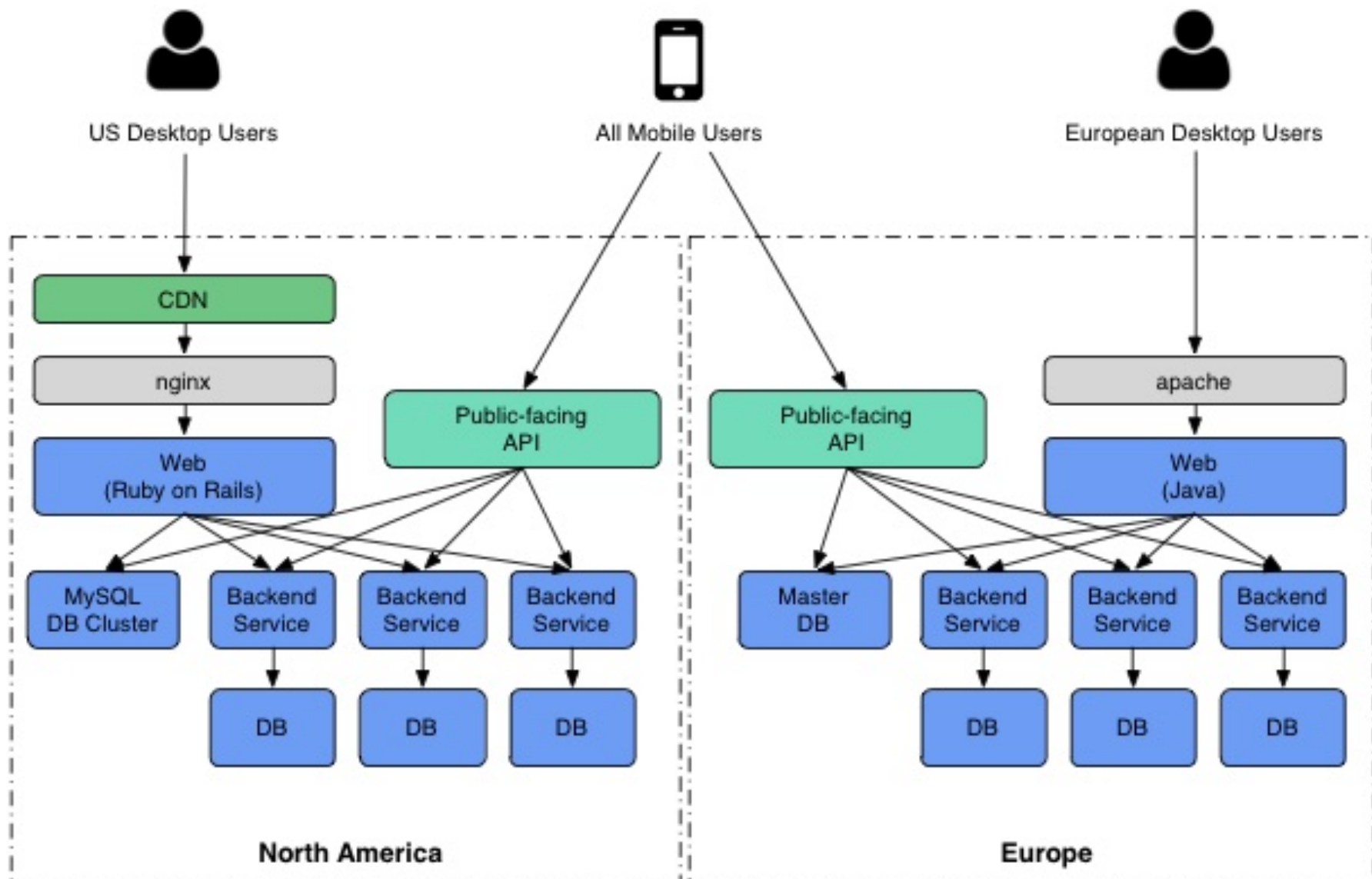
arhitectură inițială
MVC tradițional



asigurarea performanței
scalabilitate cu CDN
(*Content Distribution Network*)
și servicii de acces la date

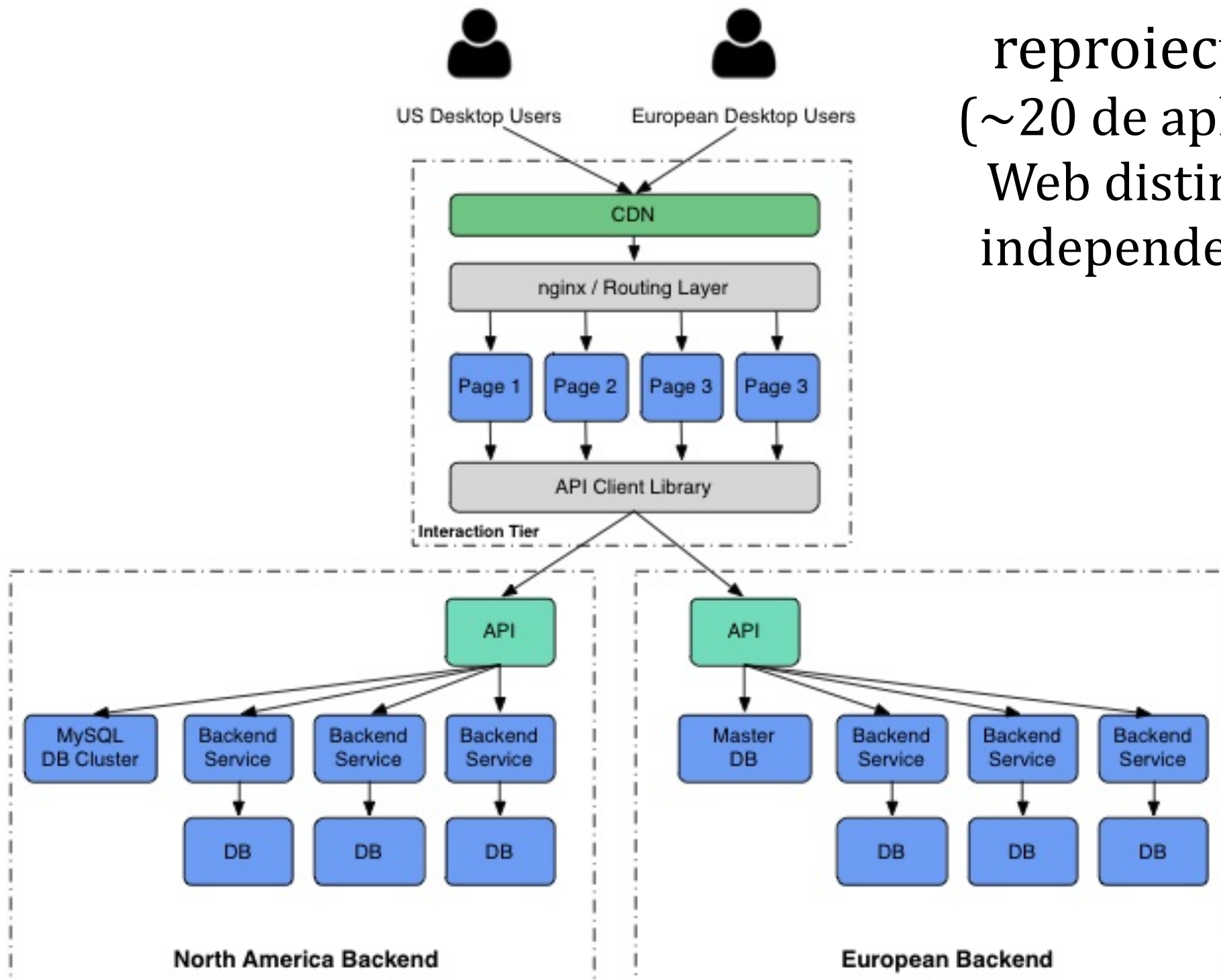


arhitectură eterogenă
implementări distincte în funcție de zona geografică



oferirea suportului
pentru interacțiuni cu dispozitive mobile
API dedicat, accesul depinzând de localizarea utilizatorului

reproiectare
(~20 de aplicații
Web distincte,
independente)



studiu de caz: netflix

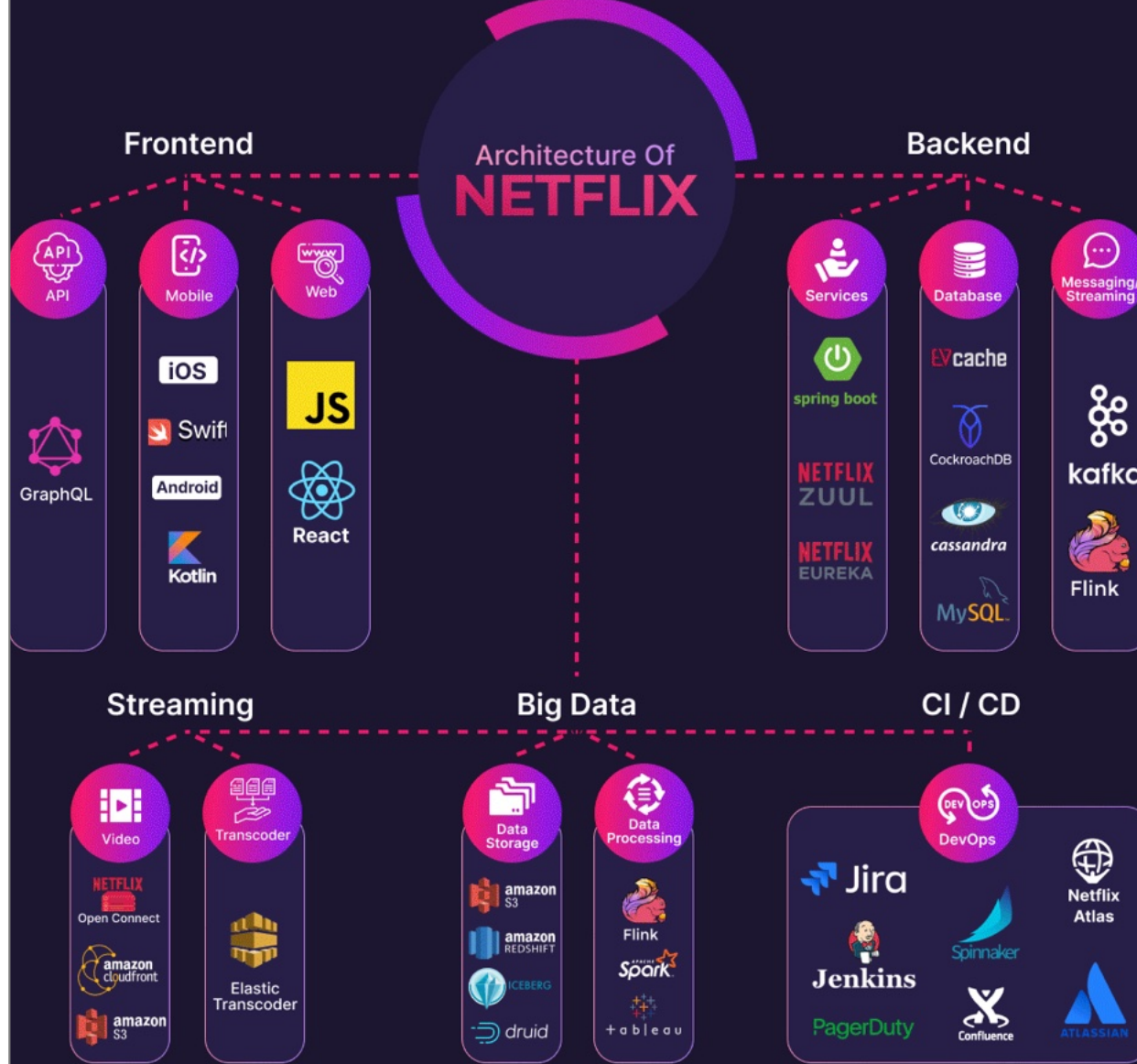
Scop: oferire de conținut video la cerere (*streaming*)
+ televiziune Web (Web TV) 🎭 🎞️ 📺 🎬

servicii disponibile pe dispozitive/platforme multiple

exploatare „în nori”

recurge și la tehnologii deschise

netflixtechblog.com



aspecte de interes:

conținut – filme, emisiuni TV,...

stocare + (de)codificare (*transcoding*)

difuzare adaptivă (*adaptive streaming*)

redare (*playback*) – suport pentru dispozitive eterogene

procesare <i>backend</i>	Flask (Python), Java, Node.js
procesare <i>frontend</i>	React, WinJS (JavaScript)
sisteme de stocare	MySQL, PostgreSQL, Oracle, Atlas-DB, Cassandra, Hadoop, Presto etc.
servicii în „nori”	Amazon EC2 (procesare video) Amazon S3 (stocare)
servicii SQL	Amazon RDS (<i>Relational DB Service</i>)
servicii NoSQL	Amazon DynamoDB
management de cod	GitHub (implementat în Ruby + C)
integrare continuă	Jenkins (implementare Java)
gestionare servere	Apache Mesos (implementare C++)
distribuire de conținut (<i>content distribution network</i>)	Open Connect CDN (FreeBSD, Nginx), Akamai, Level 3, Limelight
monitorizare	Boundary, LogicMonitor, Vector,...

highscalability.com/a-360-degree-view-of-the-entire-netflix-stack/
stackshare.io/netflix/netflix

studiu de caz: wikipedia

Scop: oferirea de conținut deschis    
via o suită de aplicații Web colaborative – *wiki-uri*

Wikipedia Foundation

menține Wikipedia, Wiktionary, Wikibooks, Wikiquote,
Wikivoyage, Wikisource, Wikimedia Commons,
Wikispecies, Wikinews, Wikiversity, Wikidata
en.wikipedia.org/wiki/Wikimedia_Foundation

studiu de caz: wikipedia

Versiune complexă a stivei de tehnologii LAMP

MediaWiki – sistemul *wiki* utilizat pentru toate serviciile implementat în PHP (versiuni 8.1+) și JavaScript

MariaDB (soluția principală de stocare – din 2013)

ImageMagick, DjVu, TeX, rsvg, ploticus etc.

(pentru procesare de conținuturi grafice în MediaWiki)

Apache HTTP Server + Nginx (servere Web)

se oferă inclusiv un API destinat dezvoltatorilor Web:

www.mediawiki.org/wiki/API:Main_page

studiu de caz: wikipedia

PHP-FPM (*FastCGI Process Manager for PHP*)

Apache Traffic Server + Varnish (*proxy + caching*)

Memcached (*caching* interogări asupra bazelor de date)

Elasticsearch (căutare textuală – implementare Java)

gdnssd (soluție C++ pentru DNS)

Apache Kafka (*streaming* de evenimente – Java + Scala)

Linux Virtual Server (*load balancing*); PyBal (monit. locală)

Debian GNU/Linux (sistem de operare)

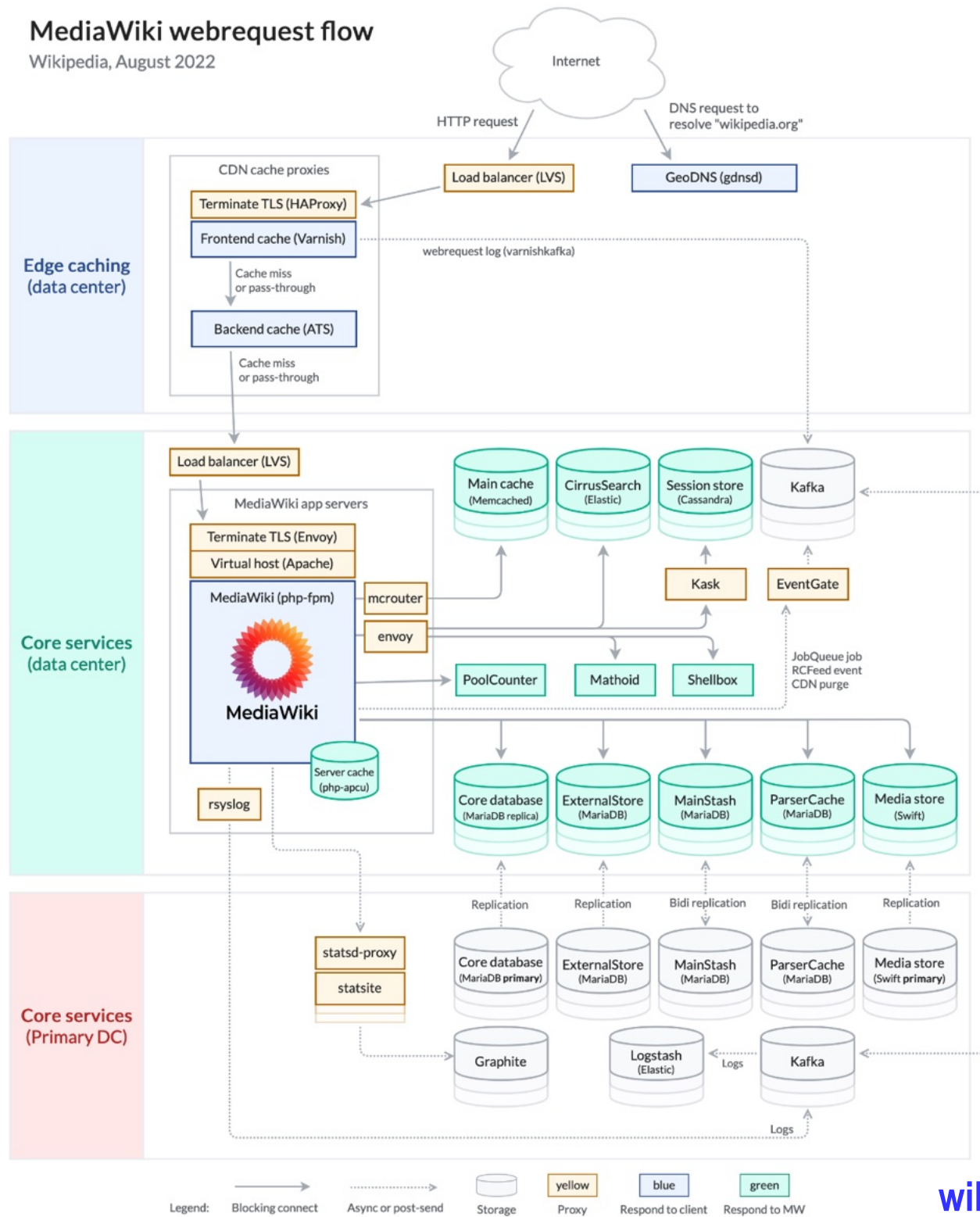
Icinga + Grafana (monitorizarea stării sistemelor)

Phabricator (raportarea deficiențelor – *bug tracking*)

meta.wikimedia.org/wiki/Wikimedia_servers

MediaWiki webrequest flow

Wikipedia, August 2022



găzduire principală:
2 main datacenters
(Ashburn, Dallas)

găzduire secundară:
2 networking sites
(Dallas, Chicago)

replicare:
5 caching sites
(San Francisco,
Amsterdam, Singapore,
Marseille, São Paulo)

Aplicație Web	Procesare la nivel de server (<i>backend</i>)	Stocare persistentă
Amazon	Perl, Java	MySQL (MariaDB), Amazon DynamoDB, Amazon SimpleDB, Amazon ElastiCache
Coursera	Django (Python), Node.js (JavaScript), Play (Scala)	MySQL, Apache Cassandra
DuckDuck Go	Node.js	PostgreSQL
Facebook	Hack, PHP (HHVM), Tornado (Python), Java, JavaScript	RocksDB, Presto, Cassandra, Beringei
Forbes	ASP.NET, Node.js, PHP 7, Ruby	MySQL (MariaDB)
Google	C++, Dart, Go, Java, Python	BigTable, MariaDB

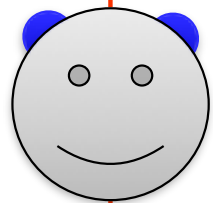
Aplicație Web	Procesare la nivel de server (<i>backend</i>)	Stocare persistentă
Linkedin	Grails (Java), JavaScript, Scala	MySQL, Oracle DB, RocksDB, Hadoop
Lyft	PHP, Flask (Python), Java, Go, C++	MongoDB, DynamoDB, Redis
Medium	Node.js, Go	Neo4j, DynamoDB, Redis
Pinterest	Django (Python), Java, Go	MySQL, Hadoop, Apache HBase, Redis, Memcached
Stack Overflow	.NET Framework (C#)	MS SQL Server, Redis
Sound Cloud	Clojure, Scala, JRuby	MySQL

Development as a Service

DigitalOcean, Google Cloud Platform, Heroku,
Jelastic, OpenStack, Windows Azure,...



mediu de execuție
(*runtime environment*)



rulare

dezvoltare

cod-sursă stocat
(*code repositories*)

instrumente de
dezvoltare (IDE)

BitBucket
GitHub

Web: GitHub Codespaces, Ona, Paiza, Replit, ...
desktop: Eclipse, Visual Studio (Code) etc.

instrumente utile la free-for.dev

studiu de caz

Evaluarea performanței unei aplicații Web
recurgând la arhitecturi diferite:

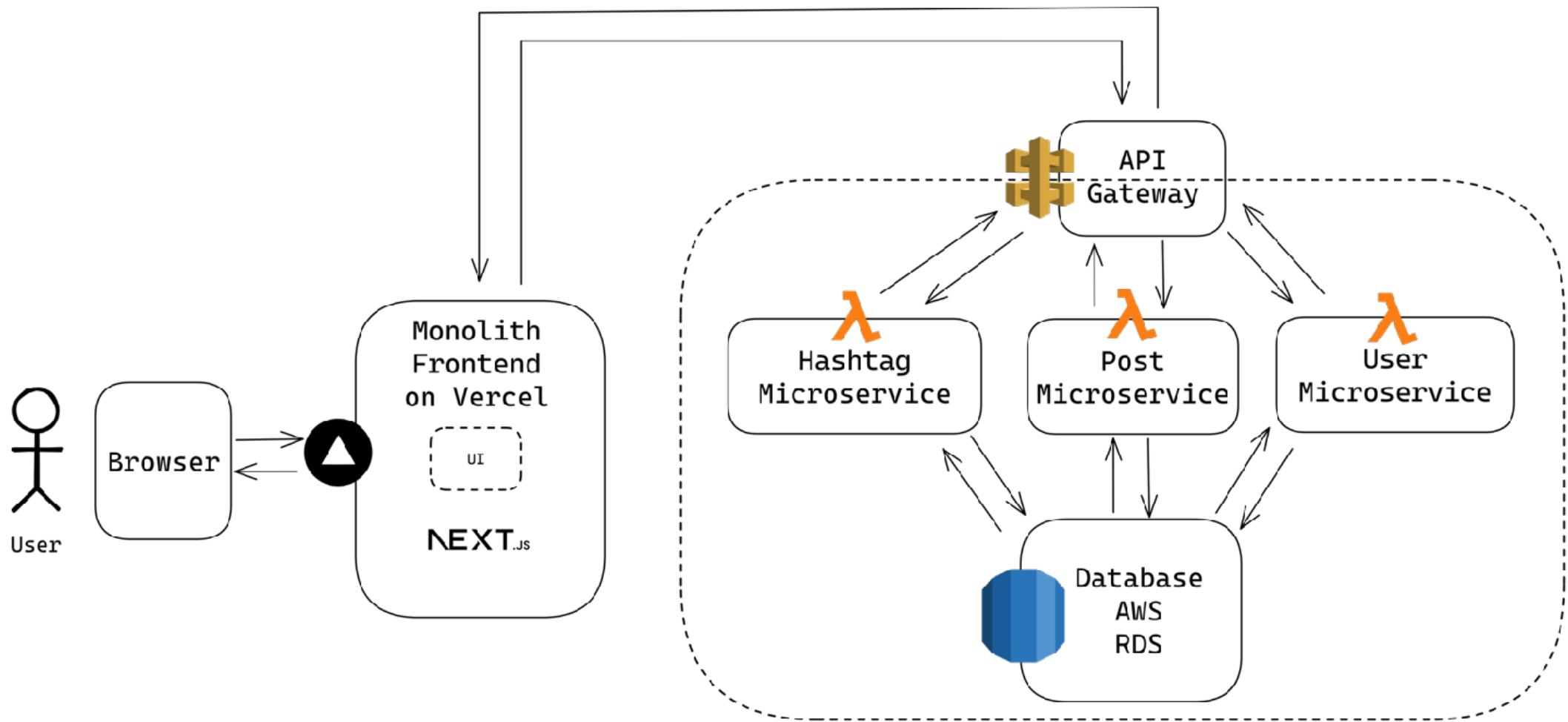
monolith FE + monolith BE

monolith FE + microservices

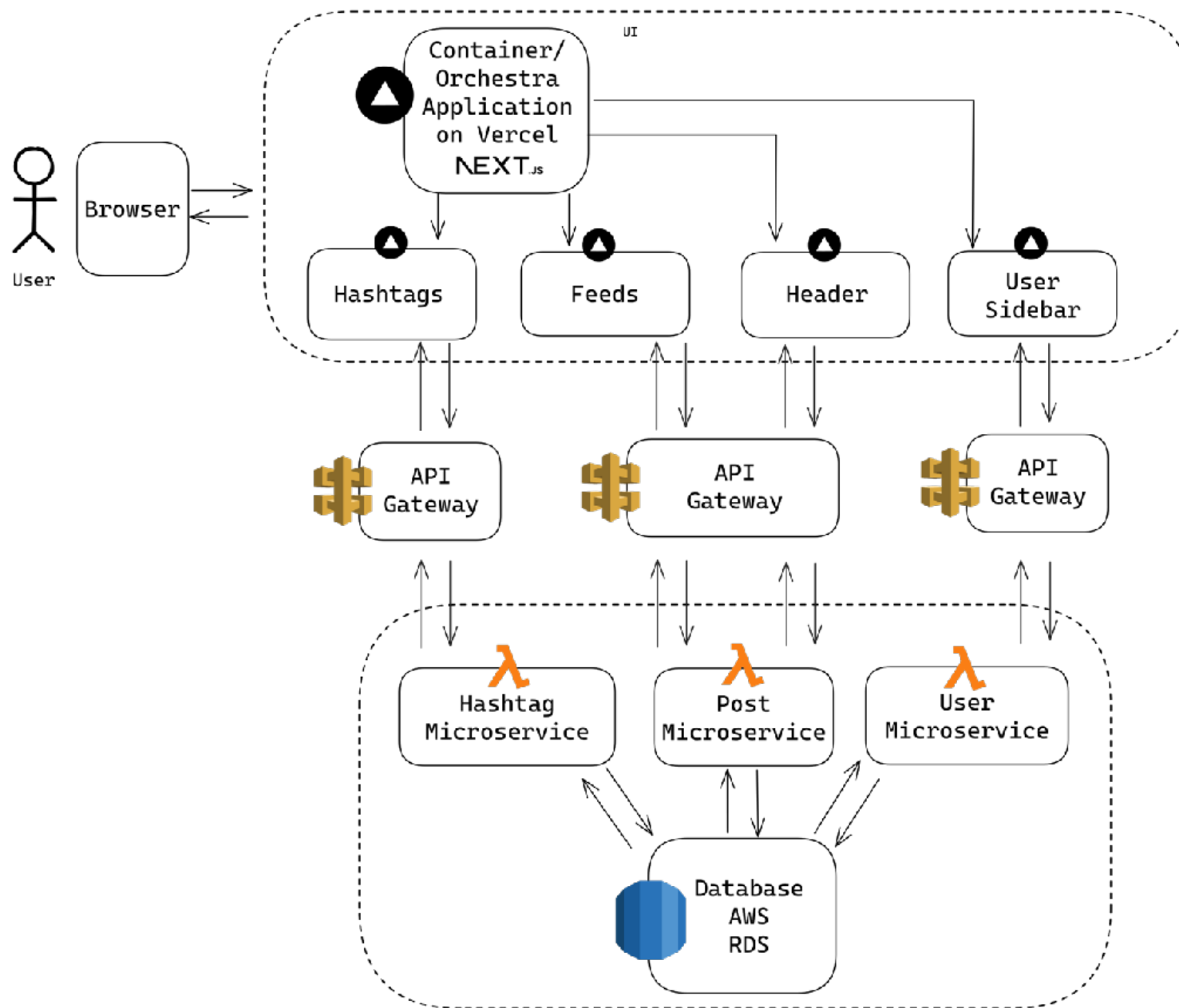
microfrontends + monolith BE

microfrontends + microservices

teză de disertație: Serghei Cunev (2023)



arhitectura aplicației Web cu *frontend* clasic (monolitic) + *backend* compus din micro-servicii implementate via FaaS



arhitectura *backend for frontend* a unei aplicații Web folosind *microservices* + *microfrontends*

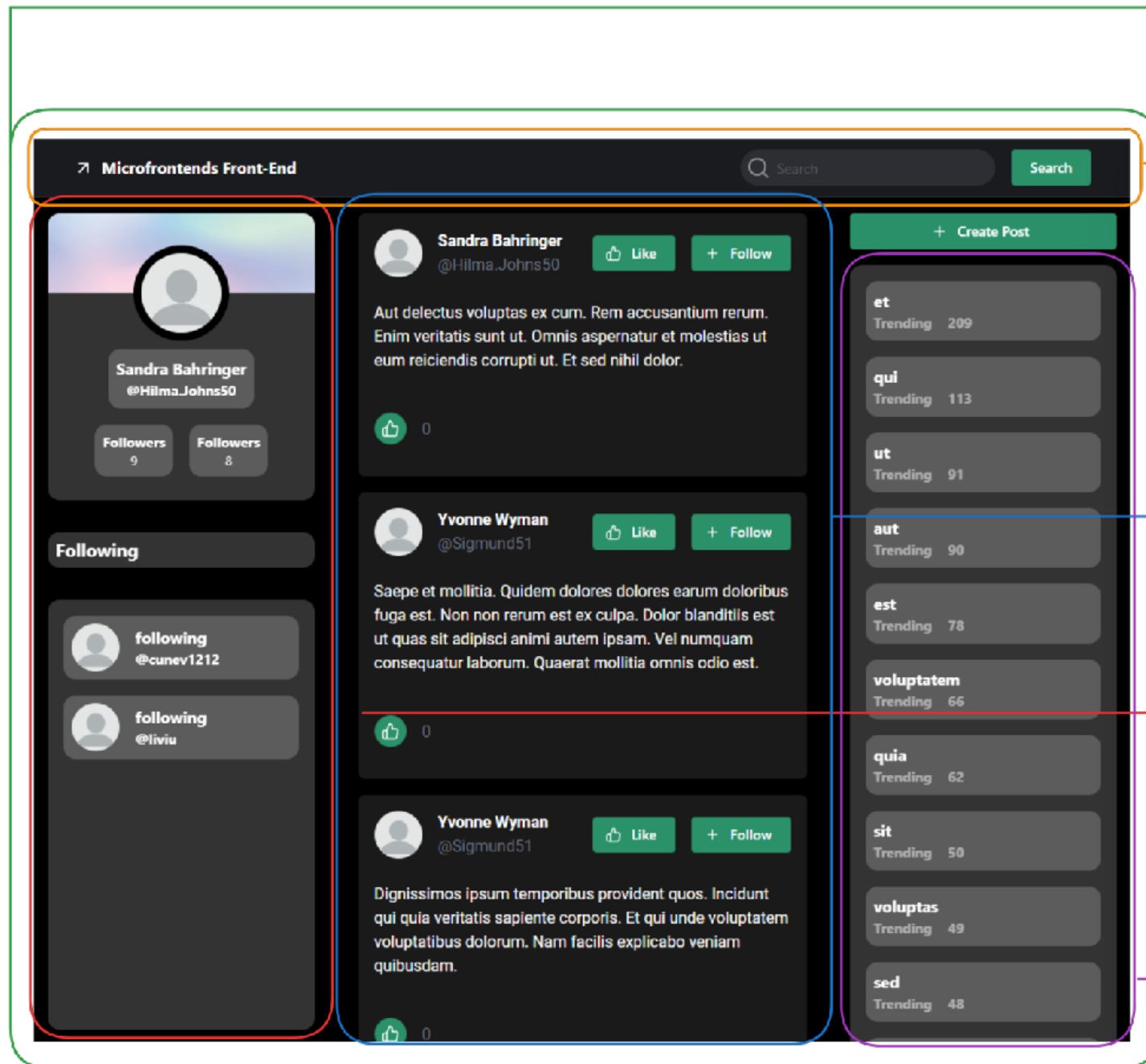
Orchestra
Application

Header
Microfrontend

Feeds
Microfrontend

User Sidebar
Microfrontend

Hashtags
Microfrontend



studiu de caz

Metrici considerate:

Time to First Byte (TTFB)

Document Object Model (DOM) Time

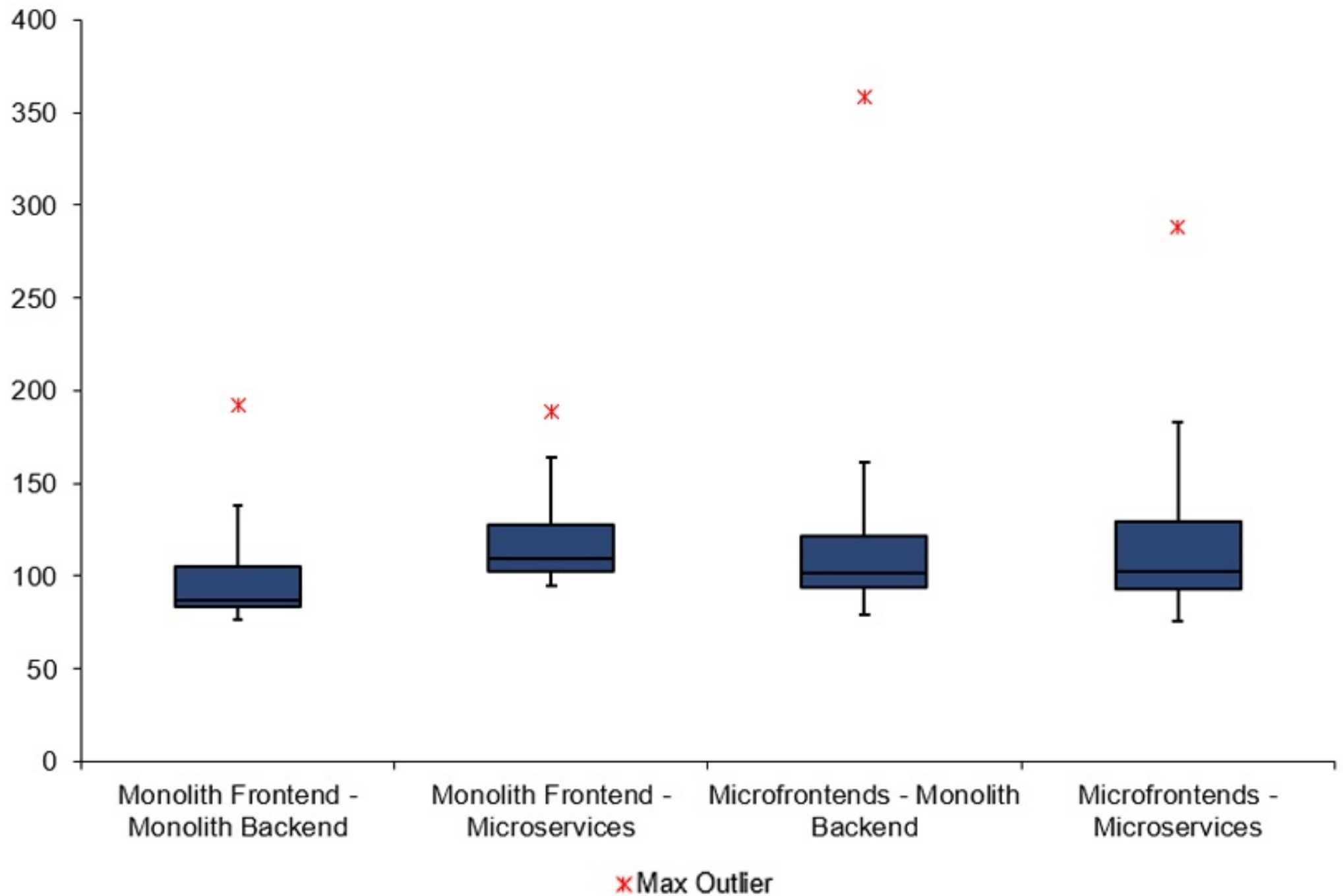
Load Time

Total Blocking Time (TBT)

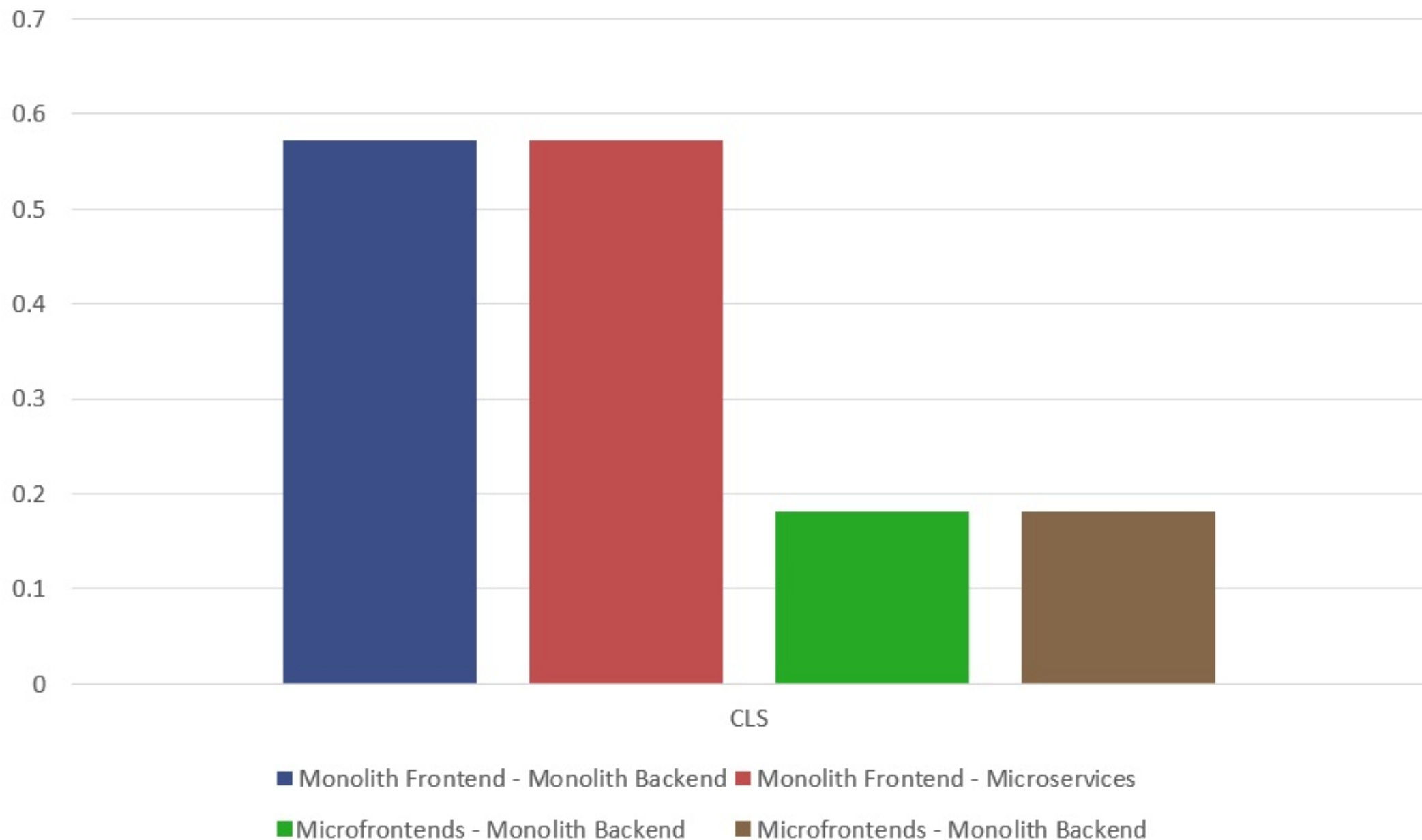
Cumulative Layout Shift (CLS)

Page Size

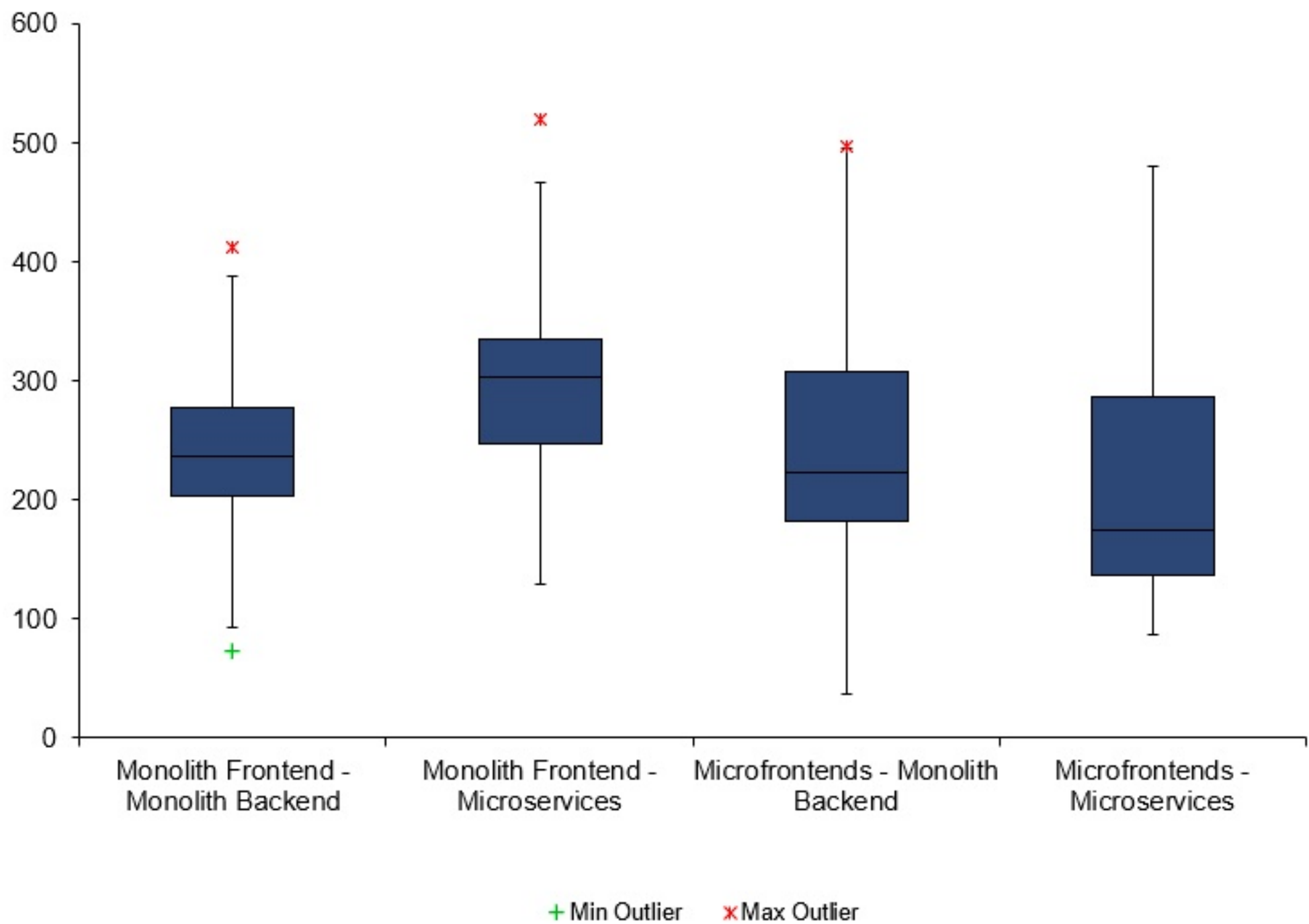
Average Response Time



vizualizarea rezultatelor măsurate: *Load Time* (ms)



măsurarea schimbării *layout*-ului interfeței Web
Cumulative Layout Shift



timpul mediu de răspuns (*average response time* în ms)

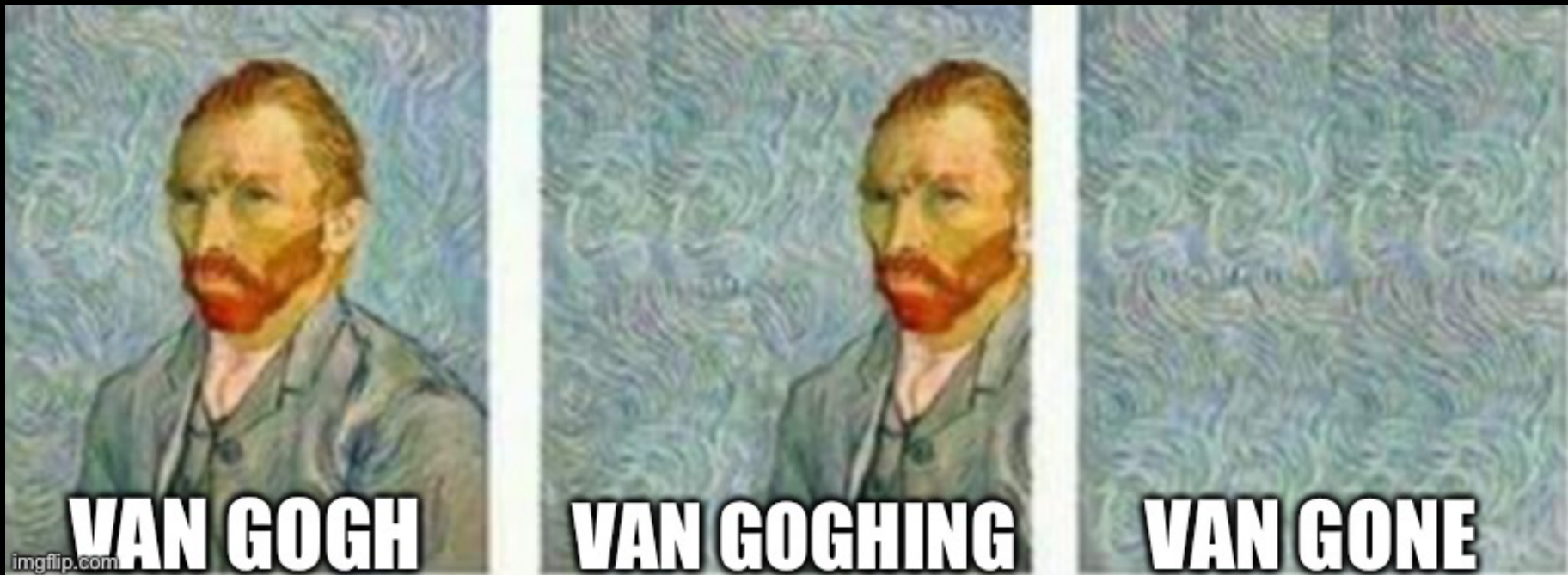
studiu de caz

Concluzii:

“While micro-services and micro-frontends introduce benefits such as modularity and scalability, our findings suggest they also introduce overheads that can negatively impact user experience metrics like TTFB, DOM Time, and Load Time.”

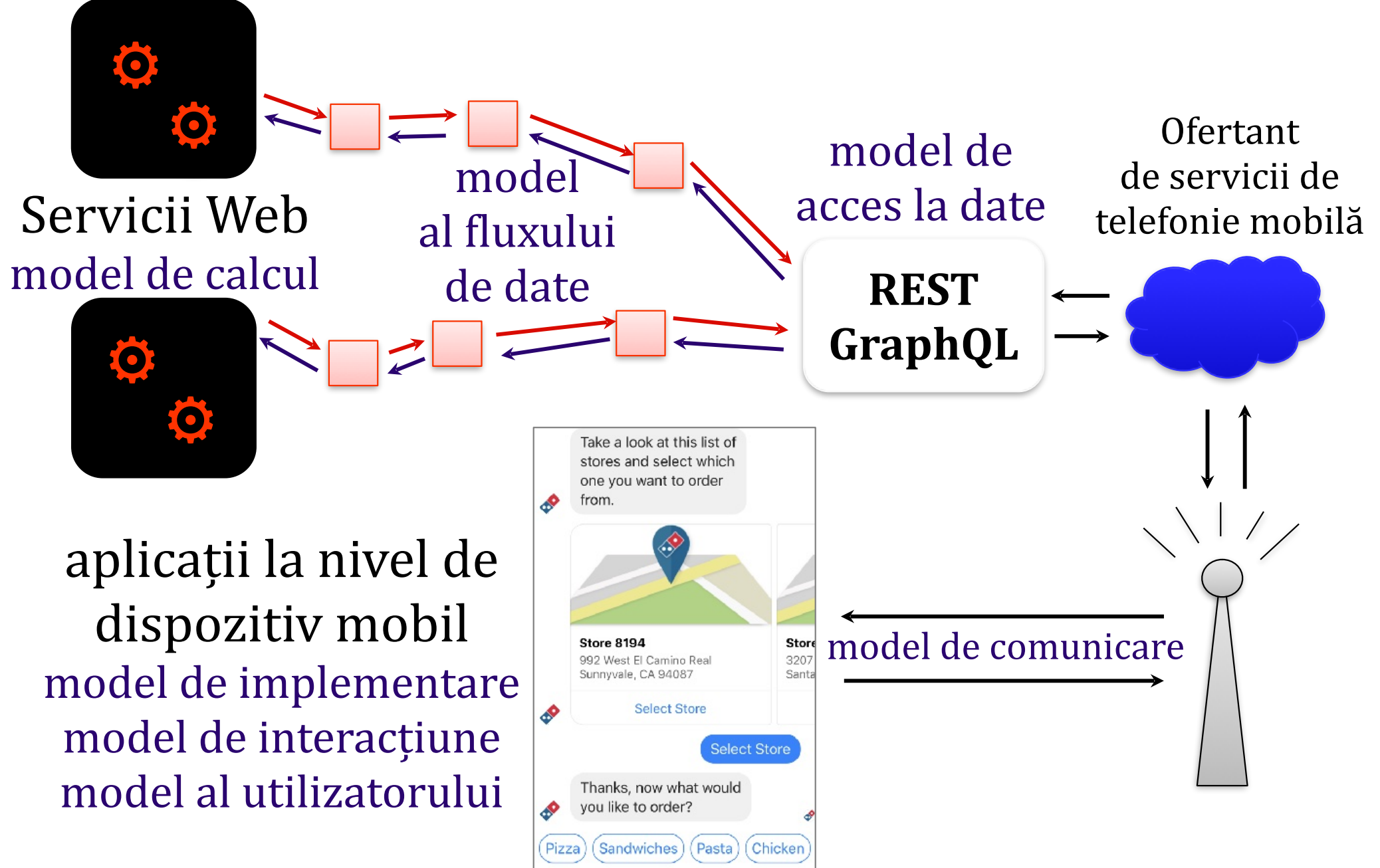
Serghei Cunev (2023)

rezumat



aplicații Web bazate pe (micro-)servicii & API-uri

aspecte vizând arhitectura sistemelor Web



episodul viitor:
aspecte vizând modelarea cunoștințelor